UNIVERSITAS GUNADARMA

FAKULTAS TEKNOLOGI INDUSTRI

IMPLEMENTASI DAN ANALISIS KINERJA *AGENTIC WORKFLOW* BERBASIS GOOGLE GEMINI DAN N8N DALAM OTOMATISASI PRODUKSI ASET VISUAL PADA *PLATFORM* ADOBE STOCK MELALUI *BOT* TELEGRAM

Disusun Oleh:

Nama	:	Alifa Novelia Sukma
NPM	:	50422171
Program Studi	:	Informatika
Pembimbing	:	Dr. Fenni Agustina, S.Kom., MMSI., M.I.Kom.

Diajukan Guna Melengkapi Sebagian Syarat

Dalam Mencapai Gelar Sarjana Strata Satu (S1)

JAKARTA

2026

PERNYATAAN ORIGINALITAS DAN PUBLIKASI

Nama	:	Alifa Novelia Sukma
NPM	:	50422171
Judul Skripsi	:	IMPLEMENTASI DAN ANALISIS KINERJA *AGENTIC WORKFLOW* BERBASIS GOOGLE GEMINI DAN N8N DALAM OTOMATISASI PRODUKSI ASET VISUAL PADA *PLATFORM* ADOBE STOCK MELALUI *BOT* TELEGRAM
Tanggal Sidang	:	20 Agustus 2026
Tanggal Lulus	:	20 Agustus 2026

Dengan ini menyatakan tulisan ini merupakan hasil karya saya sendiri dan dapat dipublikasikan sepenuhnya oleh Universitas Gunadarma. Segala kutipan dalam bentuk apa pun telah mengikuti kaidah dan etika yang berlaku. Mengenai isi dan tulisan, merupakan tanggung jawab penulis sepenuhnya, bukan Universitas Gunadarma.

Demikian, pernyataan ini saya buat dengan sebenar-benarnya dan dengan penuh kesadaran.

Jakarta, 11 Agustus 2026



(Alifa Novelia Sukma)

LEMBAR PENGESAHAN

Judul Skripsi	:	IMPLEMENTASI DAN ANALISIS KINERJA *AGENTIC WORKFLOW* BERBASIS GOOGLE GEMINI DAN N8N DALAM OTOMATISASI PRODUKSI ASET VISUAL PADA *PLATFORM* ADOBE STOCK MELALUI *BOT* TELEGRAM
Nama Mahasiswa	:	Alifa Novelia Sukma
NPM	:	50422171
Tanggal Lulus	:	20 Agustus 2026

PANITIA UJIAN

NO	NAMA	KEDUDUKAN
1	Dr. Ravi Ahmad Salim	Ketua
2	Prof. Dr. Wahyudi Priyono	Sekretaris
3	Dr. Fenni Agustina, S.Kom., MMSI., M.I.Kom.	Anggota
4	Dr. Irwan Bastian, SKom., MM.	Anggota
5	Dr. Metty Mustikasari, SKom., MSc.	Anggota

Menyetujui,

Pembimbing **Bagian Sidang Ujian**

(Dr. Fenni Agustina, S.Kom., MMSI., M.I.Kom.) (Dr. Edi Sukirman, SSi., MM., M.I.Kom.)

ABSTRAK

Alifa Novelia Sukma. 50422171.
IMPLEMENTASI DAN ANALISIS KINERJA *AGENTIC WORKFLOW* BERBASIS GOOGLE GEMINI DAN N8N DALAM OTOMATISASI PRODUKSI ASET VISUAL PADA *PLATFORM* ADOBE STOCK MELALUI *BOT* TELEGRAM.
Skripsi. Program Studi Informatika, Fakultas Teknologi Industri, Universitas Gunadarma, 2026.
Kata Kunci: *Agentic* AI, AI *Agent,* Google Gemini*,* n8n, Otomatisasi, *Adobe Stock, Bot* Telegram.

(xv + 158 + Lampiran)

Kebutuhan efisiensi dalam produksi aset visual *digital* pada *platform microstock,* seperti *Adobe Stock* terus meningkat seiring berkembangnya teknologi kecerdasan buatan. Proses pengerjaan secara manual yang mencakup riset tren, pembuatan gambar, hingga penyusunan *metadata* memerlukan waktu dan sumber daya yang signifikan. Penelitian ini bertujuan untuk merancang dan mengimplementasikan *agentic workflow* berbasis *cloud infrastructure* tanpa ketergantungan pada spesifikasi perangkat keras lokal. Sistem dibangun menggunakan *platform* otomatisasi n8n dengan Google Gemini 2.5 *Flash Lite* sebagai AI *Agent* untuk menganalisis tren pasar *microstock,* serta menyusun *metadata* gambar secara otomatis. Generasi gambar dilakukan menggunakan model FLUX 1.1 Pro melalui API *Replicate*, yang kemudian ditingkatkan kualitasnya menggunakan model *Real-ESRGAN* untuk proses *upscaling* ke resolusi 2K (2048×2048 piksel), serta Supabase *storage* sebagai media penyimpanan *cloud*. Seluruh alur kerja ini dikendalikan melalui skenario perintah tunggal (*single-trigger*) pada *Bot* Telegram yang dilengkapi fitur *error handling*. Hasil pengujian *black box testing* menunjukkan seluruh fitur berfungsi sesuai skenario (100% *pass*). Evaluasi kinerja mencatatkan rata-rata waktu pemrosesan sebesar 53,68 detik per siklus produksi dengan *throughput* sebesar 67,06 proses per jam. Implementasi sistem ini berhasil memangkas durasi produksi hingga menghasilkan efisiensi waktu sebesar 89,89% serta mereduksi beban kerja operasional (*effort*) pengguna sebesar 90,91% dibandingkan proses manual. Dengan demikian, sistem yang dikembangkan terbukti efektif dan efisien dalam mengotomatisasi rantai produksi aset visual bagi kontributor *microstock*.

Daftar Pustaka (2017-2026)

ABSTRACT

Alifa Novelia Sukma. 50422171.
IMPLEMENTATION AND PERFORMANCE ANALYSIS OF A GOOGLE GEMINI AND N8N-BASED AGENTIC WORKFLOW IN AUTOMATING VISUAL ASSET PRODUCTION ON ADOBE STOCK PLATFORM VIA TELEGRAM BOT.
Undergraduate Thesis. Faculty of Industrial Technology, Informatics Study Program, Gunadarma University, 2026.
Keywords: *Agentic* Workflow, AI Agent, Google Gemini, n8n, Automation, Adobe Stock, Telegram Bot.

(xv + 158 + Attachment)

The need for efficiency in producing digital visual assets on microstock platforms such as Adobe Stock continues to rise alongside advancements in artificial intelligence technology. The manual production workflow encompassing market trend research, image generation, and metadata compilation requires significant time and resources. This study aims to design and implement a cloud infrastructure based agentic workflow that operates independently of local hardware specifications. The system was built using the n8n automation platform, utilizing Google Gemini as an AI Agent to analyze microstock market trends and automatically generate image metadata. Image generation was executed via the FLUX 1.1 Pro model through the Replicate API, with visual quality enhanced to 2K resolution (2048×2048 pixels) using Real ESRGAN for upscaling, while Supabase Storage served as the cloud storage media. The entire workflow was managed through single trigger message commands on a Telegram Bot equipped with error handling capabilities. Black Box Testing results demonstrated that all features functioned as expected, achieving a 100 percent pass rate. Performance evaluation recorded an average execution latency of 53.68 seconds per production cycle with a throughput of 67.06 processes per hour. The implementation of this system successfully reduced production duration, achieving a time efficiency of 89.89 percent and reducing operational effort for users by 90.91 percent compared to manual processes. Thus, the developed system is proven to be effective and efficient in automating the visual asset production chain for microstock contributors.

References (2017-2026)

KATA PENGANTAR

Dengan mengucapkan segala puji dan syukur atas kehadirat Allah SWT yang telah memberikan rahmat dan karunia-Nya, sehingga penulis dapat menyelesaikan Tugas Akhir yang berjudul "IMPLEMENTASI DAN ANALISIS KINERJA *AGENTIC WORKFLOW* BERBASIS GOOGLE GEMINI DAN N8N DALAM OTOMATISASI PRODUKSI ASET VISUAL PADA *PLATFORM* ADOBE STOCK MELALUI *BOT* TELEGRAM". Adapun tujuan dari penulisan Tugas Akhir ini adalah sebagai salah satu syarat untuk menyelesaikan Program Sarjana (S1) Jurusan Informatika, Universitas Gunadarma. Dalam kesempatan ini penulis ingin menyampaikan ucapan terima kasih kepada seluruh pihak yang telah memberikan dukungan, bantuan, serta bimbingan sehingga penulisan Tugas Akhir ini dapat diselesaikan dengan baik. Penulis mengucapkan terima kasih kepada:

1. Prof. Dr. E. S. Margianti, SE., MM., selaku Rektor Universitas Gunadarma.
2. Prof. Dr. Ing. Adang Suhendra, S.Si., SKom., MSc., selaku Dekan Fakultas Teknologi Industri Universitas Gunadarma.
3. Prof. Dr. Lintang Yuniar Banowosari, SKom., MSc., selaku Ketua Program Studi Informatika Fakultas Teknologi Industri Universitas Gunadarma.
4. Dr. Edi Sukirman, S.Si., MM., M.I.Kom., selaku Kepala Bagian Sidang Ujian Universitas Gunadarma.
5. Dr. Fenni Agustina, SKom., MMSI., M.I.Kom. Selaku Dosen Pembimbing Tugas Akhir yang ditengah-tengah kesibukan aktivitasnya telah meluangkan waktu dan memberikan bimbingan, pengarahan, serta ilmunya, sehingga Tugas Akhir ini dapat terselesaikan tepat waktu.
6. Kepada Bapak Asep Sukmana dan Ibu Riva Hadijah, yang selalu memberikan doa, dukungan, dan semangat kepada penulis selama proses penyusunan Tugas Akhir ini.
7. Seluruh rekan seperjuangan di Universitas Gunadarma yang telah memberikan bantuan, dukungan, dan semangat kepada penulis.
8. Semua pihak yang tidak dapat disebutkan satu per satu yang telah membantu

dalam proses penyusunan Tugas Akhir ini.

Karena keterbatasan ilmu dan pengalaman yang dimiliki, penulis mengharapkan kritik dan saran yang membangun dari pembaca tulisan ini. Penulis juga berharap penulisan ini dapat bermanfaat dan menambah wawasan pembaca dan pembaca mampu menghasilkan inovasi-inovasi yang baru dari penulisan ini.

Jakarta, 11 Agustus 2026.



(Alifa Novelia Sukma)

DAFTAR ISI

HALAMAN JUDUL ..... **i**
PERNYATAAN ORIGINALITAS DAN PUBLIKASI ..... **ii**
LEMBAR PENGESAHAN ..... **iii**
ABSTRAK ..... **iv**
ABSTRACT ..... **v**
KATA PENGANTAR ..... **vi**
DAFTAR ISI ..... **viii**
DAFTAR TABEL ..... **xii**
DAFTAR GAMBAR ..... **xiii**
DAFTAR LAMPIRAN ..... **xiv**
1. PENDAHULUAN **1**
1.1 Latar Belakang 1
1.2 Rumusan Masalah 4
1.3 Ruang Lingkup 4
1.4 Tujuan Penelitian 5
1.5 Sistematika Penulisan 6
2. TINJAUAN PUSTAKA **8**
2.1 Kecerdasan Buatan dan Otomatisasi 8
2.1.1 *Artificial Intelligence* (AI) 8
2.1.2 *Agentic* AI 9
2.1.3 *Agentic Workflow* 10
2.1.4 *Generative Artificial Intelligence* 12
2.1.5 *Large Language Model* (LLM) 12
2.1.6 *Low-Code Platform (*n8n*)* 13
2.2 Model dan Layanan Kecerdasan Buatan 15
2.2.1 Google Gemini 15
2.2.2 OpenAI 16
2.2.3 OpenRouter 17
2.2.4 *Platform* Replicate (FLUX 1.1 Pro Model dan Real-ESRGAN) 18
2.2.5 Peningkatan Resolusi Citra Berbasis AI 19
2.2.6 *Platform* Aivene 20

2.3 Arsitektur *Server* dan *Infrastructure Deployment* 21
2.3.1 Arsitektur *Cloud* dan *Backend* 21
2.3.2 Kontainerisasi Aplikasi (Docker) 23
2.3.3 *Tunneling* Jaringan Lokal (Ngrok) 24
2.3.4 Supabase (*Storage Bucket*) 26
2.4 Komunikasi Data dan Integrasi Sistem 27
2.4.1 *Application Programming Interface* (API) dan *Webhook* 27
2.4.2 Protokol *Web* 28
2.4.3 *JavaScript* dan Format Data 29
2.4.4 *Structured Output Parser* 30
2.5 *Platform Microstock*, dan Media *Digital* 31
2.5.1 *Microstock* dan *Platform* Adobe Stock 31
2.5.2 *Search Engine Optimization* (SEO) 34
2.5.3 Gambar *Digital* 35
2.5.4 *Metadata* Gambar 35
2.6 Perangkat Pengujian dan Sistem Pembanding 36
2.6.1 VideoProc *Converte*r AI 36
2.6.2 Kling AI 37
2.6.3 Postman 38
2.7 Metode *Prototyping* 39
2.8 Metode Eksperimental 40
2.8.1 *Latency* 41
2.8.2 *Throughput* 42
2.8.3 *Bottleneck* 43
2.8.4 Efisiensi Waktu Produksi 44
2.8.5 Usaha Pengembangan (*Effort*) 45
2.9 *Black Box Testing* 46
2.10 *System Information Leakage* 46
2.11 *Unified Modeling Language* (UML) 47
2.11.1 *Use Case Diagram* 47
2.11.2 *Activity Diagram* 49
2.11.3 *Sequence Diagram* 50
2.12 *Flowchart* 52
2.13 Penelitian Terdahulu 53

3. METODE PENELITIAN.....63
3.1 Alur Penelitian.....63
3.2 Analisis Kebutuhan Sistem.....65
3.2.1 Kebutuhan Fungsional.....66
3.2.2 Kebutuhan Nonfungsional.....67
3.2.3 Kebutuhan Manajemen *Storage*.....69
3.2.4 Kebutuhan Model AI dan Estimasi Biaya.....70
3.3 Perancangan Sistem.....73
3.3.1 *Flowchart*.....73
3.3.2 Perancangan Komponen *Nodes* n8n.....74
3.3.3 *Use Case Diagram*.....80
3.3.4 *Activity Diagram*.....81
3.3.5 *Sequence Diagram*.....86
3.3.6 Perancangan Penanganan Kesalahan.....90
3.4 Pembuatan Prototipe.....92
3.4.1 Proses Iterasi Prototipe.....92
3.5 Uji Coba Sistem.....93
3.5.1 Pengujian Kualitas Luaran.....94
3.5.2 Analisis Kinerja Sistem.....97
3.5.3 Analisis Efisiensi.....99
3.6 Evaluasi Sistem.....102
4. HASIL DAN PEMBAHASAN.....105
4.1 Implementasi Sistem.....105
4.2 Implementasi Fitur *Bot* Telegram.....108
4.2.1 Fitur Menu panduan.....108
4.2.2 Fitur Riset Tren Pasar *Microstock*.....109
4.2.3 Fitur Produksi Aset Visual.....110
4.2.4 *Error* Interupsi Jaringan atau Keterbatasan Akses API.....113
4.2.5 *Error* Kesalahan Format Perintah Pengguna.....114
4.2.6 *Error Filter* Keamanan Gambar.....114
4.3 Implementasi dan Pengujian Layanan Eksternal.....115
4.3.1 Pengujian REST API Menggunakan Postman.....115
4.3.2 Rekapitulasi Hasil Pengujian API.....123
4.4 Hasil Uji Coba Sistem.....124

4.4.1 Hasil Penilaian Kualitas Luaran 125
4.4.2 Hasil Analisis Kinerja Sistem.....135
4.4.3 Hasil Analisis Efisiensi Sistem.....138
4.5 Evaluasi 142
4.5.1 Evaluasi Aspek Etika dan Hak Cipta.....147
4.5.2 Kelemahan dan Kelebihan Gambar AI.....148
5. PENUTUP.....150
5.1 Kesimpulan.....150
5.2 Saran.....151
DAFTAR PUSTAKA 152
LAMPIRAN.....L-1

DAFTAR TABEL

Tabel 2.1 Simbol *Use Case Diagram* 48
Tabel 2.2 Simbol *Activity Diagram* 49
Tabel 2.3 Simbol *Sequence Diagram*.....51
Tabel 2.4 Simbol *Flowchart* 52
Tabel 2.5 Penelitian Terdahulu 53
Tabel 3.1 Kebutuhan Perangkat Keras.....67
Tabel 3.2 Kebutuhan Perangkat Lunak.....67
Tabel 3.3 Spesifikasi *Bucket Penyimpanan* Supabase 70
Tabel 3.4 Kebutuhan Model AI dan Estimasi Biaya.....71
Tabel 3.5 Skenario Penanganan Kesalahan 91
Tabel 3.6 Skenario Pengujian dan Evaluasi Sistem 93
Tabel 3.7 Variabel Evaluasi Kualitas Visual Gambar 95
Tabel 3.8 Variabel Evaluasi Akurasi *Metadata* 96
Tabel 3.9 Parameter Evaluasi Kinerja Waktu Sistem 98
Tabel 3.10 Parameter Perbandingan Efisiensi Manual dan Otomatis.....101
Tabel 3.11 Skenario Pengujian *Black-Box Testing* Sistem 102
Tabel 4.1 Rekapitulasi Hasil Pengujian API dengan Postman 123
Tabel 4.2 Hasil Pengujian Kualitas Visual Gambar.....125
Tabel 4.3 Hasil Evaluasi Relevansi Judul dan Kata Kunci Gambar 130
Tabel 4.4 Hasil Pengujian Latensi Sistem 136
Tabel 4.5 Rincian Aktivitas dan Beban Kerja Operasional Manual 139
Tabel 4.6 Perbandingan Beban Kerja Operasional (*Desktop Environment*).....141
Tabel 4.7 Perbandingan Efisiensi Metode Manual dan Otomatisasi 141
Tabel 4.8 Hasil *Black Box Testing* 142

DAFTAR GAMBAR

Gambar 2.1 Skema Graf *Agentic Workflow* 11
Gambar 2.2 Logo n8n 14
Gambar 2.3 Logo Teknologi Google Gemini 16
Gambar 2.4 Logo Perusahaan Teknologi OpenAI 17
Gambar 2.5 Logo *Platform Cloud* Replicate 19
Gambar 2.6 Struktur Arsitektur *Cloud Computing* 23
Gambar 2.7 Logo *Platform* Kontainerisasi Docker 24
Gambar 2.8 Logo *Platform* Tunneling Ngrok 25
Gambar 2.9 Logo *Platform* Database Supabase 27
Gambar 2.10 Logo *Platform* Adobe Stock 33
Gambar 2.11 Logo *Platform* Pengujian Postman 39
Gambar 2.12 Alur Metode Pengembangan *Prototyping* 40
Gambar 3.1 Alur Metode Prototipe Eksperimental 64
Gambar 3.2 *Flowchart* sistem 73
Gambar 3.3 *Use Case Diagram* 80
Gambar 3.4 *Activity Diagram* Fitur Panduan 82
Gambar 3.5 *Activity Diagram* Fitur Riset Tren 83
Gambar 3.6 *Activity Diagram* Fitur Produksi Aset Visual Gambar 85
Gambar 3.7 *Sequence Diagram* Fitur Panduan 87
Gambar 3.8 *Sequence Diagram* Fitur Riset Tren 88
Gambar 3.9 *Sequence Diagram* Fitur Produksi Aset Gambar 89
Gambar 4.1 Arsitektur *Workflow* Otomatisasi pada n8n 106
Gambar 4.2 Antarmuka Menu panduan *Bot* (Bagian 1) 108
Gambar 4.3 Antarmuka Menu panduan *Bot* (Bagian 2) 108
Gambar 4.4 Antarmuka Menu Riset Tren (Bagian 1) 109
Gambar 4.5 Antarmuka Menu Riset Tren (Bagian 2) 109
Gambar 4.6 Antarmuka Menu Produksi Aset 111
Gambar 4.7 Hasil Gambar 112
Gambar 4.8 Detail Berkas Gambar 112
Gambar 4.9 Respon Kesalahan Interupsi Jaringan 113
Gambar 4.10 Respon Kesalahan Format Perintah 114
Gambar 4.11 Respon Kesalahan Kebijakan Konten 115
Gambar 4.12 Hasil Pengujian API FLUX 1.1 Pro 116
Gambar 4.13 Hasil Pengujian API Real-ESRGAN 117
Gambar 4.14 Hasil Pengujian API Gemini 2.5 Flash Lite (Riset) 118
Gambar 4.15 Hasil Pengujian API Gemini 2.5 Flash Lite (*Metadata*) 119
Gambar 4.16 Hasil Pengujian API Gemini 2.5 Flash Lite (*Structured Output*) 120
Gambar 4.17 Hasil Pengujian API Gemini 2.5 Flash (*Prompt Enhancer*) 121
Gambar 4.18 Hasil Pengujian API Supabase *Bucket Raw-Assets* 122
Gambar 4.19 Hasil Pengujian API Supabase *Bucket Upscaled-Assets* 123

DAFTAR LAMPIRAN

Lampiran 1: Pembuatan *Bot* Telegram L-1
Lampiran 2: Konfigurasi *Container* Docker L-1
Lampiran 3: Konfigurasi *Node* Telegram Trigger L-1
Lampiran 4: Konfigurasi *Node Request Handler* L-2
Lampiran 5: Konfigurasi *Node Feedback* Awal L-2
Lampiran 6: Konfigurasi *Node* Switch Menu *Routing* L-2
Lampiran 7: Konfigurasi *Node Feedback Prompt Start* L-3
Lampiran 8: Konfigurasi *Node Feedback* Deteksi Riset L-3
Lampiran 9: Konfigurasi *Node* AI Agent Riset Tren L-3
Lampiran 10: Konfigurasi *Node* Model *Chat* AI Agent Riset L-4
Lampiran 11: Konfigurasi *Window Buffer Memory* L-4
Lampiran 12: Konfigurasi *Node* Telegram *Feedback* Riset L-4
Lampiran 13: Konfigurasi *Node Feedback* Deteksi Pembuatan Aset L-5
Lampiran 14: Konfigurasi *Node Formatter* L-5
Lampiran 15: Konfigurasi *Node Prompt Enhancer* L-5
Lampiran 16: Konfigurasi *Headers Node Enhancer* L-6
Lampiran 17: Konfigurasi *Body* JSON *Node Enhancer* L-6
Lampiran 18: Konfigurasi *Node* HTTP *Request* Replicate FLUX 1.1 Pro L-6
Lampiran 19: Konfigurasi *Headers* HTTP *Request* L-7
Lampiran 20: Konfigurasi *Node* Cek Status Pelanggaran Konten L-7
Lampiran 21: Konfigurasi *Node Image* URL *Generator* L-7
Lampiran 22: Konfigurasi Options *Node Image* Url *Generator* L-7
Lampiran 23: Konfigurasi *Method* & URL *Node Raw-assets* L-8
Lampiran 24: Konfigurasi *Headers Node Raw-assets* L-8
Lampiran 25: Konfigurasi *Node GET Public* URL Supabase L-8
Lampiran 26: Konfigurasi *Method* dan URL pada *Node Upscale* Gambar L-8
Lampiran 27: Konfigurasi *Send Headers* pada *Node Upscale* L-9
Lampiran 28: Konfigurasi *Send Body* pada *Node Upscale* L-9
Lampiran 29: Konfigurasi *Node Get* Gambar *Upscale* L-9
Lampiran 30: Pengaturan *Convert* Format pada *Node Convert* L-10
Lampiran 31: Konfigurasi *Method* dan URL *Node* Bucket *Upscale*d-*Assets* ... L-10
Lampiran 32: Konfigurasi *Headers Node* Bucket *Upscale*d-*Assets* L-10
Lampiran 33: Konfigurasi *Body Node* Bucket *Upscale*d-*Assets* L-11
Lampiran 34: Konfigurasi *Node Image* URL *Generator* 1 ..... L-11
Lampiran 35: Konfigurasi *Prompt* dan *Output* Format *Metadata* L-11
Lampiran 36: Konfigurasi Model *Node* AI Agent *Metadata* L-11
Lampiran 37: Hasil *Output Keywords* Percobaan ke-1 ..... L-14
Lampiran 38: Hasil *Output Keywords* Percobaan ke-2 ..... L-14
Lampiran 39: Hasil *Output Keywords* Percobaan ke-3 ..... L-14
Lampiran 40: Hasil *Output Keywords* Percobaan ke-4 ..... L-14
Lampiran 41: Hasil *Output Keywords* Percobaan ke-5 ..... L-15
Lampiran 42: Hasil *Output Keywords* Percobaan ke-6 ..... L-15
Lampiran 43: Hasil *Output Keywords* Percobaan ke-7 ..... L-15

Lampiran 44: Hasil *Output Keywords* Percobaan ke-8 ..... L-15
Lampiran 45: Hasil *Output Keywords* Percobaan ke-9 ..... L-16
Lampiran 46: Hasil *Output Keywords* Percobaan ke-10 ..... L-16
Lampiran 47: Hasil *Output Keywords* Percobaan ke-11 ..... L-16
Lampiran 48: Hasil *Output Keywords* Percobaan ke-12 ..... L-16
Lampiran 49: Hasil *Output Keywords* Percobaan ke-13 ..... L-17
Lampiran 50: Hasil *Output Keywords* Percobaan ke-14 ..... L-17
Lampiran 51: Hasil *Output Keywords* Percobaan ke-15 ..... L-17
Lampiran 52: Teks Panduan Penggunaan Bot pada *Node* Telegram *Feedback Start* L-18
Lampiran 53: *Prompt* System AI Agent Riset Tren Pasar L-19
Lampiran 54: *Payload* JSON *Prompt Enhancer* L-20
Lampiran 55: JSON *Send Body* pada *Node Upscale* L-20
Lampiran 56: Hasil Plagiasi (bagian 1) L-21
Lampiran 57: Hasil Plagiasi (bagian 2) L-21
Lampiran 58: Hasil Plagiasi (bagian 3) L-22
Lampiran 59: Hasil Plagiasi (bagian 4) L-22
Lampiran 60: Hasil Plagiasi (Bagian 5) L-23
Lampiran 61: Hasil Plagiasi (Bagian 6) L-23
Lampiran 62: Hasil Plagiasi (Bagian 7) L-24
Lampiran 63: Hasil Plagiasi (Bagian 8) L-24
Lampiran 64: Hasil Plagiasi (Bagian 9) L-25
Lampiran 65: Hasil Plagiasi (Bagian 10) L-25
Lampiran 66: Hasil Plagiasi (Bagian 11) L-26

1. PENDAHULUAN

1.1 Latar Belakang

Aset visual merupakan berbagai bentuk media *digital,* seperti ilustrasi, foto, desain grafis, serta gambar berbasis kecerdasan buatan yang digunakan untuk kebutuhan pemasaran, promosi, media sosial, hingga keperluan komersial lainnya. Secara mendasar, aset visual merupakan segala sumber daya bernilai tukar yang dirancang untuk memberikan kesempurnaan citra, serta identitas kuat bagi perusahaan, yang mencakup infografis, ilustrasi, iklan, kemasan, hingga *corporate identity* (Insani & Patria, 2023). Dalam beberapa tahun terakhir, kebutuhan terhadap aset visual meningkat seiring dengan berkembangnya industri *digital* dan meningkatnya konsumsi konten pada berbagai hal. Dalam kaitannya dengan hal tersebut, sektor industri harus berinovasi untuk dapat memberikan rekomendasi pada setiap permasalahan konsumen, di mana industri konten *digital* merupakan salah satu sektor industri yang dinilai memiliki peran signifikan, baik dalam peningkatan perekonomian maupun dalam membangun keberlanjutan bagi masyarakat (Sugiono, 2020).

Dalam hal ini, pendekatan pemasaran *digital* memungkinkan pelaku usaha untuk merangkul perubahan perilaku konsumen dan menjadikannya peluang untuk merambah pasar yang lebih luas, melampaui batasan-batasan geografis dan kendala waktu yang ada dalam model pemasaran konvensional (Pitaloka & Kardoyo, 2023). Stok grafis tersebut ditujukan kepada orang-orang yang membutuhkan stok grafis secara instan untuk digunakan dalam berbagai proyek, misalnya kampanye, pemasaran, bisnis, maupun kegiatan-kegiatan yang membutuhkan stok grafis sebagai tambahan untuk melengkapinya tanpa perlu melibatkan tenaga kerja tambahan (Jeharu, 2021).

Industri *microstock* merupakan pasar untuk proses jual beli karya *digital* secara *online*, dengan harga murah dan adanya lisensi namun bebas royalti (Insani et al., 2020). *Microstock* berperan dalam memperluas distribusi konten *digital* secara *global* dan meningkatkan peluang monetisasi bagi kreator (Abimanyu & Paramita, 2025). *Microstock* diidentifikasikan sebagai sistem *crowdsourcing* yang

mendapatkan layanan, ide, atau produk dari sejumlah besar *contributor* berbentuk perpaduan antara elemen grafis secara *open source*. Hal tersebut berarti *microstock* berperan sebagai agen pengumpul stok grafis dari berbagai sumber, karena *microstock* tidak memproduksi produknya sendiri (Jeharu, 2021).

Diantara berbagai *platform microstock*, Adobe Stock menjadi salah satu yang mulai membuka peluang bagi kreator untuk mendistribusikan aset visual berbasis kecerdasan buatan secara komersial. Kebijakan tersebut memberi ruang bagi *contributor* untuk menghasilkan dan memasarkan karya *digital* dengan jangkauan yang lebih luas. Namun, meningkatnya peluang tersebut diikuti oleh tingkat persaingan yang semakin tinggi. Dalam konteks persaingan pasar, kualitas produk menjadi kunci perusahaan untuk menonjolkan keunggulan unik produk atau layanannya dibandingkan dengan apa yang ditawarkan oleh pesaing. Hal ini memungkinkan pelanggan untuk melihat atau percaya bahwa produk berkualitas menawarkan nilai tambah yang mereka harapkan (Nurfauzi et al., 2023). Pentingnya kualitas ini sangat terasa dalam industri aset visual, khususnya pada *platform microstock,* seperti Adobe Stock.

Dalam praktiknya, untuk memenuhi standar kualitas tinggi yang ditetapkan oleh Adobe Stock, proses produksi aset visual melalui kecerdasan buatan tidak dapat dikatakan sederhana. Seorang kreator harus melalui beberapa tahapan mulai dari riset tren, pembuatan perintah untuk menghasilkan gambar, proses peningkatan kualitas gambar agar memenuhi standar resolusi, hingga penyusunan *metadata,* seperti judul dan kata kunci. Keseluruhan proses tersebut dilakukan secara manual dan berulang, terutama bagi kreator yang memproduksi aset dalam jumlah besar. Menurut Adrian dan Sahono (2026), alur kerja manual dalam produksi konten *digital* berpotensi membuat hambatan efisiensi, karena banyaknya tahapan atau langkah yang harus dilakukan secara berurutan.

Selain itu, perkembangan teknologi kecerdasan buatan telah menyediakan berbagai alat bantu yang dapat mempermudah sebagian besar proses produksi tersebut. Namun, permasalahan utama dalam lingkup akademik dan implementasi praktis bukan hanya terletak pada ketersediaan alatnya, melainkan pada bagaimana berbagai teknologi terpisah tersebut dapat diintegrasikan menjadi satu sistem kerja

yang utuh dan berjalan secara otomatis. Tantangan integrasi sistem ini sejalan dengan perkembangan penelitian terdahulu yang mulai bergeser ke arah arsitektur agen cerdas atau *agentic workflow*. Singh et al. (2024) serta Jiang et al. (2025) menegaskan adanya pergeseran paradigma dari interaksi linear statis menuju alur kerja *non-linear* yang dinamis, di mana model bahasa besar tidak lagi sekadar merespons teks melainkan bertindak sebagai agen otonom yang memiliki kemampuan refleksi, perencanaan, kolaborasi, serta pemanfaatan alat eksternal atau *tool utilization*. Dalam hal efisiensi eksekusi, Niu et al. (2025) menjabarkan bahwa pendekatan *agentic workflow* masa kini harus dirancang secara modular agar mendukung penyesuaian alur kerja secara dinamis, meningkatkan toleransi kesalahan, serta mampu memangkas biaya operasional secara signifikan. Lebih lanjut, Chaudhry et al. (2025) dalam analisis performanya membuktikan bahwa visualisasi struktur internal pada orkestrasi alur kerja agen sangat krusial untuk mengoptimalkan titik keseimbangan antara tingkat akurasi, latensi, dan efisiensi sumber daya sistem berbasis *cloud*.

Guna mewujudkan otomatisasi alur kerja yang mampu mengintegrasikan logika berpikir *Artificial Intelligence* (AI) dengan eksekusi sistem tersebut, pemanfaatan *platform* berbasis *low-code* menjadi salah satu solusi praktis yang banyak dikembangkan. Fokus utama dalam pengembangan ini adalah transisi menuju *agentic* AI, di mana sistem tidak lagi sekadar menjalankan perintah linear, melainkan mampu melakukan perencanaan, pengambilan keputusan, dan penggunaan alat secara otonom untuk menyelesaikan tugas yang kompleks. Casaclang et al. (2026) dalam studi implementasinya menunjukkan bahwa *platform* otomatisasi alur kerja seperti n8n sangat efektif untuk menjembatani berbagai alat teknis konvensional dengan model bahasa besar eksternal, seperti Google Gemini. Melalui orkestrasi di dalam *platform n8n* tersebut, Casaclang et al. (2026) juga membuktikan bahwa penggabungan antara proses rutin dengan analisis berbasis kecerdasan buatan dapat memotong hambatan konsumsi waktu, serta keterbatasan sumber daya secara signifikan dalam sebuah eksekusi operasional. Sejalan dengan temuan tersebut, Hariyadi dan Soraya (2025) memperluas implementasi *platform low-code* n8n ini di sektor pendidikan tinggi untuk mengatasi hambatan prosedur

dan validitas data administrasi akademik melalui pengembangan asisten *virtual* berbasis *Retrieval Augmented Generation* (RAG).

Berdasarkan permasalahan tersebut, keterlibatan penulis sebagai seorang *microstoker* yang aktif memproduksi dan mengelola aset *digital* di Adobe Stock turut mendorong ketertarikan untuk mengembangkan sistem otomatisasi alur kerja berbasis arsitektur AI *Agent* dengan memanfaatkan *platform* n8n. Selama ini, proses produksi aset *digital* yang dijalankan penulis masih dilakukan secara manual dengan memanfaatkan berbagai perangkat terpisah, seperti Kling AI untuk pembuatan gambar, VideoProc untuk proses peningkatan kualitas gambar (*upscaling*), serta Google Gemini untuk penyusunan *metadata*. Alur kerja yang berjalan secara terpisah dan manual ini membutuhkan waktu, serta tenaga operasional yang cukup besar, sehingga mendorong timbulnya kebutuhan akan solusi yang lebih efisien.

1.2 Rumusan Masalah

Berdasarkan latar belakang masalah yang ada, maka rumusan masalah pada penelitian ini adalah:

1. Bagaimana merancang dan mengimplementasikan *agentic workflow* berbasis Google Gemini dan n8n pada *Bot* Telegram untuk otomatisasi produksi aset visual?
2. Bagaimana memangkas alur kerja produksi aset visual melalui skenario interaksi perintah tunggal (*single-trigger*) berbasis *cloud*?
3. Bagaimana tingkat kualitas, kinerja dan efisiensi waktu sistem otomatisasi yang dibangun dibandingkan dengan alur pengerjaan secara manual?

1.3 Ruang Lingkup

Pada rancang bangun otomatisasi alur kerja produksi aset visual *microstock* menggunakan arsitektur AI *agent* berbasis n8n, ditetapkan beberapa batasan untuk memfokuskan lingkup penelitian dan memastikan sistem dapat berfungsi. Adapun ruang lingkup tersebut adalah sebagai berikut:

1. Interaksi dan kendali operasional dilakukan melalui *bot* Telegram API

menggunakan bahasa Indonesia, dengan pemrosesan secara tunggal (*one-at-a-time*) untuk satu gambar per perintah.

2. Produksi aset visual berbasis model FLUX 1.1 Pro (rasio 1:1), *upscaling* dengan Real-ESRGAN melalui API Replicate, serta pembuatan *metadata* otomatis memanfaatkan model Gemini 2.5 Flash Lite.
3. Sistem memiliki fitur utama untuk produksi aset visual beserta *metadata* teks terpisah (*non-embedded*) untuk kebutuhan *platform* Adobe Stock, serta dilengkapi fitur riset tren pasar sebagai fitur pendukung.
4. Pengujian manual berbasis perbandingan menggunakan VideoProc Converter AI (alat *upscaling desktop*), Kling AI (alat pembuatan gambar), dan Gemini (alat pembuatan *metadata*).
5. Evaluasi kinerja dibatasi pada kualitas luaran (*output*), latensi pemrosesan, serta efisiensi alur kerja operasional (yang diukur dari aspek waktu dan *effort*) yang secara spesifik difokuskan pada alur produksi aset visual.
6. Durasi pemrosesan bergantung pada latensi jaringan dan stabilitas API pihak ketiga. Selain itu, potensi munculnya halusinasi AI (AI *hallucination*) merupakan karakteristik normal dari *Generative* AI berbasis probabilistik, sehingga luaran tetap memerlukan tahap verifikasi manusia (*human-in-the-loop*.

1.4 Tujuan Penelitian

Penelitian ini bertujuan untuk merancang dan membangun sistem otomatisasi alur kerja produksi aset visual gambar *microstock* untuk Adobe Stock yang berbasis arsitektur *agentic* AI dengan dukungan n8n, yaitu sebagai berikut:

1. Merancang dan mengimplementasikan *agentic workflow* berbasis Google Gemini dan n8n pada *bot* Telegram untuk otomatisasi produksi aset visual.
2. Memangkas alur kerja produksi aset visual melalui skenario interaksi perintah tunggal (*single-trigger*) berbasis *cloud*.
3. Menganalisis tingkat kualitas luaran, kinerja, dan efisiensi waktu sistem otomatisasi yang dibangun dibandingkan dengan alur pengerjaan secara manual.

1.5 Sistematika Penulisan

Penulisan Tugas Akhir ini disusun dalam lima pokok bahasan utama yang secara sistematis menjelaskan keseluruhan proses penelitian, mulai dari perumusan masalah hingga kesimpulan dan saran.

Pokok Bahasan 1 berisi Pendahuluan, yang mencakup latar belakang, rumusan masalah, ruang lingkup, tujuan penelitian, dan sistematika penulisan.

Pokok Bahasan 2 berisi Tinjauan Pustaka, yang menyajikan dasar teori, serta landasan ilmiah terkait komponen sistem dan penelitian terdahulu yang relevan sebagai acuan dalam penelitian ini.

Pokok Bahasan 3 berisi Metode Penelitian yang menguraikan tahapan pengembangan sistem secara sistematis, meliputi alur penelitian, analisis kebutuhan sistem, perancangan sistem, pembuatan prototipe, uji coba sistem, hingga evaluasi sistem. Penelitian ini menerapkan kombinasi metode *prototyping* dan eksperimental untuk membangun *agentic workflow* berbasis n8n, merancang pemodelan *Unified Modeling Language* (UML), serta merancang mekanisme penanganan kesalahan (*error handling*). Selanjutnya, bab ini menyajikan rancangan uji coba dan metode evaluasi untuk memvalidasi kualitas luaran, performa sistem, serta efisiensi waktu dan usaha operasional (*effort*) dibandingkan dengan alur pengerjaan secara manual.

Pokok Bahasan 4 berisi hasil dari implementasi sistem yang telah dikembangkan serta pembahasan mengenai kinerja sistem secara menyeluruh. Pada bab ini dijelaskan hasil implementasi sistem, implementasi antarmuka dan fitur *bot* Telegram (meliputi menu panduan, riset tren pasar, produksi aset visual, hingga penanganan berbagai skenario pesan kesalahan), serta implementasi dan pengujian layanan eksternal *Representational State Transfer Application Programming Interface* (REST API) menggunakan Postman beserta rekapitulasinya. Selanjutnya, bab ini menyajikan hasil uji coba sistem dan evaluasi yang membahas evaluasi kualitas luaran, analisis kinerja sistem, efisiensi alur kerja operasional (*effort*) dan waktu, tingkat keberhasilan produksi aset visual dibandingkan dengan metode manual, serta pembahasan mengenai kelebihan dan keterbatasan sistem yang

dikembangkan.

Pokok Bahasan 5 merupakan Penutup, yang menyajikan kesimpulan dari seluruh rangkaian penelitian, serta saran-saran pengembangan untuk meningkatkan efektivitas dan fungsionalitas sistem otomatisasi di masa mendatang.

2. TINJAUAN PUSTAKA

2.1 Kecerdasan Buatan dan Otomatisasi

Kecerdasan Buatan (*Artificial Intelligence*) dan otomatisasi alur kerja merupakan fondasi utama dalam pengembangan sistem cerdas berbasis pemrosesan bahasa dan pembuatan konten otomatis. Bagian ini membahas konsep dasar AI, *agentic* AI, *agentic Workflow, generative* AI, *large language model* (LLM), serta *platform low-code n8n* yang menjadi landasan konseptual dan arsitektur otomatisasi pada penelitian ini.

2.1.1 *Artificial Intelligence* (AI)

Artificial Intelligence (AI) merupakan cabang ilmu komputer yang berfokus pada pembangunan sistem atau mesin untuk meniru fungsi kognitif manusia, seperti *problem solving* dan pembelajaran. Pada prinsipnya, kecerdasan buatan AI adalah cabang ilmu komputer yang bertujuan untuk menciptakan sistem yang dapat melakukan tugas-tugas yang memerlukan kecerdasan manusia (Rifky et al., 2024). Teknologi ini dikembangkan untuk memproses informasi melalui algoritma terstruktur agar komputer dapat bertindak secara rasional. Melalui analisis *big data* dan pengenalan pola kompleks, sistem AI mampu membuat keputusan berbasis data secara otomatis dan efisien. Pada penelitian ini, AI menjadi dasar teori utama yang menopang seluruh pemrosesan cerdas di dalam sistem.

Secara umum, AI mencakup beberapa cabang utama yang saling melengkapi dalam membentuk sistem cerdas. Cabang utama yang menjadi dasar perkembangan AI saat ini adalah *Machine Learning* (ML), yaitu metode yang memungkinkan sistem mempelajari pola secara otomatis berdasarkan data yang diberikan. Selanjutnya terdapat *Deep Learning* (DL), yang merupakan bagian dari ML dengan memanfaatkan arsitektur jaringan saraf tiruan berlapis (*deep neural network*) untuk memproses data berdimensi tinggi, seperti citra, suara, dan teks. Cabang penting lainnya adalah *Natural Language Processing* (NLP), yang dikembangkan agar komputer dapat memahami, menganalisis, dan menghasilkan bahasa alami manusia. Selain itu, terdapat bidang *computer vision* yang berfokus pada

kemampuan mesin dalam mengenali, serta menginterpretasikan objek visual, serta *robotics* dan *expert system* yang dirancang untuk mensimulasikan penalaran pakar dalam pengambilan keputusan spesifik. Penggabungan berbagai cabang ini memungkinkan pembangunan ekosistem AI yang lebih kompleks dan adaptif. Namun demikian, meskipun AI membawa otomatisasi dan efisiensi, AI juga menimbulkan kekhawatiran tentang perpindahan pekerjaan. Tugas-tugas tertentu yang sebelumnya dilakukan oleh manusia sekarang dapat dilakukan oleh sistem AI, yang berpotensi menyebabkan pengangguran dan gangguan sosial (Sianturi et al., 2023).

Dalam konteks penelitian ini, AI dimanfaatkan sebagai landasan utama untuk membangun ekosistem pembuatan aset citra *digital* otomatis berbasis *agentic workflow*. Sistem yang dirancang mengintegrasikan beberapa pemrosesan cerdas secara berkesinambungan, di mana LLM berfungsi sebagai mesin penalaran (*reasoning engine*) untuk menerjemahkan instruksi pengguna, sedangkan model AI generatif dieksekusi untuk mensintesis gambar dan mengoptimalkan kualitas visualnya. Melalui pendekatan agen cerdas (AI *agent*), seluruh alur dari penerimaan perintah, pembuatan *prompt*, generasi citra, hingga ekstraksi *metadata* dapat berjalan secara mandiri dan terstruktur.

2.1.2 *Agentic* AI

Agentic AI merupakan paradigma baru dalam kecerdasan buatan yang merujuk pada sistem AI dengan kemampuan beroperasi secara otonom untuk mencapai tujuan tertentu melalui tindakan yang terencana dan adaptif. *Agentic* AI merujuk pada sistem kecerdasan buatan yang tidak hanya bersifat reaktif, tetapi mampu bertindak otonom, merencanakan tindakan, berkolaborasi dengan pengguna, serta menyesuaikan perilakunya berdasarkan konteks dan tujuan tertentu (Falaq & Pradana, 2025). Berbeda dengan model AI generatif pasif yang hanya memberikan respons satu kali (*single-turn*) berdasarkan *prompt* langsung, *agentic* AI mampu mengamati lingkungan (*environment*), memecah instruksi kompleks menjadi tahapan tugas kecil (*task decomposition*), serta mengambil keputusan secara mandiri tanpa memerlukan intervensi manusia secara terus-menerus.

Kemampuan otonomi ini memungkinkan sistem untuk beradaptasi secara dinamis terhadap perubahan data atau respons yang diterima selama proses eksekusi berlangsung.

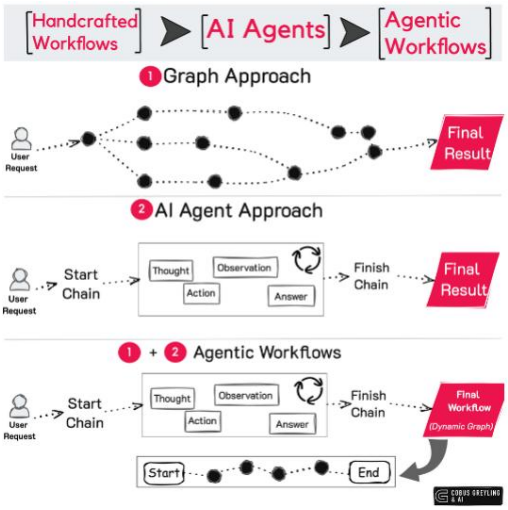
Secara arsitektural, keberhasilan ekosistem *agentic* AI ditopang oleh beberapa komponen utama. Komponen pertama adalah *brain* atau mesin penalaran (*reasoning engine*), yang memanfaatkan *Large Language Model* (LLM) untuk menganalisis konteks dan merencanakan alur kerja. Komponen kedua adalah *memory*, yang bertugas menyimpan konteks interaksi jangka pendek maupun jangka panjang guna mempertahankan konsistensi keputusan. Komponen ketiga adalah *tools*, yaitu sekumpulan fungsi eksternal, layanan basis data, atau API yang dapat dipanggil secara mandiri oleh sistem untuk mengeksekusi tindakan nyata di luar lingkungan model AI itu sendiri.

2.1.3 *Agentic Workflow*

Agentic workflow merupakan sebuah perkembangan signifikan yang menekankan kolaborasi antara manusia dan sistem cerdas (Zheng et al., 2025). Konsep ini menjadi paradigma arsitektur dalam kecerdasan buatan yang berfokus pada proses iteratif dan otonom untuk menyelesaikan tugas-tugas kompleks berdimensi tinggi. Berbeda dengan pendekatan penyuntingan perintah tradisional yang bersifat satu kali respons, *agentic workflow* mengimplementasikan model bahasa besar sebagai agen cerdas yang beroperasi dalam sebuah siklus persepsi, penalaran, tindakan, dan adaptasi. Dalam implementasinya, pengguna cukup mendefinisikan tujuan akhir, aturan batasan, serta instrumen pendukung yang dapat diakses oleh sistem. Selanjutnya, agen AI akan secara mandiri menguraikan instruksi makro tersebut menjadi tahapan tugas kecil tanpa memerlukan intervensi manusia secara terus-menerus di setiap langkah eksekusi.

Penerapan *agentic workflow* dalam arsitektur sistem informasi modern membuka peluang besar untuk menciptakan ekosistem *digital* yang lebih responsif dan cerdas. Melalui kemampuan penalaran dinamis, alur kerja ini tidak hanya mengotomatiskan tugas-tugas administratif yang berulang, tetapi juga mampu mengintegrasikan berbagai aplikasi pihak ketiga secara mandiri untuk

menyelesaikan alur kerja yang rumit. Hal ini meminimalkan hambatan operasional dan mempercepat waktu eksekusi program, karena setiap keputusan perbaikan data dapat langsung diselesaikan di tingkat agen cerdas. Pada akhirnya, implementasi teknologi ini menjadi fondasi penting dalam penelitian untuk membangun sistem otomasi yang adaptif terhadap perubahan kebutuhan pengguna. Tahapan evolusi arsitektur sistem dari pemetaan graf manual hingga pembentukan graf dinamis secara runtut dijabarkan pada Gambar 2.1.



Gambar 2.1 Skema Graf *Agentic Workflow*

(Sumber: https://cobusgreyling.medium.com/from-handcrafted-*workflow*s-to-ai-*agent*s-to-*agentic-workflow*s-49e8f85daf7d)

Berdasarkan Gambar 2.1, terdapat tiga tingkat pendekatan dalam sistem otomasi modern. Pendekatan pertama adalah *Graph Approach* yang bersifat kaku, karena seluruh jalur eksekusi dari *input* hingga hasil akhir harus ditentukan secara manual oleh manusia. Pendekatan kedua adalah AI *Agent Approach* yang mengenalkan siklus penalaran mandiri melalui proses *Thought*, *Action*, *Observation*, dan *Answer*. Pada tingkat tertinggi, kedua metode tersebut diintegrasikan menjadi *Agentic Workflows* yang mampu menyusun graf alur kerja secara dinamis dan adaptif dari awal hingga selesai.

2.1.4 *Generative Artificial Intelligence*

Generative Artificial Intelligence (*Generative* AI) merupakan cabang dari kecerdasan buatan yang berfokus pada pengembangan model cerdas yang mampu menciptakan konten baru secara sintetis dan orisinal. Berbeda dengan pendekatan AI prediktif atau diskriminatif yang hanya berfungsi untuk mengklasifikasikan data dan mengenali pola, *generative* AI mempelajari struktur dasar, serta representasi data dari kumpulan data pelatihan yang berskala besar. Dalam implementasinya, AI Generatif beroperasi menggunakan arsitektur *deep learning* tingkat lanjut, seperti *Latent Diffusion Models* (LDM) untuk gambar/video dan LLM untuk pemrosesan teks (Briliansah et al., 2026).

Secara teknis, perkembangan *generative* AI ditopang oleh arsitektur model generatif modern, seperti *Generative Adversarial Networks* (GANs), *Variational Autoencoders* (VAEs), serta *diffusion models*. Pada domain pemrosesan visual, *diffusion models* menjadi salah satu arsitektur terdepan yang bekerja dengan cara menambahkan bising acak (*noise*) pada citra secara bertahap, lalu melatih model untuk merekonstruksi kembali citra asli dari kondisi bising tersebut. Proses pengurangan bising (*denoising*) yang dikombinasikan dengan pemahaman teks memungkinkan sistem untuk memproduksi citra beresolusi tinggi yang sangat *detail* sesuai dengan petunjuk deskriptif yang diberikan.

Di samping itu, proses sintesis pada model generatif pada dasarnya memiliki sifat stokastik (*stochastic*). Sifat stokastik ini mengacu pada adanya unsur probabilitas dan keacakan terukur saat model menarik sampel dari ruang laten (*latent space*) selama proses *denoising*. Implikasinya, meskipun sistem menerima instruksi masukan (*prompt*) yang identik, model AI dapat menghasilkan variasi luaran visual yang berbeda dalam hal detail halus, tata letak, maupun pencahayaan, terutama ketika *prompt* tidak memberikan batasan spasial atau komposisi yang spesifik.

2.1.5 *Large Language Model* (LLM)

Large Language Model (LLM) merupakan arsitektur kecerdasan buatan

berbasis pemrosesan bahasa alami yang dilatih menggunakan volume data teks berskala sangat besar. Melalui proses pelatihan mendalam tersebut, LLM memiliki pemahaman konteks bahasa yang intuitif, sehingga mampu melakukan berbagai tugas kebahasaan, seperti penerjemahan, pembuat rangkuman, penyusun kode program, hingga pemrosesan penalaran kompleks (*complex reasoning*). Hal ini dikarenakan LLM adalah model bahasa yang sangat besar dan kompleks yang dapat digunakan untuk menganalisis dan menggenerasikan teks (Muhajir et al., 2025). Model ini dirancang untuk memahami masukan berupa instruksi bahasa alami dari pengguna dan menghasilkan luaran teks yang kontekstual, logis, serta koheren.

Secara teknis, keandalan LLM bertumpu pada arsitektur *transformer* yang memanfaatkan mekanisme perhatian mandiri. Mekanisme ini memungkinkan model untuk menganalisis hubungan antarkata dalam satu kalimat atau paragraf secara simultan tanpa terikat oleh urutan posisi yang kaku. Selain itu, LLM modern dilengkapi dengan kemampuan pemahaman ruang konteks (*context window*) yang luas, serta fitur luaran terstruktur, seperti format *JavaScript Object Notation* (JSON). Fitur ini sangat penting dalam pengembangan sistem otomatisasi, karena memungkinkan luaran bahasa alami yang dihasilkan oleh model diproses kembali oleh sistem komputer lain secara presisi tanpa mengalami galat penafsiran.

Dalam konteks penelitian ini, LLM bertindak sebagai mesin penalaran (*reasoning engine*) utama yang mengendalikan alur kerja cerdas. LLM bertugas untuk menganalisis masukan perintah dari pengguna, menguraikan maksud instruksi, menyusun deskripsi *prompt* yang optimal untuk pembuat gambar, serta mengekstrak *metadata* berupa judul dan kata kunci secara otomatis. Pemanfaatan LLM membuat sistem memiliki fleksibilitas tinggi dalam menerjemahkan kebutuhan pengguna menjadi instruksi teknis yang siap dieksekusi oleh modul sistem lainnya.

2.1.6 *Low-Code Platform (*n8n*)*

Low-code platform merupakan lingkungan pengembangan perangkat lunak yang memungkinkan pembuatan aplikasi maupun otomatisasi alur kerja secara visual dengan meminimalisir penulisan kode program secara manual. *Low-code*

platform dipilih sebagai *platform* pengembangan, karena kecepatan dalam pembuatan sebuah perangkat lunak, sistem pembuatan *user interface* secara *drag and drop*, bekerja dengan sistem *cloud computing*, dan banyak keunggulan lainnya (Abednego & Julianto, 2021). Pendekatan *low-code* ini meningkatkan efisiensi pengembangan sistem, mempercepat proses integrasi antar aplikasi, serta mengurangi potensi kesalahan penulisan sintaksis kode dalam membangun arsitektur otomatisasi yang kompleks.

Salah satu *platform* otomatisasi *low-code* berbasis alur kerja (*workflow-based*) yang populer adalah n8n. n8n merupakan perangkat lunak otomatisasi berbasis *node* yang bersifat *self-hosted*, sehingga memberikan kendali penuh kepada pengembang atas privasi data dan kustomisasi *server*. *Platform* ini mendukung integrasi dengan berbagai layanan pihak ketiga melalui API, *webhook*, serta modul kecerdasan buatan terintegrasi. Dengan struktur data berbasis JSON di setiap *node*-nya, n8n memudahkan proses manipulasi, pemetaan, dan pengiriman data antar sistem secara efisien dan terstruktur.

Dalam konteks penelitian ini, n8n dimanfaatkan sebagai orkestrator alur kerja utama yang menghubungkan seluruh komponen dalam ekosistem *agentic workflow*. *Platform* ini bertugas menerima perintah dari pengguna, mengelola alur komunikasi data antara model bahasa berskala besar, memicu eksekusi model generasi citra pada *server cloud*, hingga menyimpan hasil akhir dan *metadata* ke dalam basis data. Pemanfaatan n8n membuat seluruh proses otomatisasi pembuatan aset *digital* terintegrasi secara stabil, transparan, dan mudah dipelihara. Bentuk visual logo *platform* otomasi ini ditampilkan pada Gambar 2.2.



Gambar 2.2 Logo *n8n*

(Sumber: https://www.turbodocx.com/newsroom/turbodocx-launches-n8n-integration)

2.2 Model dan Layanan Kecerdasan Buatan

Pengembangan sistem pembuatan aset citra *digital* berbasis *agentic workflow* membutuhkan integrasi berbagai model, serta layanan kecerdasan buatan yang memiliki spesialisasi fungsi masing-masing. Subbab ini menguraikan landasan teoretis dan teknis mengenai kelompok LLM, *platform gateway* pemrosesan bahasa, model pembuat citra sintetis, serta algoritma pemrosesan visual untuk peningkatan kualitas gambar yang dimanfaatkan dalam penelitian ini.

2.2.1 Google Gemini

Google Gemini merupakan kelompok LLM multimodal yang dikembangkan oleh Google untuk memproses dan mengintegrasikan berbagai bentuk modalitas data secara simultan. Kemampuan multimodal membuat Gemini efektif dalam memahami deskripsi kompleks (Syahputra et al., 2026). Model ini dirancang untuk memberikan kinerja tinggi dalam pemrosesan bahasa alami, analisis logika, serta pemahaman konteks instruksi yang panjang. Fitur-fitur unggulannya memungkinkan sistem menganalisis masukan secara intuitif dan menghasilkan luaran terstruktur, seperti format JSON, yang sangat dibutuhkan dalam otomatisasi perangkat lunak.

Secara teknis, Google Gemini memanfaatkan arsitektur *transformer* multimodal yang memungkinkan pemahaman hubungan antarkata dan analisis konteks secara fleksibel. Alat ini membantu dalam menangani *reinforcement learning*, *deep learning*, serta masalah dan tugas-tugas yang berkaitan dengan edukasi *digital* (Imran & Almusharraf, 2024). Dalam arsitektur *agentic workflow*, varian model, seperti Gemini 2.5 Flash dan Gemini 2.5 Flash-Lite dimanfaatkan sebagai mesin penalaran (*reasoning engine*) yang mengeksekusi instruksi dari pengguna secara efisien. Varian ini bertugas menganalisis masukan pengguna, melakukan riset, serta pengembangan deskripsi *prompt* visual yang kaya, hingga mengekstrak *metadata* penting berupa judul dan kata kunci yang sesuai dengan gambar secara presisi dengan kecepatan pemrosesan yang tinggi.

Pemilihan Google Gemini dalam penelitian ini didasarkan pada kemampuannya menghasilkan pemrosesan logika dan pemahaman instruksi yang

sangat akurat dengan latensi yang rendah. Selain itu, fitur luaran terstruktur yang dimilikinya memastikan data riset dan *metadata* yang dihasilkan dapat dialirkan ke tahapan pemrosesan berikutnya tanpa menimbulkan kesalahan penafsiran (*syntax error*). Karakteristik ini membuat Google Gemini sangat ideal untuk membangun ekosistem otomatisasi berbasis *agentic workflow* yang responsif dan terstruktur. Bentuk identitas visual teknologi tersebut tertera pada Gambar 2.3.



Gambar 2.3 Logo Teknologi Google Gemini

(Sumber: https://logos-world.net/google-gemini-logo/)

2.2.2 OpenAI

OpenAI merupakan perusahaan teknologi AI terkemuka yang mengembangkan model-model kecerdasan buatan yang canggih (Andini et al., 2024). OpenAI API merupakan standar antarmuka pemrograman aplikasi sekaligus protokol komunikasi data yang dikembangkan oleh OpenAI dan diadopsi secara luas dalam ekosistem kecerdasan buatan. Spesifikasi API ini menyediakan struktur data, skema pemanggilan, serta format enkapsulasi *prompt* yang fleksibel, seperti fitur pemanggilan fungsi, pemanggilan alat eksternal, hingga luaran terstruktur. Karena sifatnya yang universal dan menjadi standar industri, banyak *platform* penyedia layanan kecerdasan buatan pihak ketiga menyediakan dukungan kompatibilitas (OpenAI-*compatible endpoint*) agar berbagai model bahasa dapat diakses menggunakan protokol komunikasi OpenAI.

Secara teknis, penggunaan antarmuka berbasis standar OpenAI memungkinkan sistem untuk mengonsumsi berbagai model LLM tanpa harus menggunakan pustaka atau modul integrasi khusus dari masing-masing pengembang. Dalam arsitektur *agentic workflow*, integrasi ini dihubungkan melalui modul pemrosesan (*node*) berstandar OpenAI yang dikonfigurasi dengan

mengarahkan *host* atau *base* URL ke penyedia layanan terkait. Mekanisme ini bertindak sebagai jembatan komunikasi data untuk mengalirkan instruksi pemrosesan penalaran, riset deskripsi *prompt* visual, hingga ekstraksi *metadata* berupa judul dan kata kunci yang sesuai dengan gambar.

Pemilihan pendekatan OpenAI API dalam penelitian ini didasarkan pada fleksibilitas integrasi, kemudahan kustomisasi *endpoint*, serta efisiensi pengoperasian modul dalam *platform* otomatisasi. Selain itu, penggunaan OpenAI *Software Development Kit* (SDK) dimanfaatkan sebagai pustaka perangkat lunak yang menyediakan fungsi-fungsi siap pakai untuk mempermudah eksekusi perintah dan pengelolaan interaksi dengan model AI secara terstruktur. Dengan memanfaatkan antarmuka yang kompatibel dengan standar OpenAI, sistem dapat memanggil dan mengoperasikan model Gemini 2.5 Flash-Lite secara langsung tanpa perlu merombak struktur *node* otomatisasi yang telah dibangun. Pendekatan teknis ini memastikan alur komunikasi data antar *node* pemrosesan dapat berjalan secara stabil, konsisten, dan mudah dipelihara. Bentuk identitas visual teknologi tersebut tertera pada Gambar 2.4.



Gambar 2.4 Logo Perusahaan Teknologi OpenAI

(Sumber: https://vectorseek.com/vector_logo/openai-logo-vector/)

2.2.3 OpenRouter

OpenRouter merupakan *platform* agregator layanan kecerdasan buatan (*LLM gateway*) yang menyediakan satu antarmuka pemrograman aplikasi tunggal untuk mengakses berbagai jenis model bahasa besar, termasuk model Gemini yang dikembangkan oleh Google. *Platform* ini mengonsolidasikan berbagai model mutakhir ke dalam satu pintu akses dengan manajemen kuota yang terpadu. Keunggulan utama OpenRouter terletak pada penyederhanaan manajemen kunci API, efisiensi autentikasi, serta kemampuannya menyediakan jalur akses alternatif

yang transparan terkait tingkat latensi dan biaya operasional model secara waktu nyata (*real-time*).

Pemilihan *platform* OpenRouter dalam sistem ini ditujukan secara khusus untuk mengoperasikan model Gemini 2.5 Flash yang bertindak sebagai mesin penajam perintah (*prompt enhancer*). Dengan memanfaatkan OpenRouter sebagai perantara, sistem dapat menggunakan kemampuan penalaran tinggi dari Gemini 2.5 Flash untuk mentransformasi instruksi dasar pengguna menjadi draf perintah visual yang komprehensif, terstruktur, dan optimal. Melalui metode ini, seluruh alur pemrosesan pemurnian teks perintah dapat berjalan secara stabil, konsisten, dan terhindar dari kendala pembatasan akses (*rate limit*).

2.2.4 *Platform* Replicate (FLUX 1.1 Pro Model dan Real-ESRGAN)

Replicate merupakan layanan berbasis *cloud platform* yang menyediakan infrastruktur untuk menjalankan dan mengintegrasikan berbagai model kecerdasan buatan berbasis *open-source* melalui API. Replicate memungkinkan pengguna menjalankan model pembelajaran mesin dengan API *cloud*, tanpa harus memahami seluk-beluk pembelajaran mesin atau mengelola infrastruktur secara mandiri (Replicate, n.d.). Dalam ekosistem pemrosesan visual, Replicate menyediakan akses cepat ke berbagai model generatif dan pemrosesan citra terkini yang siap dihubungkan ke dalam alur kerja otomatisasi.

Di dalam ekosistem Replicate, FLUX 1.1 Pro dan Real-ESRGAN bertindak sebagai dua model utama yang menangani tahapan sintesis dan penyempurnaan visual. FLUX 1.1 Pro merupakan model generatif tingkat lanjut berbasis arsitektur *diffusion* modern yang berfungsi menghasilkan citra *digital* sintetis berdasarkan deskripsi *prompt* hasil riset pemrosesan bahasa. Model ini mampu menghasilkan citra dalam pilihan format JPG maupun WebP secara langsung dari proses pemrosesan visual. Sementara itu, Real-ESRGAN (*Real-Enhanced Super-Resolution Generative Adversarial Networks*) berfokus pada pemrosesan peningkatan resolusi citra (*upscaling*). Model ini bekerja dengan menggunakan arsitektur GAN untuk menghasilkan citra beresolusi tinggi dari *input* beresolusi rendah (Çetin, 2023). Proses tersebut dijalankan dengan cara menghilangkan

distorsi visual (*artifact*) dan merekonstruksi detail tekstur citra hingga menghasilkan berkas luaran akhir berformat PNG.

Pemilihan *platform* Replicate didasarkan pada kemudahan integrasi antarmuka API-nya yang fleksibel dengan modul *agentic workflow*. Penggunaan varian FLUX 1.1 Pro dipilih, karena proses pemrosesan gambarnya yang lebih cepat serta menghasilkan visual yang berkualitas dan bermutu tinggi. Melalui API Replicate, *agentic workflow* dapat memicu eksekusi pembuatan citra berformat JPG atau WebP oleh FLUX 1.1 Pro dan secara otomatis mengalirkannya ke model Real-ESRGAN untuk peningkatan resolusi menjadi format PNG secara terpadu, stabil, dan konsisten. Bentuk identitas visual teknologi tersebut tertera pada Gambar 2.5.



Gambar 2.5 Logo *Platform Cloud* Replicate

(Sumber: https://seeklogo.com/vector-logo/611690/replicate-icon)

2.2.5 Peningkatan Resolusi Citra Berbasis AI

Peningkatan resolusi citra berbasis kecerdasan buatan (AI-*based image upscaling*) merupakan teknik pemrosesan citra *digital* yang memanfaatkan model pembelajaran mendalam (*deep learning*) untuk meningkatkan dimensi piksel dan kualitas visual suatu gambar. Teknik *super-resolution* citra berbasis *deep learning* dapat membantu dalam mengekstraksi fitur spasial dari citra beresolusi rendah yang diambil dengan teknologi saat ini (Nandal et al., 2024). Berbeda dengan metode interpolasi tradisional yang sekadar mereplikasi piksel di sekitarnya, sehingga sering kali menghasilkan efek kabur (*blur*), pendekatan berbasis kecerdasan buatan ini mampu merekonstruksi detail tekstur yang hilang secara cerdas dengan mempelajari pola fitur visual dari kumpulan data beresolusi tinggi.

Secara teknis, proses *super-resolution* berbasis AI melatih jaringan saraf

konvolusional (*Convolutional Neural Network* atau CNN) maupun arsitektur *Generative Adversarial Networks* (GAN) untuk memprediksi struktur spasial, ketajaman garis tepi, dan persepsi tekstur pada skala yang lebih besar. Dalam konteks pemrosesan aset *digital*, teknik ini bertindak sebagai tahapan penyempurnaan (*post-processing*) untuk memperbaiki kompresi awal atau cacat visual (*artifact*) pada citra hasil sintesis generatif. Pemrosesan ini memastikan bahwa setiap objek, batas kontur, dan pencahayaan direkonstruksi, sehingga kerapatan piksel (*density*) meningkat secara signifikan.

Pemilihan teknik peningkatan resolusi citra berbasis AI dalam penelitian ini didasarkan pada kebutuhan untuk menghasilkan aset visual yang memenuhi standar mutu tinggi. Citra mentah hasil sintesis awal sering kali memiliki keterbatasan ukuran piksel yang belum optimal untuk kebutuhan publikasi *mikrostock* maupun pencetakan. Dengan menerapkan tahapan *upscaling* berbasis AI secara otomatis di dalam alur kerja sistem, aset gambar yang dihasilkan tidak hanya memiliki dimensi yang lebih besar dan berformat PNG yang jernih, tetapi juga mempertahankan ketajaman visual secara konsisten dan efisien.

2.2.6 *Platform* Aivene

Platform Aivene merupakan infrastruktur pengembangan modern yang dirancang untuk membangun, menguji, dan mempublikasikan agen kecerdasan buatan (AI *agents*) skala produksi secara efisien tanpa perlu mengelola infrastruktur yang rumit. Melalui antarmuka konsol yang intuitif, *platform* ini memungkinkan penyusunan agen AI hanya dalam hitungan menit, serta menyediakan fitur *playground* untuk menguji alur kerja sebelum diterapkan ke kode program. Selain itu, Aivene memiliki fitur *model swappable* yang memudahkan pengguna untuk berganti model AI (seperti OpenAI, Claude, Gemini, atau DeepSeek) secara instan. Keunggulan ini memungkinkan pergantian model AI dilakukan tanpa perlu merombak alur logika sistem maupun mengonfigurasi ulang API *key* dan *software development kit* (SDK) secara terpisah.

Penerapan *platform* Aivene memberikan manfaat signifikan dalam mempercepat siklus pengembangan dan meningkatkan fleksibilitas integrasi alur

kerja otomatisasi. Dalam riset ini, Aivene dimanfaatkan sebagai penyedia infrastruktur *host* yang dihubungkan ke *platform* n8n melalui *node* OpenAI untuk menjalankan model Gemini 2.5 Flash Lite. Penggunaan Aivene memungkinkan pemanfaatan arsitektur OpenAI-*compatible endpoint*, sehingga model Gemini dapat dipanggil secara lancar melalui antarmuka *node* OpenAI di n8n tanpa hambatan kompatibilitas. Hal ini tidak hanya memangkas waktu konfigurasi infrastruktur dan efisiensi biaya, tetapi juga memberikan perlindungan dari ketergantungan penyedia layanan (*vendor lock-in*), karena model AI dapat diganti sewaktu-waktu tanpa merombak skenario alur kerja yang telah dibangun di n8n.

2.3 Arsitektur *Server* dan *Infrastructure Deployment*

Arsitektur *server* merupakan fondasi utama dalam merancang infrastruktur komputasi yang stabil untuk menangani alur kerja sistem modern. Arsitektur *server* merujuk pada sistem komputasi terdistribusi di mana tugas-tugas dibagi antara komputer *server* dan komputer klien (ScienceDirect, n.d.). Dalam konteks ini, penerapan *agentic workflow* untuk pembuatan dan pengelolaan aset citra *digital* mengandalkan struktur *server* untuk memproses logika bisnis dan komunikasi antar-*node* secara efisien. Landasan teknis ini mencakup perancangan arsitektur *cloud* dan *backend*, teknologi kontainerisasi aplikasi, mekanisme *tunneling* jaringan, serta layanan penyimpanan data terdistribusi agar seluruh komponen sistem terintegrasi, aman, dan beroperasi secara optimal.

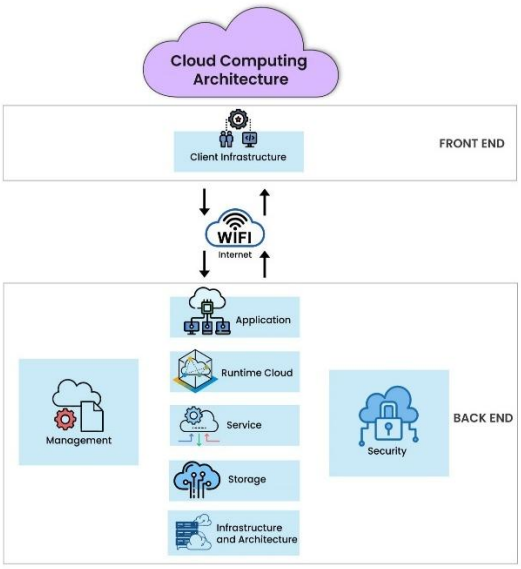
2.3.1 Arsitektur *Cloud* dan *Backend*

Arsitektur *cloud* dan *backend* merupakan fondasi struktural yang mengatur aliran dan pemrosesan data dalam ekosistem otomatisasi. *Cloud computing* merupakan sebuah arsitektur teknologi yang dapat digunakan untuk membantu menyelesaikan masalah dalam hal penggunaan sumber daya fisik komputer dan media penyimpanan (Wardhana & Assegaff, 2017). Pendekatan *backend-as-a-service* atau *self-hosted cloud* memberikan fleksibilitas tinggi untuk menghubungkan layanan pihak ketiga melalui API. Dalam *agentic workflow*, *backend* bertindak sebagai pusat kendali yang mengatur instruksi dinamis,

memantau agen kecerdasan buatan, dan mengelola antrean data secara *real-time*.

Dalam otomatisasi produksi aset visual ini, sistem memanfaatkan arsitektur *backend* untuk mengeksekusi logika pemrosesan data. *Backend* adalah ruang logis dengan fungsionalitas dan pengoperasian aplikasi perangkat lunak atau sistem informasi (Herdiyatmoko, 2022). Infrastruktur ini diwujudkan melalui *platform* n8n secara *self-hosted* di dalam kontainer Docker agar lingkungan eksekusi tetap konsisten dan terisolasi. Selain itu, layanan Ngrok digunakan sebagai *secure tunneling* berbasis *Hypertext Transfer Protocol Secure* (HTTPS) untuk menghubungkan *webhook* Telegram Bot API ke n8n, sehingga instruksi pengguna dapat diproses secara *real-time*.

Sebagai motor penggerak utama, *platform* pengelolaan alur berbasis *node* mengotomatisasi interaksi antar-API secara terstruktur dan modular. Setiap *node* merepresentasikan fungsi spesifik, seperti menangani *request* HTTP, mengeksekusi logika pencabangan, hingga mengelola data biner secara asinkron. Pendekatan modular pada *backend* ini menjaga integritas data serta mencegah kesalahan sistem yang merembet. Dalam pemrosesan perintah tunggal, alur dari penalaran agen, pemanggilan model FLUX 1.1 Pro, hingga peningkatan resolusi melalui Real-ESRGAN dapat dipantau secara terisolasi tanpa membebani memori utama, jika terjadi *timeout* pada API eksternal. Visualisasi pembagian komponen komputasi *cloud* yang mendasari struktur sistem ini diperlihatkan pada Gambar 2.6.



Gambar 2.6 Struktur Arsitektur *Cloud Computing*

(Sumber: https://www.infosectrain.com/blog/what-is-*cloud*-computing-architecture-and-its-components)

Berdasarkan Gambar 2.6, seluruh komponen *frontend* dan *backend* dihubungkan secara terintegrasi melalui jaringan internet untuk menjamin kelancaran sirkulasi data. Di sisi *frontend*, infrastruktur klien mengirimkan instruksi pengguna yang kemudian diterima oleh sisi *backend* untuk diproses ke dalam lapisan aplikasi, layanan, dan penyimpanan. Integrasi ini dipayungi oleh sistem manajemen dan keamanan terpusat, sehingga mampu mengisolasi kegagalan sistem pada satu lapisan tanpa mengganggu stabilitas ekosistem *cloud computing* secara keseluruhan.

2.3.2 Kontainerisasi Aplikasi (Docker)

Docker adalah sebuah *project open-source* yang menyediakan *platform* terbuka untuk *developer* maupun *sysadmin* untuk dapat membangun, mengemas, dan menjalankan aplikasi dimanapun di dalam sebuah *container* (Dwiyatno et al., 2020). Berbeda dengan teknologi virtualisasi tradisional menggunakan *virtual machine* yang memerlukan sistem operasi tamu (*guest OS*) penuh di atas hipervisor, kontainerisasi berjalan langsung di atas kernel sistem operasi induk dan berbagi

sumber daya perangkat keras secara berbagi pakai. Docker hadir sebagai *platform open source* paling populer yang mengotomatisasi proses pembuatan, distribusi, dan pengelolaan kontainer tersebut.

Dalam siklus pengembangan sistem modern, Docker mengatasi masalah klasik inkonsistensi lingkungan eksekusi perangkat lunak ketika dipindahkan antar-*server* yang berbeda. Menggunakan konsep "*build once run anywhere"*, sebuah aplikasi orkestrasi alur kerja yang dikemas di dalam kontainer Docker dijamin akan berjalan dengan perilaku teknis yang sama, baik saat dijalankan di komputer lokal maupun saat diterapkan di ekosistem *cloud*. Portabilitas tinggi ini dicapai, karena Docker mengisolasi seluruh dependensi internal aplikasi dari modifikasi sistem operasi luar.

Secara teknis, fungsionalitas Docker didasarkan pada tiga komponen utama, yaitu Docker *image*, Docker *registry*, dan Docker *container*. Docker *image* merupakan cetak biru statis dan bersifat siap baca (*read only*) yang tersusun dari lapisan *layer* instruksi untuk membentuk lingkungan sistem yang diinginkan. Ketika *image* tersebut dieksekusi oleh Docker *engine* sebagai *runtime*, maka terbentuklah Docker *container* sebagai unit komputasi aktif yang menjalankan fungsi logika bisnis aplikasi secara terisolasi. Bentuk identitas visual dari teknologi kontainerisasi ini tertera pada Gambar 2.7.



Gambar 2.7 Logo *Platform* Kontainerisasi Docker

(Sumber: https://www.docker.com/company/newsroom/media-resources/)

2.3.3 *Tunneling* Jaringan Lokal (Ngrok)

Penerowongan jaringan aman (*secure tunneling*) merupakan metode teknologi yang memungkinkan sebuah *server* lokal di balik proteksi *firewall* atau *Network Address Translation* (NAT) dapat diakses secara publik. Ngrok adalah

aplikasi yang memungkinkan bagi seseorang untuk mengekspos *server local* menjadi *public* internet (Parlika et al., 2021). Hambatan keterbatasan alamat *Internet Protocol* (IP) lokal diatasi oleh Ngrok, yang hadir sebagai perangkat lunak *platform* layanan bertindak sebagai *reverse proxy* untuk membuat jalur komunikasi aman dua arah dari komputer lokal menuju *server* publik Ngrok *global*. Melalui mekanisme ini, pengembang dapat melakukan pengujian sistem secara langsung tanpa perlu melakukan konfigurasi perutean *port* (*port forwarding*) yang rumit pada *router* jaringan lokal.

Secara teknis, Ngrok bekerja dengan membangun koneksi terenkripsi dari komputer lokal ke *server cloud* publik untuk menghasilkan sebuah alamat tautan publik unik berbasis protokol *Hypertext Transfer Protocol Secure* (HTTPS). Alamat tersebut bertindak sebagai pintu masuk untuk meneruskan lalu lintas data luar, termasuk fitur pesan balik otomatis atau *webhook* dari Telegram *Bot* API menuju sistem orkestrasi n8n. Jalur aman ini menjamin keamanan transmisi data teks maupun berkas biner dari risiko intersepsi pihak ketiga selama melintasi internet. Meskipun demikian, kinerja koneksi ini sangat bergantung pada faktor eksternal, seperti latensi internet dan stabilitas *server* penyedia layanan. Oleh sebab itu, karakteristik penerowongan ini menjadi parameter penting dalam menganalisis total durasi waktu eksekusi sistem saat mentransfer data dari antarmuka pengguna menuju komponen pengolah gambar internal. Bentuk identitas visual dari *platform secure tunneling* ini ditampilkan pada Gambar 2.8.



Gambar 2.8 Logo *Platform Tunneling* Ngrok

(Sumber: https://osamadev.medium.com/ngrok-a-developers-gateway-to-localhost-7132dc1f4117)

2.3.4 Supabase (*Storage Bucket*)

Supabase merupakan sebuah *backend-as-a-service* berbasis *open source* yang dirancang untuk membangun arsitektur aplikasi tanpa harus mengelola infrastruktur *server* secara manual. Supabase merupakan *database open source* yang hadir sebagai alternatif dari Firebase (Rashidi, Brata, & Brata, 2022). Layanan ini dibangun di atas fondasi basis data relasional PostgreSQL dan mengintegrasikan berbagai fitur esensial, seperti autentikasi, fungsi instan, serta penyimpanan objek (*object storage*). Di dalam ekosistem pemrosesan berkas multimedia, ketersediaan media penyimpanan eksternal yang andal sangat krusial untuk memisahkan beban kerja komputasi dengan kapasitas penyimpanan data agar performa *server* utama tetap terjaga secara optimal.

Komponen *object storage* di dalam Supabase dikelola melalui sebuah wadah terisolasi yang disebut sebagai *bucket*. Fitur *bucket* ini dirancang khusus untuk menyimpan dan mengorganisasikan data tidak terstruktur berukuran besar, seperti berkas gambar, audio, maupun video, secara efisien melalui identifikasi kunci berbasis objek unik. Setiap *bucket* dapat dikonfigurasi secara fleksibel, baik dalam mode privat maupun publik, serta dilengkapi dengan sistem kendali hak akses yang ketat menggunakan aturan keamanan tingkat baris. Integrasi ini memungkinkan sistem luar untuk melakukan proses unggah, unduh, maupun manajemen berkas atau *file management* secara instan melalui pemanggilan API resmi yang didukung oleh alamat tautan *endpoint* unik.

Implementasi *storage bucket* Supabase berfungsi sebagai media penampung sementara yang aman bagi aset gambar hasil pemrosesan resolusi tinggi dalam ekosistem *cloud*. Alamat tautan publik yang dihasilkan oleh *bucket* mempermudah pengiriman berkas besar ke antarmuka *bot* komunikasi tanpa membebani memori internal sistem orkestrasi alur kerja. Karakteristik manajemen penyimpanan yang andal ini mendukung kelancaran aliran data secara asinkronus sekaligus meminimalkan risiko kehilangan berkas akibat kegagalan komputasi lokal. Bentuk identitas visual dari *platform* layanan basis data ini tertera pada Gambar 2.9.



Gambar 2.9 Logo *Platform Database Supabase*

(Sumber: https://seeklogo.com/vector-logo/435678/supabase)

2.4 Komunikasi Data dan Integrasi Sistem

Implementasi sistem *agentic workflow* yang menghubungkan berbagai layanan pihak ketiga memerlukan mekanisme pertukaran informasi yang andal. Pemahaman teknis ini mencakup arsitektur pertukaran data secara *real-time*, protokol jaringan *web*, standar format representasi data, serta metode pemformatan keluaran teks dari LLM. Seluruh komponen tersebut dikaji untuk memastikan aliran data *digital* terintegrasi secara aman, presisi, dan efisien di dalam ekosistem otomatisasi.

2.4.1 *Application Programming Interface* (API) dan *Webhook*

Application Programming Interface (API) merupakan sekumpulan aturan, protokol, dan perangkat standar yang menjembatani komunikasi antaraplikasi. Melalui API, sebuah sistem dapat mengakses fungsi atau data dari *platform* lain tanpa harus meng*et*ahui implementasi kode internalnya secara mendalam. Selain itu, API adalah sebuah teknologi yang memfasilitasi pertukaran informasi atau data antara dua atau lebih aplikasi perangkat lunak (Hanafi et al., 2017). Dalam arsitektur komputasi modern, API umumnya beroperasi menggunakan mekanisme *stateless*, di mana setiap *request* dari klien membawa informasi lengkap agar *server* dapat langsung memproses dan mengembalikan respons yang sesuai.

Berbeda dengan API konvensional yang mengharuskan sistem melakukan pengecekan data secara berkala atau *polling*, *webhook* menawarkan mekanisme komunikasi data asinkronus yang berbasis peristiwa atau *event driven*. Melalui *webhook*, *server* sumber akan mengirimkan data secara otomatis dalam bentuk berkas muatan atau *payload* langsung menuju alamat tautan tar*get* sesaat setelah suatu peristiwa spesifik terjadi. Efisiensi ini meminimalkan konsumsi sumber daya

komputasi dan memangkas waktu tunggu, karena sistem penerima tidak perlu mengirimkan permintaan berulang kali untuk mendapatkan pembaruan informasi yang paling baru dari penyedia layanan luar.

Integrasi antara API dan *webhook* dalam sistem otomatisasi memiliki peran vital untuk menjaga kelancaran alur kerja dari hulu ke hilir. Mekanisme *webhook* digunakan untuk menangkap instruksi teks dari pengguna secara seketika melalui antarmuka pesan luar, yang kemudian langsung memicu aktifnya pusat kendali orkestrasi internal. Sementara itu, komponen API bertindak sebagai eksekutor yang memanggil berbagai fungsi eksternal mulai dari penalaran model bahasa besar, pembuatan gambar, hingga proses peningkatan kualitas citra *digital*. Sinergi kedua teknologi transmisi ini memastikan bahwa parameter waktu eksekusi sistem tetap berjalan efisien dengan tingkat latensi yang minimal.

2.4.2 Protokol *Web*

Hypertext Transfer Protocol (HTTP) merupakan protokol lapisan aplikasi yang menjadi fondasi utama pertukaran data *digital* di jaringan internet *global*. HTTP juga menjadi dasar komunikasi dari *World Wide Web* (WWW), dimana HTTP merupakan aturan dalam meminta dan menjawab antara *client* dan *server* (Rosiani, 2021). Dalam arsitektur integrasi sistem, penentuan tindakan komputasi terhadap sumber daya *server* didefinisikan secara spesifik melalui metode HTTP. Metode standar yang umum digunakan meliputi *POST* untuk mengirim data baru, *GET* untuk mengambil informasi, serta *PUT* atau *PATCH* untuk memperbarui data.

Setelah menerima dan memproses metode permintaan dari klien, *server* akan mengembalikan tanggapan yang disertai dengan kode status HTTP sebagai indikator hasil eksekusi operasi tersebut. Di dalam ekosistem integrasi berbasis API, beberapa kode status memiliki urgensi teknis yang tinggi untuk memantau keberhasilan alur kerja komputasi. Kode status 200 atau OK menandakan bahwa permintaan berhasil diproses secara umum dan data dikembalikan ke klien, sedangkan kode status 201 atau *Created* memberikan konfirmasi spesifik bahwa permintaan sukses dan telah menghasilkan sumber daya baru di dalam *server*. Sebaliknya, kode status 429 atau *Too Many Requests* bertindak sebagai indikator

kritis yang menunjukkan bahwa klien telah melampaui batas kuota pemanggilan yang diizinkan oleh penyedia layanan luar dalam periode tertentu.

Untuk mengidentifikasi posisi spesifik dari sumber daya atau layanan API di internet, protokol HTTP memanfaatkan sistem pengalamatan universal yang dikenal sebagai *Uniform Resource Locator* (URL). URL menunjukan alamat dari sebuah *homepage* atau menunjukan sumber daya internet, yaitu alamat suatu dokumen atau program yang ingin ditampilkan atau digunakan (Arif, 2020). Dalam arsitektur sistem, URL bertindak sebagai alamat *digital* unik yang mengarah ke *endpoint* aplikasi tujuan. Pemanfaatan protokol *Hypertext Transfer Protocol Secure* (HTTPS) terenkripsi memastikan seluruh pertukaran teks dan berkas biner berlangsung aman, sementara validitas metode, serta kode status respons berpengaruh langsung terhadap efisiensi durasi eksekusi sistem.

2.4.3 *JavaScript* dan Format Data

JavaScript merupakan bahasa pemrograman tingkat tinggi yang telah berkembang menjadi bahasa universal di sisi *server* melalui lingkungan *runtime* modern. Fleksibilitasnya menjadi fondasi utama dalam arsitektur otomatisasi untuk mengeksekusi logika asinkron, memanipulasi berkas objek, serta mengolah pertukaran data secara efisien. Dalam praktiknya, *JavaScript* memungkinkan pengerjaan instruksi kompleks dan eksekusi skrip dinamis secara *real-time* tanpa mengganggu kelancaran proses utama sistem. Di dalam ekosistem pengaturan alur kerja, fungsionalitas ini menjembatani kalkulasi, penyaringan perintah, hingga pengondisian parameter sebelum data dialirkan ke komponen kecerdasan buatan.

Untuk memastikan pertukaran informasi antaraplikasi yang dibangun dengan bahasa pemrograman berbeda dapat berlangsung lancar, *JavaScript* melahirkan standardisasi format representasi data berbasis teks ringan yang dikenal sebagai *JavaScript Object Notation* (JSON). Format JSON bersifat independen dari bahasa pemrograman apa pun, sehingga dapat digunakan sebagai media pertukaran data antarbahasa pemrograman (Susilawati, 2022). Format ini dirancang agar mudah dibaca oleh manusia sekaligus mudah diurai oleh mesin komputasi. Struktur data di dalam berkas JSON tersusun atas kombinasi pasangan nama, serta nilai yang

merepresentasikan objek, ataupun daftar terurut yang merepresentasikan larik data. Karakteristik pertukaran data yang minimalis ini memangkas ukuran beban muatan data secara signifikan sehingga transfer pesan teks melalui internet dapat berlangsung dengan *bandwidth* yang sangat hemat.

Implementasi JavaScript dan format data JSON dalam sistem otomatisasi memiliki peran krusial sebagai media standardisasi komunikasi data dari awal hingga akhir alur kerja. Ketika sebuah instruksi dikirimkan dari antarmuka luar, data tersebut akan dikonversi menjadi dokumen JSON terstruktur agar dapat dipahami dan dipetakan oleh *platform* pengatur alur kerja. Melalui baris kode pemrograman *JavaScript* yang disematkan pada fungsi *node* internal, sistem dapat mengekstrak nilai spesifik dari dokumen JSON tersebut secara instan untuk kebutuhan manipulasi teks lanjutan sebelum dikirimkan kembali ke *server* tujuan. Pola interaksi data yang teratur ini memberikan kontribusi nyata dalam meminimalkan latensi pemrosesan dan mencegah terjadinya kegagalan akibat struktur format data yang tidak kompatibel.

2.4.4 *Structured Output Parser*

Structured output parser merupakan sebuah komponen algoritma dalam arsitektur kecerdasan buatan yang berfungsi untuk memaksa dan mengubah luaran teks tidak terstruktur dari model bahasa besar menjadi format data yang baku dan teratur. Model bahasa besar atau LLM pada dasarnya dilatih untuk menghasilkan teks generatif dalam bentuk percakapan bebas atau narasi alami manusia yang terkadang tidak konsisten. Kelemahan sifat *non* deterministik ini memicu munculnya kendala besar ketika model kecerdasan buatan tersebut diintegrasikan dengan sistem komputer lain yang membutuhkan ketepatan format data mutlak agar dapat diproses oleh baris kode program secara otomatis.

Secara teknis, mekanisme *structured output parser* bekerja dengan cara menyisipkan instruksi penstrukturan skema yang sangat ketat di dalam teks perintah atau *prompt* yang dikirimkan ke model bahasa besar. Skema tersebut mendefinisikan secara spesifik kunci, tipe data, serta batasan struktur nilai yang wajib dipenuhi oleh model saat memberikan jawaban balik. Setelah LLM

menghasilkan luaran teks, komponen *parser* ini akan secara otomatis melakukan validasi dan pembersihan teks guna memastikan bahwa data yang diterima telah terkonversi seutuhnya menjadi struktur objek JSON yang *valid* dan bebas dari teks narasi pendukung yang tidak diperlukan.

Implementasi *structured output parser* dalam sistem otomatisasi ini krusial untuk menjamin kerapian *metadata* yang dihasilkan oleh Google Gemini. Saat model menyusun judul dan kata kunci, komponen *parser* ini otomatis membersihkan luaran dari format yang berantakan, seperti tanda pagar atau *hashtag*, karakter baris baru yang tidak teratur, serta teks narasi yang tidak diperlukan. Dengan mewajibkan hasil akhir menjadi dokumen JSON yang bersih dan rapi, data *metadata* tersebut dapat langsung dipetakan secara presisi oleh *server* internal tanpa risiko kegagalan pemrosesan saat dikirimkan menuju API *platform* tujuan akhir.

2.5 *Platform Microstock*, dan Media *Digital*

Implementasi otomatisasi aset visual memerlukan pemahaman mendasar mengenai *platform* pendistribusian, strategi visibilitas *digital*, karakteristik media visual, serta pengelolaan informasi pada berkas. Bagian ini membahas landasan teori yang mencakup *microstock* dan *platform* Adobe Stock, konsep *search engine optimization* (SEO), karakteristik gambar *digital*, hingga penataan *metadata* gambar. Seluruh komponen tersebut dikaji untuk memberikan gambaran secara rinci mengenai proses penyiapan dan pengelolaan karya visual sebelum diunggah ke media *digital*.

2.5.1 *Microstock* dan *Platform* Adobe Stock

Microstock merupakan model bisnis yang mengumpulkan, melisensikan, dan mendistribusikan aset *digital,* seperti foto, ilustrasi, dan video secara massal melalui internet. Berbeda dengan lisensi gambar eksklusif tradisional yang bernilai tinggi, industri *microstock* mengandalkan volume penjualan besar dengan harga lisensi terjangkau. *Microstock* diidentifikasi sebagai jenis sistem *crowdsourcing,* yaitu suatu model pemecahan masalah secara daring oleh sejumlah besar

individu/kontributor berbentuk perpaduan antara elemen transparan dan demokratisnya *open source* (Martadireja, 2018). Berbasis konten ciptaan pengguna, para kreator *global* dapat mendaftar sebagai kontributor dan mengunggah karyanya ke *platform* untuk dipasarkan secara internasional.

Di dalam industri media *digital*, Adobe Stock hadir sebagai *platform* terkemuka yang secara resmi mengizinkan komersialisasi aset berbasis kecerdasan buatan atau AI. Keunggulan utamanya terletak pada integrasi ekosistem yang terhubung langsung secara mulus dengan aplikasi desain profesional, seperti Adobe Photoshop dan Adobe Illustrator. Integrasi matang ini memungkinkan jutaan pembeli untuk mencari dan melisensikan aset visual secara instan, sehingga menciptakan peluang komersialisasi yang tinggi bagi aset buatan manusia maupun aset hasil komputasi kecerdasan buatan.

Proses pendaftaran sebagai kontributor pada *platform* Adobe Stock melibatkan beberapa tahapan sistematis untuk memastikan akun dan aset yang dikirimkan memenuhi standar kualifikasi *platform*. Tahapan pendaftaran tersebut meliputi delapan langkah terstruktur:

1. Mengakses Halaman Pendaftaran

   Calon kontributor mengunjungi situs *web* resmi Adobe Stock contributor *https://contributor.stock.adobe.com/* untuk masuk ke dalam portal khusus kontributor.

2. Pembuatan Akun

   Pengguna mengisi formulir pendaftaran secara benar menggunakan *email* aktif, atau mengintegrasikannya lewat akun Google atau Facebook.

3. Verifikasi Akun

   Calon kontributor melakukan aktivasi akun dengan mengeklik tautan verifikasi yang dikirimkan oleh pihak Adobe Stock ke *email*.

4. Melengkapi Profil Pengguna

   Setelah akun aktif, pengguna melengkapi informasi profil, seperti nama lengkap, biografi singkat, dan tautan portofolio opsional.

5. Menentukan Jenis Konten

   Kontributor memilih kategori karya yang akan difokuskan (fotografi,

vektor, video, atau musik) sesuai dengan regulasi Adobe Stock.

6. Pengunggahan Portofolio

   Kontributor mengunggah sampel karya terbaik beresolusi tinggi yang memenuhi standar teknis *platform* melalui menu pengiriman konten.

7. Penambahan *Metadata*

   Penambahan *metadata* dilakukan dengan mengisi kolom *input* judul (*title*) dan kata kunci (*keywords*) yang tersedia pada antarmuka sistem. Pengguna wajib memasukkan informasi tersebut ke dalam formulir agar aset yang diunggah dapat teridentifikasi dan terindeks dengan baik.

8. Peninjauan dan Persetujuan

   Pihak Adobe Stock akan melakukan proses kurasi aset dengan estimasi waktu yang bervariasi, mulai dari 2 sampai 3 hari hingga beberapa minggu, sebelum karya yang disetujui secara otomatis tersedia untuk diperjualbelikan.

Meskipun mengizinkan kecerdasan buatan, Adobe Stock menerapkan kebijakan ketat yang mewajibkan kontributor mencentang kolom pernyataan pembuatan gambar menggunakan AI sebelum dikirimkan ke tim kurasi. Kegagalan mematuhi regulasi centang ini dapat berdampak pada penolakan aset secara permanen atau penutupan akun kontributor dari sistem *server*. Oleh sebab itu, pemahaman standar regulasi ini menjadi fondasi krusial bagi pengembangan alur kerja otomatisasi agar aset visual yang dihasilkan lolos verifikasi komersial secara aman. Identitas visual dari *platform* penyedia aset *digital* ini dapat dilihat pada Gambar 2.10.



Gambar 2.10 Logo *Platform* Adobe Stock

(Sumber: https://seeklogo.com/vector-logo/380168/adobe-stock)

2.5.2 *Search Engine Optimization* (SEO)

Search Engine Optimization (SEO) merupakan serangkaian proses sistematis untuk meningkatkan visibilitas dan posisi halaman *web* atau aset *digital* pada hasil pencarian organik. Dalam ekosistem *digital*, pengguna mengandalkan kata kunci spesifik untuk menemukan informasi atau produk. Tujuan dari SEO adalah menempatkan sebuah *website* pada posisi teratas hasil pencarian berdasarkan kata kunci atau *keywords* tertentu yang ditar*get*kan (Artanto & Nurdiyansyah, 2017). Melalui strategi SEO yang tepat, struktur data aset *digital* dioptimalkan agar algoritma mesin pencari dapat dengan mudah, memahami, dan mengindeks konten sebagai hasil yang relevan.

Pada implementasi pasar *digital* berskala internasional, prinsip SEO tidak hanya terbatas pada mesin pencari umum, melainkan juga diterapkan pada sistem pencarian internal *platform* agensi gambar komersial. Algoritma pencarian di dalam ekosistem ini bekerja secara otomatis untuk menyortir jutaan gambar berdasarkan tingkat kesesuaian antara kata kunci pencarian konsumen dengan *metadata* yang tertanam pada berkas citra. Penataan kata kunci yang relevan, susunan judul yang deskriptif, serta ketepatan penentuan kategori aset menjadi faktor penentu utama yang memengaruhi algoritma untuk merekomendasikan karya tersebut di halaman pertama hasil pencarian pembeli.

Integrasi optimasi SEO dalam alur kerja otomatisasi produksi aset memiliki korelasi langsung terhadap efisiensi pemasaran dan nilai komersial dari karya yang dihasilkan. Melalui penalaran agen kecerdasan buatan, proses riset tren pasar dapat dikonversikan secara instan menjadi rangkaian kata kunci strategis yang memiliki volume pencarian tinggi namun dengan tingkat kompetisi yang berimbang. Penyusunan *metadata* berbasis SEO yang rapi dan terstruktur ini memastikan aset visual hasil generasi model tidak hanya berkualitas tinggi secara estetika, tetapi juga memiliki probabilitas yang besar untuk ditemukan dan dilisensikan oleh konsumen *global*.

2.5.3 Gambar *Digital*

Gambar *digital* adalah representasi informasi visual yang dikodekan sebagai data biner, yang memerlukan dekode agar dapat dilihat di layar sebagai kisi piksel. Asal usul gambar *digital* dapat ditelusuri kembali ke awal tahun 1960, didorong oleh perkembangan di bidang-bidang, seperti kedokteran dan citra satelit, khususnya selama konteks sejarah penting, seperti perang dingin (Goodman, 2022). Selain itu, gambar *digital* dapat digunakan sebagai sarana untuk menyimpan informasi penting. Informasi penting tersebut misalnya bukti-bukti transaksi, surat-surat berharga, foto-foto yang berisikan informasi pribadi yang tidak ingin dipublikasikan, dan lain-lain (Saputra & Saragih, 2020).

Untuk tampilan *digital*, ukuran *file* dan kompatibilitas adalah faktor penting, sehingga terdapat beberapa format gambar yang direkomendasikan untuk digunakan dalam kebutuhan *digital*. Format gambar *digital* tersebut adalah sebagai berikut:

1. *Joint Photographic Experts Group* (JPEG)

   Memiliki ukuran file kecil dan kompatibel di hampir semua *platform*. Namun, kompresi *lossy*-nya dapat menurunkan kualitas gambar. Format ini ideal untuk foto *digital*, gambar *web*, dan media sosial.
2. *Portable Network Graphics* (PNG*)*

   Mendukung latar transparan dan menggunakan kompresi *lossless* untuk menjaga kualitas gambar tetap utuh. Kekurangannya ada pada ukuran *file* yang lebih besar. Format ini sangat cocok untuk logo dan ikon.
3. *Web Picture* (WebP)

   Menawarkan ukuran *file* sangat kecil dengan kualitas tinggi, serta mendukung transparansi dan animasi. Kekurangannya belum kompatibel dengan browser lama. Format ini digunakan untuk mempercepat *loading website*.

2.5.4 *Metadata* Gambar

Metadata gambar merupakan data terstruktur yang berisi informasi rinci mengenai suatu berkas citra *digital* tanpa mengubah konten visualnya. Informasi

ini mencakup aspek teknis (seperti resolusi, format berkas, serta pengaturan kamera) maupun aspek deskriptif, seperti judul, deskripsi, dan kata kunci (*keywords*). Kehadiran *metadata* ini berfungsi penting untuk mempermudah identifikasi, pengorganisasian, pencarian citra (*searchability*) pada sistem, serta menjaga riwayat dan hak cipta dari berkas gambar tersebut.

Metadata deskriptif, yang mencakup elemen, seperti judul, deskripsi, dan kata kunci, merupakan komponen standar dalam dokumentasi citra *digital* yang diatur oleh spesifikasi internasional, seperti *international press telecommunications council* (IPTC) dan *extensible metadata platform* (XMP). Dalam konteks pengembangan sistem ini, pembuatan atribut-atribut tersebut secara otomatis berfungsi sebagai mekanisme pembentukan *metadata* deskriptif yang mengubah informasi visual menjadi data terstruktur. Pengintegrasian *metadata* deskriptif ini memungkinkan sistem untuk melakukan pengindeksan, kategorisasi, dan pencarian citra (*searchability*) secara efisien dalam basis data tanpa perlu memproses ulang data piksel dari gambar.

2.6 Perangkat Pengujian dan Sistem Pembanding

Analisis kinerja sistem otomatisasi memerlukan indikator pembanding untuk mengukur tingkat efisiensi alur kerja dan kualitas aset yang dihasilkan. Bagian ini menguraikan landasan teoritis mengenai perangkat peningkat kualitas gambar berbasis *desktop*, *platform* generasi visual Kling AI, serta aplikasi pengujian integrasi antarmuka pemrograman. Seluruh instrumen tersebut dikaji untuk memberikan dasar objektif dalam mengevaluasi parameter waktu eksekusi dan efisiensi operasional dari sistem otomatisasi yang dikembangkan.

2.6.1 VideoProc Converter AI

VideoProc Converter AI merupakan perangkat lunak multimedia berbasis kecerdasan buatan untuk meningkatkan kualitas, mengonversi, dan memproses video serta gambar (Digiarty, 2025). Perangkat lunak ini mampu merestorasi citra lama yang buram menjadi lebih tajam, memperhalus transisi gerakan visual, serta menghidupkan kembali warna foto yang pudar melalui dukungan teknologi

akselerasi perangkat keras yang responsif. Di dalam operasionalnya, proses peningkatan resolusi atau *upscaling* pada perangkat ini dapat diterapkan secara luas pada berbagai format masukan gambar populer, antara lain format *portable network graphics* (PNG), *joint photographic experts group* (JPG atau JPEG), *tagged image file format* (TIFF), *web picture* (WEBP), *bitmap image file* (BMP). Fungsionalitas tersebut melengkapi fitur pendukung lainnya, seperti konversi format media, kompresi ukuran *file*, perekaman aktivitas layar, serta alat penyuntingan multimedia secara cepat.

Pemanfaatan VideoProc Converter AI berfungsi sebagai indikator pembanding konvensional tingkat lokal untuk mengevaluasi efisiensi sistem. Kecepatan eksekusi kecerdasan buatan pada perangkat *desktop* ini sangat bergantung pada unit pemrosesan, di mana pemrosesan berbasis *Central Processing Unit* (CPU) memerlukan durasi jauh lebih lama akibat keterbatasan inti paralel dibandingkan dengan *Graphics Processing Unit* (GPU). Dalam aspek operasional perangkat lunak, optimalisasi kinerja komputasi dan kapasitas beban kerja juga ditentukan oleh skema distribusi lisensi, di mana versi gratis memiliki batasan kecepatan, serta pembatasan jumlah unggahan maksimal 5 foto dalam sekali eksekusi, sementara kapasitas massal tanpa batasan hanya didukung pada versi premium.

2.6.2 Kling AI

Kling AI merupakan sebuah *platform* AI generatif berbasis *cloud* yang dirancang khusus untuk memproduksi aset visual berkualitas tinggi dari instruksi teks. Kling AI adalah model generatif berbasis kecerdasan buatan yang dirancang untuk menghasilkan video melalui teknik pembelajaran mendalam (Wikipedia, 2026). *Platform* ini dikembangkan menggunakan model difusi tingkat lanjut yang mampu menginterpretasikan *prompt* pengguna secara presisi. Tanpa mengandalkan spesifikasi perangkat keras komputer lokal, seluruh proses *render* diproses pada infrastruktur *server* eksternal yang diakses secara instan melalui *web browser*.

Secara teknis, proses pembuatan citra pada Kling AI ditenagai oleh arsitektur penalar kecerdasan buatan berskala besar yang berjalan di atas klaster *server*

komputasi berperforma tinggi. Ketika pengguna memasukkan deskripsi teks, sistem kecerdasan buatan akan menganalisis semantik kata tersebut, memetakan relasi antar objek, lalu menyusun matriks piksel gambar baru dengan komposisi pencahayaan, serta *detail* tekstur yang tajam. Seluruh pemrosesan data tersebut dieksekusi secara asinkronus di sisi *cloud* penyedia layanan, sehingga pengguna cukup menunggu hasil akhir tanpa memisahkan konsumsi memori utama komputer lokal.

Pemanfaatan Kling AI berfungsi sebagai indikator pembanding komparatif berbasis *cloud* untuk mengevaluasi efisiensi alur kerja. Kinerja komputasi dan batasan antrean ditentukan oleh skema lisensi, di mana versi gratis memiliki keterbatasan, karena menghasilkan luaran gambar yang memiliki *watermark*, sementara hasil bersih tanpa tanda air hanya didukung pada versi premium berbayar. Karakteristik operasional tersebut mengharuskan pengguna membuka situs *web* secara konstan untuk melakukan proses masuk akun, pengetikan perintah teks, penyesuaian rasio aspek, serta pengunduhan *file* gambar satu per satu melalui antarmuka peramban.

2.6.3 Postman

Postman adalah sebuah perangkat lunak klien *Representational State Transfer* (REST) berbasis *web* yang tersedia sebagai ekstensi pada Google Chrome (Raharja & Wijayanto 2023). Postman merupakan sebuah *platform* aplikasi yang dirancang khusus untuk menyederhanakan siklus pengembangan, pengujian, dan dokumentasi antarmuka pemrograman aplikasi atau API. Perangkat lunak ini menyediakan antarmuka grafis yang intuitif bagi pengembang untuk membangun, mengirim, dan menganalisis permintaan jaringan tanpa harus menulis baris kode program pengujian secara manual. Di dalam arsitektur integrasi sistem modern, keberadaan alat pengujian jenis ini sangat krusial untuk memastikan, bahwa setiap titik ujung komunikasi dapat merespons paket data secara benar sebelum fungsi tersebut diintegrasikan ke dalam pusat kendali otomatisasi yang lebih besar.

Secara teknis, fungsionalitas Postman beroperasi dengan cara mensimulasikan perilaku klien saat melakukan interaksi pengiriman data menuju

komponen *server* tar*get*. Pengguna dapat menentukan metode HTTP yang ingin diuji, menyusun parameter tautan URL, serta menyisipkan berkas muatan berbentuk struktur dokumen JSON maupun objek biner secara fleksibel. Setelah permintaan tersebut dikirimkan, Postman akan menangkap pesan respons secara utuh, menampilkan kode status yang dikembalikan, serta mengukur durasi waktu tanggapan jaringan secara presisi. Kapasitas visualisasi data yang transparan ini mempermudah proses deteksi kesalahan konfigurasi atau kendala batasan waktu eksekusi pada layanan pihak ketiga secara instan. Identitas visual dari *platform* pengujian API ini diperlihatkan pada Gambar 2.11.



Gambar 2.11 Logo *Platform* Pengujian Postman

(Sumber: https://dashboardicons.com/icons/postman)

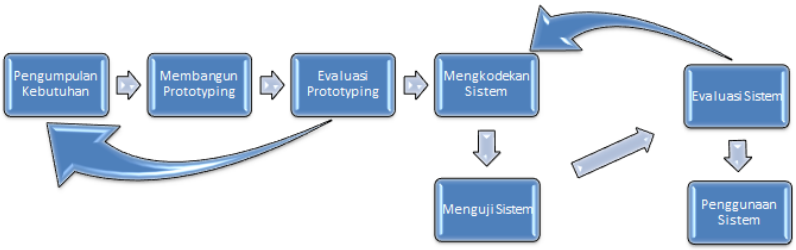
2.7 Metode *Prototyping*

Metode *prototyping* merupakan pendekatan rekayasa perangkat lunak yang berfokus pada pembuatan rancangan awal sistem secara cepat dan berulang sebelum pengembangan produk akhir. Versi awal ini berfokus pada representasi visual dan interaksi yang memungkinkan pengguna memahami konsep dan fungsionalitas sistem (Kustanto, Khalil & Noe'man 2024). Berbeda dengan model sekuensial kaku, seperti *waterfall*, pendekatan ini fleksibel dalam mengadaptasi perubahan kebutuhan teknis. Interaksi langsung terhadap model kerja ini membantu mendeteksi kesalahan konfigurasi dan kendala integrasi lebih awal, sehingga meminimalkan risiko pembengkakan biaya perbaikan.

Secara teknis, implementasi metode ini mengacu pada lima tahapan siklus yang saling berkesinambungan. Proses dimulai dari tahap komunikasi untuk menyamakan persepsi mengenai tujuan umum sistem, lalu diikuti perencanaan

cepat untuk membatasi prioritas pengembangan. Langkah berikutnya adalah pemodelan rancangan cepat guna menyusun cetak biru aliran data, dilanjutkan dengan pembangunan prototipe menjadi sistem fungsional yang siap diuji. Siklus ini diakhiri oleh tahap penerapan, penyerahan, dan umpan balik, di mana hasil evaluasi dari model kerja tersebut dijadikan dasar untuk merencanakan perbaikan pada siklus berikutnya hingga seluruh fungsionalitas mencapai standar yang diinginkan.

Penerapan metode ini didasari oleh arsitektur agen kecerdasan buatan berbasis n8n yang bergantung pada layanan pihak ketiga. Komponen penting, seperti Google Gemini, FLUX 1.1 Pro, dan Real-ESRGAN tidak dapat dioptimalkan secara teoritis, melainkan harus dikalibrasi parameternya secara langsung melalui uji coba. Pengujian format instruksi (*prompt*) dan waktu tanggap (*latency*) API eksternal memerlukan penyesuaian secara berkala untuk mendapatkan kombinasi performa yang paling efisien. Oleh karena itu, metode ini dipilih, karena menyediakan kerangka kerja fleksibel untuk menguji keandalan sistem otomatisasi berbasis *cloud* ini secara bertahap. Ilustrasi tahapan umum metode ini dapat dilihat pada Gambar 2.12.



Gambar 2.12 Alur Metode Pengembangan *Prototyping*

(Sumber: https://medium.com/dot-intern/sdlc-metode-prototype-8f50322b14bf)

2.8 Metode Eksperimental

Metode eksperimental dalam rekayasa perangkat lunak adalah pendekatan yang berfokus pada pengujian hubungan sebab-akibat antarvariabel melalui

manipulasi dan pengukuran terukur. Penelitian eksperimen merupakan satu-satunya tipe penelitian yang lebih akurat/teliti dibandingkan dengan penelitian lain, dalam menentukan relasi hubungan sebab akibat (Akbar et al., 2023). Landasan teori ini digunakan untuk menganalisis perilaku arsitektur perangkat lunak, efisiensi eksekusi data, serta keandalan infrastruktur berbasis *cloud* secara objektif, sehingga karakteristik teknis alur kerja *digital* yang dibangun dapat dipetakan secara terukur.

Metode ini digunakan untuk menggambarkan struktur performa dan perilaku operasional sistem secara kuantitatif, sehingga proses analisis efisiensi alur kerja menjadi lebih mudah dipahami. Pendekatan eksperimen juga banyak diterapkan pada sistem otomatisasi, karena mampu menyediakan landasan penilaian yang jelas dan terstandarisasi. Beberapa metrik evaluasi kinerja yang umum digunakan dalam metode ini dilakukan melalui pengukuran waktu tunda (*latency*), jumlah pemrosesan data (*throughput*), dan hambatan alur (*bottleneck*).

2.8.1 *Latency*

Latency adalah parameter yang mendefinisikan jeda waktu atau durasi yang mengalir sejak sebuah permintaan transmisi data dikirimkan oleh pengirim hingga respons penuh diterima kembali oleh penerima. Secara operasional, pengukuran latensi pada integrasi layanan berbasis *web* dapat dihitung menggunakan konsep waktu perjalanan pulang-pergi atau *round trip time* (RTT). Pendekatan ini mengukur total waktu yang dihabiskan sejak permintaan dikirimkan oleh klien hingga respons kembali diterima secara utuh, yang melibatkan akumulasi dari latensi permintaan dan latensi jawaban. Formula matematis untuk menghitung nilai latensi operasional tersebut didefinisikan sebagai berikut (Indusface, 2026).

$$Latency_{RTT} = L_{req} + L_{res} \tag{2.1}$$

Untuk memetakan komponen penundaan secara mendetail di sepanjang jalur transmisi, formula di atas dapat dipecah berdasarkan selisih stempel waktu pada setiap titik interaksi yang ditunjukkan pada Persamaan berikut (Indusface, 2026).

$$Latency_{RTT} = (T_{sub} - T_{init}) + (T_{rec} - T_{send}) \quad (2.2)$$

Keterangan:

a. $Latency_{RTT}$: Total latensi perjalanan pulang-pergi.
b. L_{req} : Latensi durasi permintaan.
c. L_{res} : Latensi durasi jawaban.
d. T_{init} : Stempel waktu kirim awal dari klien (nilai awal).
e. T_{sub} : Stempel waktu terima di sistem perantara (pengiriman).
f. T_{send} : Stempel waktu kirim balik dari *server* asal (pengiriman).
g. T_{rec} : Stempel waktu terima akhir di klien (diterima).

Nilai rata-rata latensi diperoleh dengan membagi total latensi yang dihasilkan selama pengujian dengan jumlah sampel yang diuji (Kristiyanto, Primananda & Kartikasari 2019). Rumus perhitungan rata-rata latensi dapat dituliskan sebagai berikut.

$$\text{Rata} - \text{rata } Latency = \frac{Total\ Latency}{n} \quad (2.3)$$

Keterangan:

a. Total $Latency$: Jumlah keseluruhan nilai latensi hasil pengujian.
b. n : Jumlah sampel pengujian.

Pengukuran rata-rata latensi ini berfungsi sebagai indikator utama untuk menilai tingkat responsivitas alur kerja *digital* yang dibangun. Nilai rata-rata yang semakin kecil menunjukkan, bahwa proses transmisi data dan eksekusi instruksi berjalan dengan semakin cepat dan efisien.

2.8.2 *Throughput*

Throughput merupakan parameter teoretis yang mendefinisikan volume muatan data yang berhasil ditransmisikan dan diproses secara benar dari pengirim menuju penerima dalam satu satuan waktu. Karakteristik metrik ini ditentukan oleh

rasio antara volume keluaran yang dihasilkan terhadap durasi waktu pengamatan yang berjalan. Semakin banyak tugas yang berhasil diselesaikan secara tuntas dalam rentang waktu yang semakin singkat, maka efisiensi kapasitas produksi sistem akan meningkat secara signifikan. Hubungan timbal balik antara kecepatan transfer dan waktu tunda ini dapat diukur secara empiris melalui perhitungan matematis yang terstruktur. Berikut persamaan untuk menentukan nilai *throughput* dari latensi.

$$Throughput = \frac{\text{ukuran data terima } (bit)}{latency\ (detik)} \tag{2.4}$$

Pengujian terhadap parameter ini berfungsi memetakan batas kemampuan tertinggi suatu rancangan sebelum dioperasikan secara massal. Perhitungan nilai *throughput* diperoleh setelah nilai latensi, sehingga kedua metrik ini memiliki hubungan erat. Melalui formulasi matematika yang tepat, data keluaran metrik ini memberikan landasan argumen yang *valid* mengenai skala produktivitas arsitektur yang sedang diteliti.

2.8.3 *Bottleneck*

Bottleneck adalah kondisi di mana performa dan kelancaran seluruh sistem mengalami penurunan drastis akibat adanya satu komponen atau *node* yang memiliki kapasitas pemrosesan lebih rendah daripada laju aliran data yang masuk. Fenomena teoretis ini memicu penumpukan antrean data pada satu titik spesifik, sehingga memaksa komponen lain yang memiliki performa lebih cepat untuk berada dalam posisi menunggu. Identifikasi titik sumbat sangat penting untuk memetakan letak hambatan utama yang membatasi efisiensi operasional sistem secara keseluruhan.

Untuk mendeteksi letak komponen yang menjadi penyebab hambatan, analisis dilakukan dengan mengukur seberapa besar sumbangan durasi waktu dari tiap *node* terhadap total waktu eksekusi. Berdasarkan metode analisis kontribusi waktu tersebut, formula dasar untuk menghitung nilai persentase durasi waktu dari suatu komponen spesifik dinyatakan pada persamaan berikut:

$$P_{\text{komponen}} = \frac{\text{T}_{\text{komponen}}}{L_{\text{total}}} \times 100\% \qquad (2.5)$$

Keterangan:

a. P_{komponen} : Persentase kontribusi durasi waktu komponen.
b. T_{komponen} : Durasi waktu eksekusi pada komponen spesifik.
c. L_{total} : Total latensi waktu tunggu seluruh sistem.

Melalui perhitungan persentase ini, komponen yang paling banyak menyita waktu eksekusi dapat diidentifikasi secara akurat sebagai titik sumbat utama. Hasil evaluasi numerik ini memberikan landasan objektif untuk melakukan optimalisasi atau penyesuaian parameter pada *node* yang terbukti membatasi nilai *throughput* sistem.

2.8.4 Efisiensi Waktu Produksi

Pengujian efisiensi waktu produksi merupakan metode evaluasi kinerja sistem yang bertujuan untuk mengukur sejauh mana penerapan otomatisasi mampu memangkas durasi pemrosesan siklus kerja dalam menghasilkan suatu luaran dibanding proses manual. Efisiensi waktu diukur dengan membandingkan total waktu dari awal hingga akhir yang dibutuhkan sistem, mulai dari penerimaan instruksi masukan, eksekusi alur kerja, hingga tersedianya produk akhir, terhadap standar durasi pekerjaan operasional. Secara teoretis, penerapan teknologi. seperti otomatisasi berbasis alur kerja dan integrasi antarmuka pemrograman aplikasi dirancang untuk mengeliminasi waktu tunggu, mengurangi langkah yang berulang, serta mempercepat distribusi aset secara konsisten. Oleh karena itu, pengujian ini menjadi indikator utama dalam mengukur potensi penghematan durasi operasional pada alur kerja produksi *digital*.

Dalam konteks pembuatan aset kreatif dan *digital*, pengukuran efisiensi waktu dilakukan dengan mencatat durasi eksekusi dari setiap tahap pemrosesan pada berbagai tingkat kompleksitas instruksi pengguna. Peningkatan kecepatan

produksi tidak hanya dinilai dari performa individual setiap modul, melainkan dari akumulasi keseluruhan proses pemrosesan instruksi, generasi visual, hingga tahap penyempurnaan resolusi gambar. Hasil evaluasi efisiensi waktu ini memberikan landasan kuantitatif untuk menilai kestabilan sistem dalam menangani variasi beban kerja, sekaligus memvalidasi bahwa sistem otomatisasi mampu meningkatkan tingkat luaran per satuan waktu secara signifikan tanpa menurunkan standar kualitas produk yang dihasilkan.

2.8.5 Usaha Pengembangan (*Effort*)

Pengujian atau evaluasi usaha pengembangan merupakan bentuk analisis kuantitatif dan kualitatif untuk mengukur besarnya alokasi sumber daya manusia, waktu, serta tingkat kerumitan interaksi manual yang dapat dikurangi setelah sistem otomatisasi diimplementasikan. Dalam pengembangan perangkat lunak dan otomatisasi alur kerja, usaha pengembangan umumnya direpresentasikan melalui jumlah langkah kerja manual, durasi keterlibatan pengguna secara langsung, serta kompleksitas teknis yang harus ditangani oleh pembuat konten. Pengukuran ini penting untuk membuktikan bahwa integrasi agen kecerdasan buatan dan titik otomatisasi berhasil mentransformasi tugas konvensional yang repetitif menjadi alur kerja yang lebih ringkas dan terstruktur.

Pengujian usaha pengembangan dilakukan dengan membandingkan skenario kerja berbasis interaksi manual penuh terhadap skenario kerja otomatis yang digerakkan oleh sistem dari awal hingga akhir. Penurunan nilai usaha pengembangan tercermin ketika pengguna hanya perlu memberikan satu instruksi awal, sementara seluruh tahapan kompleks, seperti pengayaan instruksi, pengunduhan berkas, pemrosesan media, hingga penyimpanan ke direktori akhir dieksekusi secara mandiri oleh sistem. Dengan terbuktinya efisiensi usaha pengembangan, sistem tidak hanya menekan potensi kesalahan manusia, tetapi juga mengoptimalkan produktivitas kerja dengan mengalihkan fokus pengguna dari pekerjaan teknis yang sifatnya rutin ke aktivitas yang lebih bernilai strategis.

2.9 *Black Box Testing*

Black box testing merupakan metode validasi perangkat lunak yang berfokus sepenuhnya pada fungsionalitas eksternal sistem tanpa memeriksa kode program internalnya. Terkait hal tersebut, pengujian *black box* dimaksudkan untuk meng*et*ahui apakah fungsi masukan dan keluaran dapat berjalan dengan baik dan sesuai dengan yang diharapkan, jika fungsi masukan sesuai dengan keluaran, maka fungsi tersebut dinyataan *valid* (Masripah & Ramayanti, 2019). Metode ini membantu memastikan sistem merespons data masukan dengan tepat, mendeteksi kesalahan antarmuka, serta mengamati kesesuaian luaran berdasarkan spesifikasi kebutuhan yang telah ditetapkan.

Dalam penerapannya, pengujian kotak hitam dilakukan dengan menggunakan dua skenario utama, yaitu pengujian dengan data yang benar dan data yang salah. Pengujian dengan data yang benar bertujuan untuk memastikan, bahwa sistem dapat memproses perintah pengguna dan memicu respons otomatis dengan lancar. Sementara itu, pengujian dengan data yang salah dilakukan untuk melihat kemampuan sistem dalam menahan masukan yang tidak sesuai, serta memastikan bahwa fitur penanganan kesalahan dapat memunculkan peringatan kegagalan secara tepat tanpa membuat sistem berhenti bekerja secara mendadak.

2.10 *System Information Leakage*

System Information Leakage atau kebocoran informasi sistem merupakan kondisi di mana *detail* internal sistem terungkap kepada pihak yang tidak berwenang. Keamanan sistem informasi mencakup perlindungan terhadap pengungkapan *detail* internal yang tidak perlu kepada pihak luar. Masalah yang umum terjadi adalah ketika pesan kesalahan internal, seperti jejak tumpukan (*stack trace*) atau *database dump* ditampilkan kepada pengguna. Untuk mencegah hal tersebut, situs harus merespons dengan hasil yang dirancang khusus yang bermanfaat bagi pengguna tanpa mengungkapkan *detail* internal yang tidak perlu (OWASP, 2025). Informasi internal yang tersembunyi mencegah kebocoran potensi kelemahan sistem sekaligus menjaga kenyamanan antarmuka pengguna.

Secara teknis, kebocoran informasi sistem terjadi ketika sebuah perangkat

lunak secara tidak sengaja mengekspos *detail* konfigurasi internal, seperti variabel lingkungan (*environment variables*), pengaturan sistem, atau informasi pelacakan kesalahan (*debugging*), kepada pihak yang tidak berwenang. Pengungkapan yang tidak disengaja ini dapat memberikan wawasan berharga kepada penyerang untuk memetakan arsitektur infrastruktur, sehingga mempermudah perencanaan dan pelaksanaan tindakan eksploitasi lebih lanjut.

2.11 *Unified Modeling Language* (UML)

Unified Modeling Language (UML) adalah metode pemodelan secara visual untuk sarana perancangan sistem berorientasi objek, atau definisi UML, yaitu sebagai suatu bahasa yang menjadi standar pada visualisasi, perancangan dan juga pendokumentasian sistem (Affandi & Syahputra, 2018). Melalui pendekatan ini, pengembang dapat menggambarkan kebutuhan fungsional, aliran proses bisnis, interaksi pengguna terhadap sistem, serta skema komunikasi antar-komponen teknis secara gamblang sebelum tahap implementasi kode program dilakukan.

Melalui pemodelan visual ini, struktur statis maupun perilaku dinamis dari sistem dapat dipetakan dengan jelas, sehingga proses analisis kebutuhan perancangan menjadi lebih mudah dipahami oleh seluruh pihak yang terlibat. Dalam pengembangan sistem integrasi berbasis API, dokumentasi terstandarisasi ini sangat penting untuk memastikan seluruh komponen modular dapat terhubung dengan selaras. Beberapa jenis diagram UML yang digunakan sebagai landasan perancangan sistem ini meliputi *Use Case Diagram*, *Activity Diagram*, dan *Sequence Diagram*.

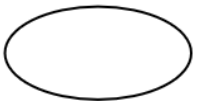
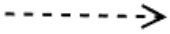
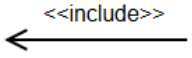
2.11.1 *Use Case Diagram*

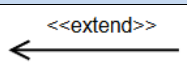
Use Case Diagram merupakan diagram yang menjelaskan manfaat sistem dari sudut pandang orang yang berada di luar sistem atau *actor* (Laipaka, 2017). Model visual ini memberikan gambaran ringkas mengenai hubungan fungsionalitas yang disediakan oleh sistem dengan subjek luar yang berinteraksi dengannya. Melalui diagram ini, pengembang dapat memetakan kebutuhan utama, serta batasan operasional perangkat lunak secara *global*, tanpa harus menjabarkan *detail* teknis

atau algoritma internal pemrosesan data di dalamnya.

Relasi yang digunakan dalam diagram ini meliputi asosiasi, ketergantungan, dan generalisasi. Asosiasi menunjukkan interaksi langsung antara aktor dan *Use Case* melalui garis penghubung. Relasi ketergantungan terbagi menjadi *include* untuk pemanggilan fungsionalitas yang bersifat wajib, serta *extend* untuk pemanggilan yang hanya terjadi pada kondisi khusus tertentu. Sementara itu, generalisasi merupakan hubungan pewarisan antar-komponen sejenis, di mana salah satu komponen dapat mewarisi atau memodifikasi sifat dari komponen dasar lainnya. Simbol penotasian beserta penjelasan dari setiap komponen dan relasi di dalam diagram ini disajikan secara terperinci pada Tabel 2.1.

Tabel 2.1 Simbol *Use Case Diagram*

No	Simbol	Nama	Keterangan
1		*Actor*	Mewakili peran pengguna atau sistem luar saat berinteraksi dengan fungsi sistem.
2		*Use case*	Menunjukkan fungsi atau layanan utama yang disediakan oleh sistem kepada pengguna.
3		*Association*	Berfungsi sebagai garis penghubung interaksi antara aktor dengan fungsi sistem tertentu.
4		*Generalization*	Menunjukkan hubungan turunan atau spesialisasi fungsi antar elemen yang sejenis
5		*Include Relationship*	Menandakan bahwa suatu fungsi sistem wajib menjalankan fungsi sistem lainnya agar proses selesai.
6		*Extend*	Menandakan adanya fungsi tambahan

No	Simbol	Nama	Keterangan
		*Relationship*	pilihan yang akan berjalan jika kondisi tertentu terpenuhi.

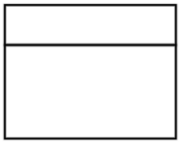
2.11.2 *Activity Diagram*

Activity Diagram merupakan diagram yang menggambarkan aliran aktivitas atau urutan proses kerja di dalam sistem secara sekuensial. Pemodelan visual ini berfokus pada pemetaan logika internal, titik percabangan kondisi, jalur eksekusi paralel, hingga kondisi akhir dari suatu siklus kerja. Setiap *use case* minimal terdapat satu *activity diagram* (Narulita et al., 2024). Dalam sistem otomatisasi, diagram ini sangat krusial untuk memetakan bagaimana data masukan mengalir dari satu *node* pemrosesan menuju *node* berikutnya secara otomatis.

Komponen utama yang menyusun diagram ini meliputi titik awal (*initial state*), aktivitas (*activity*), percabangan keputusan (*decision*), penggabungan atau pemecahan jalur (*fork/join*), serta titik akhir (*final state*). Seluruh elemen tersebut digunakan untuk menggambarkan bagaimana sistem merespons masukan dan mengelola percabangan logika ketika menghadapi kesalahan integrasi. Simbol penotasian beserta penjelasan dari setiap komponen dasar di dalam diagram ini secara terperinci disajikan pada Tabel 2.2.

Tabel 2.2 Simbol *Activity Diagram*

No	Simbol	Nama	Keterangan
1		*Initial* State	Menandakan titik awal dimulainya alur dalam diagram aktivitas.
2		*Action*	Menunjukkan suatu tindakan atau aktivitas pekerjaan yang dilakukan oleh sistem.

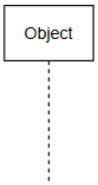
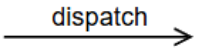
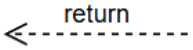
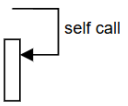
No	Simbol	Nama	Keterangan
3		*Join*	Berfungsi menggabungkan kembali beberapa alur aktivitas yang berjalan bersamaan menjadi satu alur.
4		*Final State*	Menandakan titik akhir atau selesainya seluruh alur aktivitas sistem.
5		*Swimlane*	Memisahkan kolom pengelompokan aktor atau komponen sistem yang bertanggung jawab atas aktivitas tertentu.

2.11.3 *Sequence Diagram*

Sequence Diagram adalah jenis diagram UML yang digunakan untuk menggambarkan interaksi antar-objek atau komponen sistem berdasarkan kronologi urutan waktu terjadinya komunikasi tersebut. *Sequence diagram* menggambarkan bagaimana peristiwa (aktivitas) dalam *use case* yang dipetakan ke dalam operasi kelas objek dalam *class diagram* (Wayahdi & Ruziq 2023). Diagram ini secara terperinci menampilkan pengiriman pesan yang mengalir secara kronologis di antara *lifeline* setiap komponen. Melalui pemodelan visual ini, pengembang dapat memahami aliran data, logika *error handling*, serta urutan pemanggilan pertukaran data antarmuka pemrograman aplikasi atau API eksternal secara berurutan.

Pemodelan dengan diagram ini bermanfaat untuk memetakan skenario interaksi data dan mendeteksi keterlambatan respons antar-API secara kronologis. Melalui penggambaran visual yang terstruktur, pengembang dapat memastikan alur pengiriman pesan berjalan logis, serta mekanisme penanganan kesalahan berfungsi tepat saat sistem mengalami kegagalan. Notasi standar yang digunakan untuk menggambarkan seluruh proses tersebut, seperti *actor*, *lifeline*, *activation bar*, *synchronous message*, *dispatch message*, *asynchronous message*, hingga *self call*, secara terperinci disajikan pada Tabel 2.3.

Tabel 2.3 Simbol *Sequence Diagram*

No	Simbol	Nama	Keterangan
1		*Actor*	Merepresentasikan pengguna atau sistem luar yang berinteraksi dengan alur sistem.
2		*Lifeline*	Menunjukkan masa hidup atau keberadaan suatu objek selama proses interaksi berlangsung.
3		*Activation bar*	Menunjukkan bahwa suatu objek sedang aktif dalam melakukan suatu proses komputasi.
4		*Synchronous message*	Menggambarkan pengiriman pesan yang mengharuskan pengirim menunggu umpan balik selesai diproses.
5		*Dispatch message*	Menggambarkan pengiriman pesan pemicu untuk membuat atau menginstansiasi objek baru di dalam sistem.
6		*Asynchronous messsage*	Menunjukkan nilai kembalian (*return value*) ke objek penerima dengan arah panah tertuju pada objek tersebut.
7		*Self call*	Menggambarkan sebuah objek yang mengirim pesan atau memanggil fungsi operasi ke dirinya sendiri

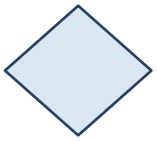
2.12 *Flowchart*

Flowchart atau bagan alir merupakan pemodelan visual yang digunakan untuk menggambarkan langkah-langkah penyelesaian masalah atau urutan proses kerja secara sekuensial dan sistematis. *Flowchart* juga menggambarkan urutan logis dari suatu prosedur pemecahan masalah, sehingga *flowchart* dapat dipahami sebagai langkah-langkah pemecahan masalah yang ditulis dalam simbol-simbol tertentu (Khesya, 2021). Dalam sistem otomatisasi, pemodelan ini membantu memetakan logika algoritma dan aliran data melalui simbol geometris standar untuk memberikan gambaran logis mengenai bagaimana instruksi dieksekusi secara berurutan oleh setiap komponen modular.

Flowchart berfungsi penting dalam memvisualisasikan alur pengembangan perangkat lunak, mulai dari analisis kebutuhan hingga implementasi. Selain itu, alat ini membantu memperjelas logika program saat pengkodean, mendukung efisiensi pengujian kesalahan, serta berfungsi sebagai dokumentasi sistem yang mudah dipahami. Simbol geometris standar beserta penjelasan fungsi dari setiap komponen dasar yang digunakan di dalam bagan alir ini secara terperinci disajikan pada Tabel 2.4 berikut.

Tabel 2.4 Simbol *Flowchart*

No	Simbol *Flowchart*	Nama	Keterangan
1		*Terminator*	Menandakan titik awal atau akhir dari alur program.
2		*Process*	Menunjukkan proses pengolahan data atau eksekusi perintah komputer.
3		*Document*	Menyatakan masukan atau keluaran dalam bentuk dokumen cetak.

No	Simbol *Flowchart*	Nama	Keterangan
4		*Decision*	Menunjukkan pilihan keputusan untuk menentukan jalur program selanjutnya.
5		Data	Menggambarkan proses memasukkan data atau menampilkan hasil keluaran.
6		*On-page Reference/Connector*	Penghubung alur diagram alir yang terputus pada halaman sama.
7		*Off-Page Reference/ Off-Page Connector*	Penghubung alur diagram alir yang terputus pada halaman berbeda.
8		*Flow*	Tanda panah penunjuk arah aliran jalannya proses sistem.

2.13 Penelitian Terdahulu

Bagian ini membahas beberapa penelitian sebelumnya yang memiliki keterkaitan dengan topik sistem otomatisasi ini. Berbagai studi tersebut berfungsi sebagai dasar teori dan acuan untuk memahami metode yang telah diterapkan sebelumnya. Melalui kajian ini, dapat ditemukan berbagai temuan yang mendukung pengembangan alur kerja produksi aset visual yang lebih efisien. Perbandingan penelitian yang relevan dapat dilihat pada Tabel 2.5.

Tabel 2.5 Penelitian Terdahulu

No	Judul	Peneliti	Hal yang Dianalisis	Kekurangan	Kelebihan
1	*Self-Hosted*	Hareign	Penerapan	Integrasi n8n	Memanfaatka

No	Judul	Peneliti	Hal yang Dianalisis	Kekurangan	Kelebihan
	Workflow Automation for AI-Based Cybersecurity Operation, Journal of The Colloquium for Information Systems Security Education, vol. 13, no. 1, p. 21, 2026. https://doi.org/10.53735/cisse.v13i1.241	Casaclang, Bianca Ionescu, Yoohwan Kim, dan Ju-Yeon Jo	n8n sebagai *platform* otomasi *workflow self-hosted* yang diintegrasikan dengan LLM dan *tool* keamanan untuk operasional *cybersecurity.*	dan AI berfokus pada analisis teks, serta pemindaian keamanan siber, belum menyentuh otomasi produksi aset visual, pemrosesan citra (*upscaling*), maupun riset pasar *microstock*.	n *platform* n8n secara *self-hosted* yang terintegrasi dengan LLM untuk mengotomatisasi alur kerja kompleks, memangkas waktu pemrosesan, serta menekan biaya operasional secara signifikan.
2	Implementasi *Chatbot AI* Berbasis n8n untuk Layanan Informasi Administrasi Akademik Departemen	Intan Nur Rahma Hariyadi dan Dila Umnia Soraya	Kinerja *chatbot* AI berbasis n8n, Gemini, dan Telegram untuk layanan akademik.	Ruang lingkup implementasi sistem masih terbatas pada fungsi layanan penataan informasi	Berhasil menguji performa nyata integrasi n8n, Google Gemini, dan Telegram menggunaka

No	Judul	Peneliti	Hal yang Dianalisis	Kekurangan	Kelebihan
	Teknik Elektro dan Informatika. JEPIN (Jurnal Edukasi dan Penelitian Informatika), vol. 11, no. 3, pp. 442-447, DOI: 10.26418/jp.v11i3.101635.			teks statis (tanya-jawab), bukan pada otomatisasi alur kerja produksi dan distribusi aset visual.	n pengujian SUS dengan hasil kepuasan sistem yang sangat tinggi.
3	*Murakkab: Resource-Efficient Agentic Workflow Orchestration in Cloud Platforms*, arXiv preprint, 2025. https://doi.org/10.48550/arXiv.2508.18298	Gohar Irfan Chaudhry, Esha Choukse, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Adam Belay, dan Ricardo Bianchini	Perancangan sistem orkestrasi *Murakkab* untuk mengoptimalkan efisiensi alokasi sumber daya dan latensi eksekusi pada *agentic workflow*	Sistem yang dikembangkan berorientasi pada optimasi infrastruktur *low-level cloud scheduling*, bukan pada kemudahan pembuatan alur kerja tingkat aplikasi.	Menyediakan analisis performa mendalam terkait titik keseimbangan (*trade-offs*) antara akurasi, latensi, dan efisiensi biaya pada orkestrasi *cloud*.

No	Judul	Peneliti	Hal yang Dianalisis	Kekurangan	Kelebihan
			berbasis *cloud*.		
4	Rancang Bangun Buku Tamu *Digital* Menggunakan Otomasi N8n Dengan Metode Agile, JATI (Jurnal Mahasiswa Teknik Informatika), vol. 10, no. 2, pp. 3106-3113, 2026. https://doi.org/10.36040/jati.v10i2.17927	M. Faisal Zhafran Farros dan Anggay Luri Pramana	Perancangan buku tamu *digital* berbasis n8n untuk otomasi pencatatan, QR *Code*, dan notifikasi.	Penggunaan n8n masih terbatas pada otomasi administrasi dasar, belum mengintegrasikan *Agentic AI*, generasi aset visual, pembuatan *metadata*, maupun interaksi *single-trigger*.	Memanfaatkan n8n untuk mengotomatisasi alur kerja pencatatan buku tamu, pemrosesan basis data, pembuatan QR Code, dan notifikasi secara terintegrasi, serta teruji berfungsi baik lewat *black box testing*.
5	Integrasi n8n Dan Algoritma *Generative Chatbot* Untuk Transformasi *Digital*	Edi Purwanto, Sheila Putri Maharani Sandjoko, dan Syahridho	Model sistem informasi PMB terotomatisasi menggunakan integrasi	Implementasi AI berbasis n8n masih berfokus pada alur *conversational agent* (FAQ &	Berhasil mengintegrasikan n8n dengan *Generative* AI untuk menciptakan *persona CS*

No	Judul	Peneliti	Hal yang Dianalisis	Kekurangan	Kelebihan
	Penerimaan Mahasiswa Baru, SEMASTER: Seminar Nasional Teknologi Informasi & Ilmu Komputer, vol. 4, no. 1, pp. 19-25, 2025. https://journal.unilak.ac.id/index.php/Semaster/article/view/30961	Arjuna Syahputra	n8n dan *Generative Chatbot* dengan metode R&D pada studi kasus UNILAK.	panduan teks), belum memanfaatkan *agentic workflow* untuk pemrosesan/eksekusi tugas multi-langkah yang lebih kompleks.	AI "Sheila" yang mampu merespons FAQ secara *real-time*, komunikatif, dan memandu pendaftaran administratif secara terotomatisasi.
6	*Flow: Modularized Agentic Workflow Automation, arXiv preprint, 2025. https://doi.org/10.48550/arX*	Boye Niu, Yiliao Song, Kai Lian, Yifan Shen, Yu Yao, Kun Zhang, dan Tongliang	Kerangka kerja otomatisasi *agentic workflow* berbasis graf yang modular.	Fokus optimasi masih terbatas pada struktur graf internal, belum diuji pada integrasi *multi-platform*	Menyediakan kerangka kerja berbasis graf yang modular untuk meningkatkan fleksibilitas dan toleransi kesalahan

No	Judul	Peneliti	Hal yang Dianalisis	Kekurangan	Kelebihan
	iv.2501.07834	Liu		pihak ketiga seperti media sosial atau *mikrostock*.	sistem secara *real-time*.
7	*A Review of AI Image Generator: Influences, Challenges, and Future Prospects for Architectural Field*. *Journal of Artificial Intelligence in Architecture*, Vol. 2, No. 1, Hal. 29–40, 2023, DOI: 10.24002/jarina.v2i1.6662.	Enjellina, Eleonora Vilgia Putri Beyan, dan Anastasya Gisela Cinintya Rossy	Kajian literatur sistematis mengenai pengaruh, tantangan, dan prospek penerapan teknologi *AI Image Generator* dalam proses perancangan arsitektur.	Penelitian berfokus pada tinjauan literatur bidang arsitektur, sehingga belum mengimplementasikan alur kerja terotomasi (*agentic workflow*). Sistem juga belum mendukung pembuatan *metadata* otomatis serta integrasi antarmuka *bot Telegram*.	Berhasil memetakan potensi AI *Image Generator* dalam memperluas imajinasi desain melalui alternatif visual berkualitas tinggi, serta mengidentifikasi tingkat kemahiran *prompt*ing dan prospek AI sebagai alat *rendering* independen.

No	Judul	Peneliti	Hal yang Dianalisis	Kekurangan	Kelebihan
8	Pengembangan *Knowledge Management System (KMS)* menggunakan *Agentic AI* dengan Pendekatan Vibe Coding, Jurnal Prima Manajemen, vol. 1, no. 2, pp. 351-364, 2025. https://journal.al-afif.org/index.php/jpm/article/view/624	Fitrah Izul Falaq dan Muh. Ihsan Yudha Pradana	Rancang bangun KMS berbasis *Agentic AI* menggunakan *vibe coding*, kerangka *Design Thinking*, dan model SECI.	Pengembangan KMS konvensional rumit dan memakan waktu lama, serta masih memerlukan riset lanjutan terkait dinamika kolaborasi manusia dan AI.	Aplikasi mampu mempercepat proses pengembangan tanpa mengurangi kualitas sistem, serta mencapai validitas pengujian *Black-box* sebesar 90%.
9	Pemanfaatan *Generative Artificial Intelligence* (AI) Dalam *Efisiensi Production* Konten Media Sosial Untuk	Ferry Adrian dan Muhammad Fauzi Sahono	Dampak efisiensi integrasi AI generatif dalam siklus pembuatan konten.	*Platform AI* yang dianalisis masih dijalankan secara terpisah oleh pengguna (manual),	Berhasil membuktikan secara empiris bahwa integrasi AI generatif efektif memotong

No	Judul	Peneliti	Hal yang Dianalisis	Kekurangan	Kelebihan
	Pemasaran *Digital* UMKM, Jurnal Media Akademik (JMA), vol. 4, no. 3, 2026. https://doi.org/10.62281/b8xd5j13			belum mengimplementasikan sistem otomatisasi otonom berbasis *agentic workflow*.	waktu produksi konten kreatif dari tahap ideasi hingga eksekusi visual.
10	*Enhancing AI Systems with Agentic Workflows Patterns in Large Language Model*, *2024 IEEE World AI IoT Congress (AIIoT)*, pp. 527-532, 2024. https://doi.org/10.1109/AIIoT61789.2024.10578990	Singh, Aditi Singh, Abul Ehtesham, Saket Kumar, dan Tala Talaei Khoei	Pemetaan pola dasar *agentic workflow* untuk meningkatkan produktivitas LLM.	Studi kasus yang digunakan masih bersifat umum (tanya-jawab dan analisis saham), belum menyentuh otomatisasi produksi konten multimedia.	Memetakan empat pola dasar agen cerdas (*reflection, planning, multi-agent*, dan *tool utilization*) untuk meningkatkan produktivitas LLM.

Berdasarkan penelitian terdahulu, penerapan *agentic workflow*, n8n, LLM, dan arsitektur multi-agen terbukti meningkatkan efisiensi operasional di berbagai bidang. Penggunaan n8n dan *cloud* mampu memangkas waktu eksekusi, serta biaya operasional (Casaclang et al., 2026; Chaudhry et al., 2025). Selain itu, integrasi n8n dengan AI generatif dan *bot Telegram* terbukti memberikan kepuasan pengguna yang tinggi, serta mempercepat proses administrasi secara *real-time* (Hariyadi & Soraya, 2025; Purwanto et al., 2025; Farros & Pramana, 2026).

Penerapan kerangka kerja *agentic* AI berbasis graf dan model *vibe coding* juga terbukti meningkatkan fleksibilitas sistem, serta mempercepat pengembangan aplikasi (Niu et al., 2025; Falaq & Pradana, 2025). Pada industri kreatif, penggunaan AI generatif dan pemetaan pola *agentic workflow* efektif memotong waktu produksi konten media sosial, memperluas imajinasi visual, dan mengoptimalkan produktivitas LLM (Adrian & Sahono, 2026; Enjellina et al., 2023; Singh et al., 2024).

Namun, penelitian sebelumnya masih terbatas pada fungsi administrasi dasar, analisis teks, atau optimasi *cloud* tingkat rendah. Sebagian kajian lainnya masih bersifat teoritis, berupa riset literatur, atau dijalankan secara manual oleh pengguna. Belum ada sistem yang menyediakan otomatisasi otonom *end-to-end* yang menghubungkan *bot* Telegram secara langsung dengan pemrosesan aset visual, *upscaling*, *metadata*, hingga analisis *microstock*.

Berdasarkan keterbatasan tersebut, penelitian ini mengusulkan **Implementasi dan Analisis Kinerja *Agentic Workflow* Berbasis Google Gemini dan n8n dalam Otomatisasi Produksi Aset Visual pada *Platform* Adobe Stock Melalui *Bot* Telegram**. Sistem ini mengusung alur kerja otonom berbasis *single-trigger* di n8n, di mana satu perintah tunggal pada *bot* Telegram secara otomatis mengeksekusi *prompt enhancement* menggunakan Gemini 2.5 Flash, pembuatan gambar menggunakan FLUX 1.1 Pro, penyimpanan aset pada Supabase *storage*, peningkatan resolusi (*upscaling*) menggunakan Real-ESRGAN, serta pembuatan *title* dan *keywords* menggunakan Gemini 2.5 Flash-Lite. Selain itu, sistem dilengkapi dengan fitur pendukung berupa penanganan kesalahan (*error handling*) untuk menjaga keandalan proses, serta fitur riset tren berbasis instruksi

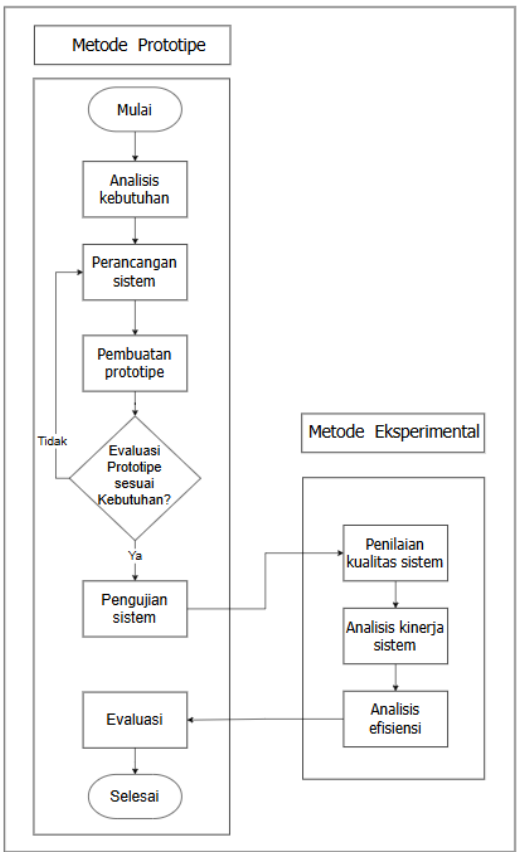
menggunakan Gemini 2.5 Flash-Lite. Pendekatan ini diharapkan dapat memangkas proses manual, menekan *effort* operasional, serta menyajikan analisis kinerja sistem yang terukur.

3. METODE PENELITIAN

3.1 Alur Penelitian

Sistem otomatisasi produksi aset visual gambar ini dikembangkan dengan mengintegrasikan *agentic workflow* berbasis Google Gemini dan n8n, yang dirancang untuk membantu kontributor dalam melakukan efisiensi produksi pada *platform* Adobe Stock melalui *bot* Telegram. Sistem ini secara otomatis memproses permintaan pengguna melalui Telegram, mulai dari pengolahan aset visual hingga ekstraksi *metadata* SEO yang telah disesuaikan dengan kebutuhan teknis, serta standar kualitas *platform microstock*. Arsitektur yang dibangun mengandalkan kemampuan penalaran model bahasa besar untuk melakukan interpretasi perintah secara dinamis, sehingga sistem mampu beradaptasi terhadap variasi *input* teks dari pengguna tanpa memerlukan perombakan kode program. Dengan demikian, pengguna dapat melakukan manajemen aset secara lebih efisien tanpa harus melakukan proses manual yang berulang, yang pada akhirnya dapat meningkatkan produktivitas, serta mempercepat waktu pemasaran aset *digital* ke pasar internasional.

Dalam penelitian ini, pengembangan sistem dilakukan dengan menerapkan kombinasi metode prototipe dan metode eksperimental. Metode prototipe mencakup tahapan pengembangan sistem secara iteratif mulai dari analisis kebutuhan hingga evaluasi sistem, sedangkan metode eksperimental digunakan untuk menguji dan menganalisis kinerja sistem secara mendalam. Kombinasi kedua metode ini dipilih, karena mampu mendukung proses pengembangan yang terstruktur, sistematis, serta memungkinkan analisis kinerja yang objektif berbasis data empiris, sehingga sistem yang dibangun dapat mencapai keandalan operasional sesuai dengan tujuan yang telah ditetapkan. Alur pelaksanaan penelitian yang mengintegrasikan metode prototipe dan metode eksperimental dalam pengembangan sistem ini secara sistematis dapat dilihat pada Gambar 3.1.



Gambar 3.1 Alur Metode Prototipe Eksperimental

Penjelasan setiap tahapan pada Gambar 3.1 adalah sebagai berikut:

1. Analisis Kebutuhan

   Tahap analisis kebutuhan merupakan tahap awal yang bertujuan untuk menganalisis seluruh kebutuhan sistem yang akan dikembangkan. Pada tahap ini dilakukan studi literatur terkait otomatisasi *workflow* n8n, API bot Telegram, integrasi model *Generative AI* (Google Gemini, FLUX 1.1 Pro, dan Real-ESRGAN), serta manajemen *cloud storage*. Selain itu, dilakukan pula identifikasi kebutuhan fungsional dan kebutuhan non-fungsional sistem, perancangan struktur kebutuhan manajemen *storage*, serta penentuan kebutuhan model AI beserta estimasi biayanya.

2. Perancangan Sistem

   Tahap perancangan sistem merupakan tahap pembuatan cetak biru (*blueprint*) arsitektur sistem berdasarkan hasil analisis kebutuhan. Pada tahap ini dilakukan penyusunan *flowchart* alur kerja otomatisasi n8n, perancangan

komponen *nodes* n8n, serta pemodelan UML yang mencakup *use case diagram*, *activity diagram*, dan *sequence diagram* untuk memetakan interaksi data antara Telegram dan layanan pendukung (Google Gemini, OpenAI, OpenRouter, Replicate, serta Supabase). Selain itu, dirancang pula mekanisme perancangan penanganan kesalahan (*error handling*) guna menjaga kestabilan eksekusi *workflow*.

3. Pembuatan Prototipe

   Proses merealisasikan rancangan menjadi sistem fungsional menggunakan n8n, *bot* Telegram, Supabase, dan integrasi API AI (OpenAI, OpenRouter, Replicate). Tahap ini dilakukan secara iteratif. Apabila prototipe belum memenuhi kebutuhan pada tahap evaluasi, proses dikembangkan kembali ke tahap perancangan sistem untuk dilakukan penyempurnaan. Jika sudah sesuai, alur berlanjut ke pengujian sistem.

4. Pengujian Sistem

   Tahap pengujian sistem dilakukan pada prototipe yang telah stabil menggunakan metode eksperimental dalam kondisi operasional nyata. Proses diawali dengan penilaian kualitas aset visual yang dihasilkan, dilanjutkan dengan pengujian kinerja (*performance*) untuk mengukur durasi pemrosesan (*latency*), kapasitas eksekusi (*throughput*), dan identifikasi hambatan (*bottleneck*) pada *workflow* n8n serta API *bot* Telegram. Tahap ini diakhiri dengan analisis efisiensi waktu dan beban kerja untuk membandingkan durasi, serta jumlah interaksi operasional sistem otomatisasi terhadap proses kerja manual.

5. Evaluasi

   Tahap akhir untuk memvalidasi seluruh fungsionalitas dari perspektif pengguna. Validasi dilakukan menggunakan metode *black box testing* untuk memastikan seluruh alur kerja sistem bebas dari kesalahan sebelum proyek dinyatakan selesai.

3.2 Analisis Kebutuhan Sistem

Analisis kebutuhan dilakukan untuk memastikan bahwa sistem yang

dikembangkan mampu memenuhi tujuan utama, yaitu melakukan otomatisasi produksi aset visual secara efisien melalui *bot* Telegram berdasarkan tren pasar dan preferensi pengguna. Kebutuhan sistem ini terbagi menjadi empat aspek utama, yaitu kebutuhan fungsional, kebutuhan nonfungsional, kebutuhan manajemen *storage*, serta kebutuhan model AI dan estimasi biaya.

3.2.1 Kebutuhan Fungsional

Kebutuhan fungsional menjelaskan fungsi utama sistem dalam menerima instruksi masukan dari Telegram, menganalisis tren menggunakan AI *agent*, memproses pembuatan dan peningkatan kualitas gambar melalui integrasi API eksternal, hingga mengirimkan hasil akhirnya secara otomatis kembali kepada pengguna. Adapun kebutuhan fungsional dari sistem otomatisasi produksi aset visual ini adalah sebagai berikut:

1. Sistem mampu menerima pesan pengguna melalui Telegram dan memberikan respon awal bahwa pesan telah diterima, lalu mengidentifikasi jenis perintah yang masuk, serta menginformasikan bahwa permintaan tersebut sedang diproses.
2. Sistem mampu menjalankan AI *agent* berbasis Google Gemini 2.5 Flash Lite untuk menganalisis tren pasar sesuai permintaan pengguna.
3. Sistem mampu mengoptimalkan *prompt* pengguna menggunakan Gemini 2.5 Flash, memproduksi gambar dengan FLUX 1.1 Pro, meningkatkan resolusi ke 2K (JPG) menggunakan Real-ESRGAN, mengekstraksi *title* dan minimal 40 *keywords* menggunakan Gemini 2.5 Flash Lite, menyimpan *file* ke Supabase, serta mengirimkan hasil akhirnya ke pengguna melalui Telegram.
4. Sistem mampu merespon perintah/*start* untuk menampilkan panduan penggunaan, syarat, serta ketentuan layanan *bot*.
5. Sistem dilengkapi fungsi validasi untuk mendeteksi kesalahan format perintah, serta pelanggaran kebijakan konten (*content policy*). Selain itu, sistem menerapkan mekanisme *exception handling* untuk mengantisipasi gangguan koneksi atau kegagalan API, sehingga pengguna menerima pesan

galat (*error message*) yang informatif.

6. Sistem mampu menyederhanakan alur kerja pengguna menjadi satu tindakan tunggal (*single-trigger action*) guna memangkas durasi pemrosesan manual dan memastikan hasil aset sesuai instruksi.

3.2.2 Kebutuhan Nonfungsional

Kebutuhan nonfungsional dalam sistem ini mencakup berbagai aspek penting, termasuk perangkat keras yang mendukung kinerja sistem, serta perangkat lunak yang digunakan untuk memastikan operasional berjalan dengan optimal.

1. Perangkat Keras

Tabel 3.1 Kebutuhan Perangkat Keras

Nama Alat	Spesifikasi
Laptop Lenovo IdeaPad S340	CPU: AMD Ryzen 5 3500U (4 Cores, 8 Logicals) GPU: AMD Radeon Vega Mobile Graphics OS: Microsoft *Window*s 11 Home Single Language *RAM*: 8 GB *Storage*: 1 TB SSD

2. Perangkat Lunak

Tabel 3.2 Kebutuhan Perangkat Lunak

No	Nama Alat	Fungsi
1	Docker	*Platform* kontainerisasi untuk mengisolasi, menjalankan, dan mengelola *instance server* n8n di dalam laptop.
2	n8n	*Platform* otomatisasi alur kerja (*workflow automation*) berbasis *node* untuk mengelola logika sistem dan mengintegrasikan berbagai API

No	Nama Alat	Fungsi
		secara otomatis.
3	Ngrok	Layanan *tunneling* yang berjalan di dalam Docker untuk menyediakan URL HTTPS publik agar *server* Telegram dapat mengirimkan data *webhook* ke n8n.
4	Postman	Perangkat lunak uji coba API yang digunakan untuk melakukan validasi respons dan pengiriman data pada bagian *backend*.
5	Telegram *Bot* API	Protokol komunikasi untuk menghubungkan antarmuka Telegram *dengan platform* n8n sebagai *input-output* perintah.
6	Supabase	*Platform backend-as-a-service* berbasis PostgreSQL yang digunakan untuk menyimpan *metadata* aset visual.
7	Replicate API	Replicate API adalah *platform* infrastruktur AI berbasis *cloud* yang digunakan untuk menjalankan model *generative* FLUX 1.1 Pro dan peningkatan resolusi Real-ESRGAN.
8	Model Gemini 2.5 Flash Lite	Layanan kecerdasan buatan berbasis agen cerdas untuk melakukan analisis tren pasar serta menghasilkan kata kunci dan judul aset, dan memperbaiki struktur *keywords*.
9	Model Gemini 2.5 Flash	Layanan kecerdasan buatan berbasis agen cerdas untuk melakukan penyempurnaan instruksi (*prompt enhancer*).
10	OpenRouter API	Layanan agregator API untuk menghubungkan dan mengakses model Gemini 2.5 Flash secara fleksibel guna menjalankan proses penyempurnaan instruksi (*prompt enhancer*).

No	Nama Alat	Fungsi
11	Model FLUX 1.1 Pro	Layanan kecerdasan buatan berbasis model *generative* AI dari Black Forest Labs yang diintegrasikan untuk memproses pembuatan aset visual berkualitas tinggi berasio 1:1 secara otomatis sesuai instruksi teks.
12	Model Real-ESRGAN (*daanelson*)	Model AI berbasis GAN pada GPU Nvidia A100 untuk melakukan peningkatan resolusi (*upscaling*) dan kualitas citra secara otomatis
13	JSON	Format pertukaran data standar yang digunakan untuk mengirim dan menerima paket informasi antar-API di dalam sistem.
14	OpenAI-*Compatible* API (Aivene)	Digunakan untuk mengintegrasikan dan memanggil model Gemini 2.5 Flash Lite ke dalam *workflow* n8n melalui infrastruktur Aivene yang memanfaatkan standarisasi format protokol API OpenAI.

3.2.3 Kebutuhan Manajemen *Storage*

Sistem penyimpanan pada penelitian ini menggunakan konsep *cloud storage* berbasis *platform* Supabase *storage*. Layanan ini difungsikan secara internal oleh sistem sebagai media perantara (*staging area*), untuk mengorganisasi dan menampung berkas citra *digital* secara terpusat melalui dua *bucket* utama berdasarkan tahapan pemrosesan alur kerja. Penyimpanan ini dirancang khusus untuk mekanisme internal sistem tanpa menyediakan akses langsung bagi pengguna (*end-user*), sehingga hanya berfungsi sebagai jembatan pemrosesan data antar-layanan API sebelum hasil akhirnya dikirimkan kepada pengguna melalui Telegram. Spesifikasi dari masing-masing *bucket* dirangkum pada Tabel 3.3.

Tabel 3.3 Spesifikasi *Bucket* Penyimpanan Supabase

Nama *Bucket*	Jenis Akses	Fungsi / Peruntukan
Raw-assets	*Public Read / Authenticated Write*	Menampung berkas citra hasil generasi awal (*text-to-image*) dari model FLUX 1.1 Pro sebelum ditingkatkan resolusinya.
Upscaled-assets	*Public Read / Authenticated Write*	Menampung berkas citra hasil peningkatan resolusi (*upscaling*) dari model Real-ESRGAN yang siap digunakan untuk ekstraksi *metadata* oleh Gemini 2.5 Flash Lite.

Berdasarkan spesifikasi pada tabel di atas, penerapan hak akses *Public Read / Authenticated Write* pada *bucket raw-assets* dan *upscaled-assets* memberikan keseimbangan antara fleksibilitas alur kerja dan keamanan data. Akses baca bersifat publik (*public read*) memungkinkan model AI eksternal mengunduh citra secara langsung melalui URL tanpa otentikasi tambahan, sedangkan akses tulis terproteksi (*authenticated write*) memastikan hanya sistem terotorisasi yang dapat mengunggah atau memperbarui berkas di dalam kedua *bucket* Supabase *storage* tersebut.

3.2.4 Kebutuhan Model AI dan Estimasi Biaya

Pada bagian ini ditentukan jenis model AI yang digunakan, tahapan pemrosesan, fungsi utama setiap model, jalur API *gateway*, serta estimasi biaya unit dalam satuan United States *Dollar* (USD). Rincian kebutuhan model AI ditunjukkan pada Tabel 3.4.

Tabel 3.4 Kebutuhan *Model* AI dan Estimasi Biaya

Model AI	**Tahapan Pemrosesan**	**Fungsi Utama**	**API *Gateway***	**Estimasi Biaya Unit (USD)**
Gemini 2.5 Flash	*Prompt Enhancer*	Mengoptimalkan dan memperjelas deskripsi instruksi (*prompt*) untuk generasi gambar.	OpenRouter (Google Vertex)	± $0.0000644 / eksekusi
FLUX 1.1 Pro	Generasi Gambar	Mengenerasi gambar awal sesuai *prompt.*	Replicate API	$0.04 / Gambar
Real-ESRGAN (real-esrgan-a100)	*Upscaling*	Meningkatkan resolusi gambar (s*uper-resolution*).	Replicate API	$0.0019 / Eksekusi
Gemini 2.5 Flash Lite	Riset Tren, *Output Parsing*, & Ekstraksi *Metadata*	Menganalisis tren topik visual, menyusun struktur data (*output parser*), serta menghasilkan *title* dan *keywords* dari gambar.	Aivene (OpenAI-*Compatible*)	$0.05 / MTok (*input*) $0.20 / MTok (*output*)

Tabel 3.4 menyajikan spesifikasi kebutuhan model AI, serta estimasi biaya operasional yang digunakan dalam alur kerja pemrosesan aset *microstock*. Pemilihan setiap model disesuaikan dengan peran teknis, efisiensi pemrosesan, dan

transparansi skema harga dari masing-masing penyedia layanan.

Alur pemrosesan diawali pada tahap pengoptimalan instruksi masukan (*prompt enhancer*), di mana model Gemini 2.5 Flash dimanfaatkan untuk mengolah dan memperjelas deskripsi *prompt* awal pengguna. Model ini diakses melalui API *gateway* OpenRouter (terhubung ke *backend* Google Vertex AI) dengan estimasi biaya sebesar ± $0.0000644 per eksekusi.

Selanjutnya, instruksi yang telah teroptimasi diteruskan ke tahap generasi gambar (*text-to-image*) menggunakan model FLUX 1.1 Pro melalui Replicate API dengan biaya $0.04 per gambar untuk menghasilkan aset visual awal. Setelah aset visual terbentuk, sistem melakukan tahap peningkatan resolusi (*upscaling*) menggunakan model Real-ESRGAN (*real-esrgan-a100*) via Replicate API dengan estimasi biaya sebesar $0.0019 per pemrosesan guna menghasilkan gambar beresolusi tinggi.

Tahap pemrosesan utama diakhiri dengan ekstraksi *metadata* dan penyusunan struktur data (*output parsing*), di mana model Gemini 2.5 Flash Lite menganalisis gambar hasil *upscale* untuk merumuskan *title* dan *keywords* secara terstruktur. Model ini diakses melalui *API gateway* Aivene (*OpenAI-Compatible*) dengan skema tarif $0.05 per 1 juta token *input* dan $0.20 per 1 juta token *output*. Selain dalam alur utama produksi, model Gemini 2.5 Flash Lite melalui *API gateway* Aivene juga difungsikan sebagai fitur dukungan untuk riset tren, yang bertugas menganalisis topik visual, serta kata kunci populer di pasar *microstock* dengan skema tarif token yang sama.

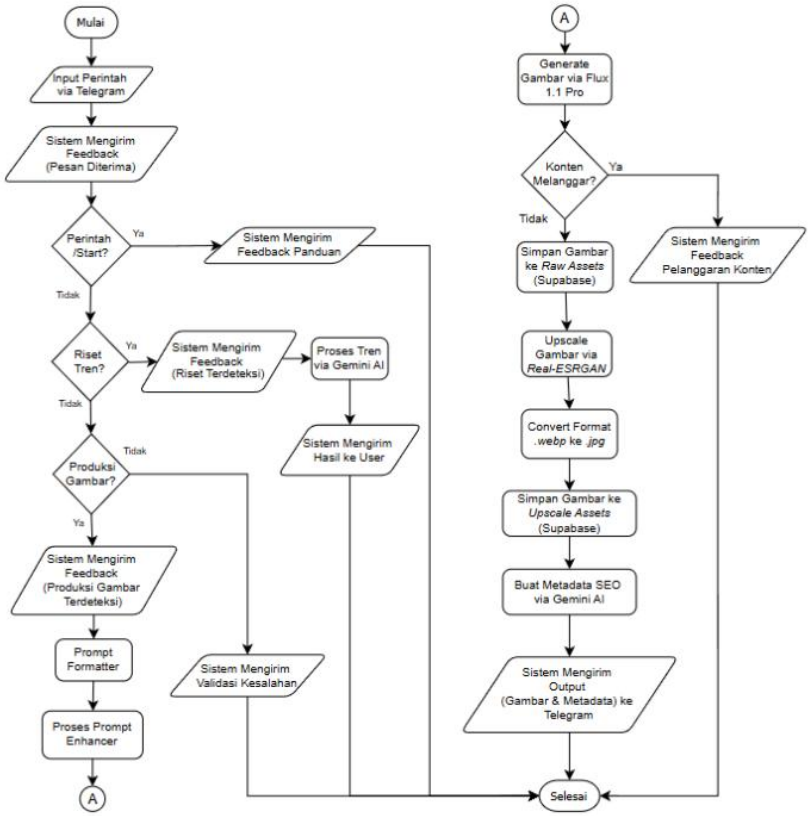
Berdasarkan rincian tarif dari masing masing modul tersebut, total estimasi biaya operasional yang dibutuhkan untuk memproduksi satu aset gambar secara lengkap mulai dari tahap pengoptimalan *prompt*, generasi visual, peningkatan resolusi (*upscaling*), hingga ekstraksi *metadata* adalah sebesar ± $0.042 (atau sekitar Rp.680,- per gambar). Sementara itu, untuk eksekusi fitur dukungan riset tren yang berdiri sendiri, pengeluaran yang dibutuhkan sangat efisien, yaitu sebesar ± $0.0001 (atau sekitar Rp1,6 per satu kali eksekusi riset). Kalkulasi ini menunjukkan bahwa akumulasi biaya pemrosesan berbasis otomatisasi API ini sangat ekonomis dan terjangkau untuk skenario produksi aset visual secara massal.

3.3 Perancangan Sistem

Perancangan sistem dilakukan untuk merancang urutan proses dan komponen teknis pada otomatisasi alur kerja produksi aset *microstock*. Tahap ini mencakup penyusunan *flowchart* alur kerja, perancangan komponen *nodes* n8n, pemodelan UML yang terdiri atas *use case diagram*, *activity diagram*, dan *sequence diagram*, serta perancangan mekanisme penanganan kesalahan guna memvisualisasikan interaksi pengguna dan alur pertukaran data antar-layanan secara terstruktur.

3.3.1 *Flowchart*

Flowchart digunakan untuk menggambarkan urutan langkah logis dan aliran proses pengerjaan sistem secara rinci dari awal hingga akhir. Bagan alur yang menjelaskan jalannya proses otomatisasi alur kerja produksi aset *microstock* ini dapat dilihat pada Gambar 3.2.



Gambar 3.2 *Flowchart* Sistem

Berdasarkan Gambar 3.2, sistem dimulai saat pengguna memasukkan perintah melalui Telegram yang langsung direspons dengan pesan konfirmasi penerimaan oleh sistem. Selanjutnya sistem menyaring perintah tersebut ke dalam tiga jalur. Jika perintah berupa awal atau *start,* maka sistem mengirimkan panduan. Jika berupa riset tren, maka sistem mendeteksi permintaan lalu memprosesnya melalui AI Gemini 2.5 Flash Lite dan mengirimkan hasilnya. Sedangkan jika berupa produksi gambar, maka sistem mengirimkan notifikasi deteksi lalu meneruskan teks ke modul penata format dan peningkat teks perintah sebelum dialirkan ke kolom pemrosesan utama melalui konektor A. Perintah yang tidak memenuhi ketiga kondisi tersebut akan langsung menerima validasi kesalahan dari sistem dan alur selesai.

Melalui konektor A teks perintah yang telah ditingkatkan langsung diproses oleh mesin pembuat gambar FLUX 1.1 Pro. Sistem kemudian memeriksa hasil pembuatan gambar. Jika mengandung pelanggaran konten, maka sistem mengirimkan notifikasi pelanggaran. Namun jika tidak, maka gambar mentah langsung disimpan ke basis data Supabase pada *bucket raw-assets*. Gambar tersebut kemudian ditingkatkan resolusinya menggunakan Real-ESRGAN dan formatnya dikonversi dari WEBP menjadi JPG. Setelah gambar hasil peningkatan disimpan kembali ke Supabase pada *bucket upscaled-assets*, sistem menggunakan Gemini 2.5 Flash Lite untuk membuat data deskripsi otomatis bagi keperluan optimasi mesin pencari atau SEO. Pada tahap akhir sistem mengirimkan hasil akhir berupa gambar berkualitas tinggi beserta *metadata* (judul dan kata kunci) ke Telegram pengguna dan proses dinyatakan selesai.

3.3.2 Perancangan Komponen *Nodes* n8n

Untuk merealisasikan alur kerja pada *flowchart*, sistem dikonfigurasikan menggunakan berbagai komponen *nodes* pada *platform* n8n. *Nodes* yang digunakan terdiri dari *nodes* bawaan (*core/native nodes*), serta *nodes* integrasi API (HTTP *request*) untuk menghubungkan layanan eksternal. Rincian komponen *nodes* disajikan pada Tabel 3.5.

Tabel 3.5 Rincian Komponen *Nodes* n8n pada Sistem

No	Nama *Node* pada Sistem	Jenis / Tipe *Node* n8n	Kategori Integrasi	Deskripsi Fungsi Utama
1	*Telegram Trigger*	*Telegram Trigger*	*Native / Core Node*	Menerima pesan teks dari pengguna via Telegram sebagai pemicu *workflow*.
2	*Request Handler*	*Edit Fields (Set)*	*Native / Core Node*	Memproses dan merapikan data masukan sebelum diteruskan ke alur berikutnya.
3	*Feedback* Awal	Telegram (*sendMessage*)	Native / *Core Node*	Mengirimkan notifikasi awal bahwa pesan telah diterima dan diproses.
4	Menu	*Switch* (mode: *Rules*)	Native / Core *Node*	Memilah cabang alur berdasarkan jenis perintah atau pesan pengguna.
5	*Feedback Start*	Telegram (*sendMessage*)	*Native / Core Node*	Mengirimkan tampilan menu panduan dan panduan saat perintah */start* dipicu.
6	*Feedback Research*	Telegram (*sendMessage*)	*Native / Core Node*	Mengirimkan notifikasi awal bahwa permintaan riset tren sedang diproses.
7	*Research Assistant*	AI *Agent*	*Native / Core Node*	Mengoordinasikan alur agen cerdas untuk memproses analisis tren pasar.
8	OpenAI - Gemini-2.5-Flash-Lite2	OpenAI *Model*	*External API* (Gemini via OpenAI SDK)	Sub-*node* model LLM yang menyediakan kapabilitas kecerdasan buatan untuk *node Research Assistant*.
9	*Simple*	*Window Buffer*	*Native / Core*	Menyimpan konteks riwayat

No	Nama *Node* pada Sistem	Jenis / Tipe *Node* n8n	Kategori Integrasi	Deskripsi Fungsi Utama
	Memory	*Memory*	*Node*	percakapan singkat pada alur riset.
10	Kirim *Research Trend*	Telegram (*sendMessage*)	*Native / Core Node*	Mengirimkan hasil laporan riset tren pasar ke Telegram pengguna.
11	*Feedback* Pesan *Error*	Telegram (*sendMessage*)	*Native / Core Node*	Mengirimkan pesan peringatan, jika masukan pengguna tidak *valid*.
12	*Feedback Generate*	Telegram (*sendMessage*)	*Native / Core Node*	Mengirimkan pesan notifikasi ke Telegram, bahwa permintaan pembuatan gambar terdeteksi dan sedang diproses.
13	*Prompt Formatter*	*Edit Fields / Code*	*Native / Core Node*	Memvalidasi dan merapikan format teks *prompt* masukan dari pengguna.
14	*Prompt Enhancer*	HTTP *Request*	*External API* (OpenRouter)	Mengirimkan *prompt* ke model AI untuk memperkaya deskripsi visual secara otomatis.
15	*Generate Image* - FLUX 1.1 Pro	HTTP *Request*	*External API* (Replicate)	Mengirimkan *prompt* akhir ke API FLUX 1.1 Pro untuk proses pembuatan gambar.
16	Cek Pelanggaran	*If (Conditional*)	*Native / Core Node*	Memeriksa kondisi apakah *prompt* atau hasil generasi terdeteksi melanggar

No	Nama *Node* pada Sistem	Jenis / Tipe *Node* n8n	Kategori Integrasi	Deskripsi Fungsi Utama
				kebijakan konten.
17	*Image* URL *Generator*	HTTP *Request* / *Code*	*Native / External API*	Mengekstrak dan menyusun tautan (URL) gambar mentah hasil generasi untuk proses berikutnya.
18	*Storage Raw-assets* - Supabase	HTTP *Request*	*External Database* (Supabase)	Mengunggah dan menyimpan berkas gambar hasil generasi awal (*raw asset*) ke penyimpanan Supabase.
19	Ambil Gambar dari *Raw-assets*	HTTP *Request*	*External Database (*Supabase*)*	Mengambil tautan atau berkas gambar mentah dari Supabase untuk diproses lebih lanjut.
20	*Upscale* gambar Real-ESRGAN	HTTP *Request*	*External API (*Replicate*)*	Mengirimkan berkas gambar mentah ke model Real-ESRGAN untuk meningkatkan resolusi gambar.
21	Ambil Gambar Real-ESRGAN	HTTP *Request*	*External API (*Replicate*)*	Mengunduh atau mengambil hasil gambar yang telah ditingkatkan resolusinya dari *API* Real-ESRGAN.
22	*Convert* .WEBP-.JPG	*Convert to File / Code*	*Native / Core Node*	Mengonversi format berkas gambar dari .WEBP menjadi .JPG sesuai standar *microstock*.
23	*Storage*	HTTP *Request*	*External*	Mengunggah dan menyimpan

No	Nama *Node* pada Sistem	Jenis / Tipe *Node* n8n	Kategori Integrasi	Deskripsi Fungsi Utama
	Bucket Image-Upscaled		*Database* (Supabase)	berkas gambar *.jpg* hasil *upscale* ke *bucket* akhir Supabase.
24	*Image URL Generator1*	HTTP *Request / Code*	*Native / External API*	Mengekstrak dan menyusun *public* URL untuk berkas gambar akhir yang tersimpan di Supabase.
25	*AI Agent-Metadata*	AI *Agent*	*Native / Core Node*	Mengoordinasikan alur agen cerdas untuk menganalisis gambar akhir dan mengekstrak *metadata* komersial.
26	OpenAI - Gemini-2.5-flash-lite	OpenAI *Chat Model*	*External API* (Gemini via OpenAI *SDK*)	Sub-*node* LLM utama yang dihubungkan ke *AI Agent-Metadata* untuk menganalisis visual gambar.
27	*Structured Output Parser*	*Structured Output Parser*	*Native / Core Node*	Memastikan format keluaran dari AI *Agent* berbentuk struktur data JSON yang rapi (*title* & *keywords*).
28	OpenAI - Gemini-2.5-flash-lite1	OpenAI *Chat Model*	*External API* (Gemini via OpenAI *SDK*)	Sub-*node* LLM pendukung yang dihubungkan ke *Structured Output Parser* untuk memvalidasi skema data.
29	Kirim ke Pengguna	Telegram (*sendMessage*)	*Native / Core Node*	Mengirimkan berkas gambar *.jpg* akhir beserta *metadata* (*title* & *keywords*) ke

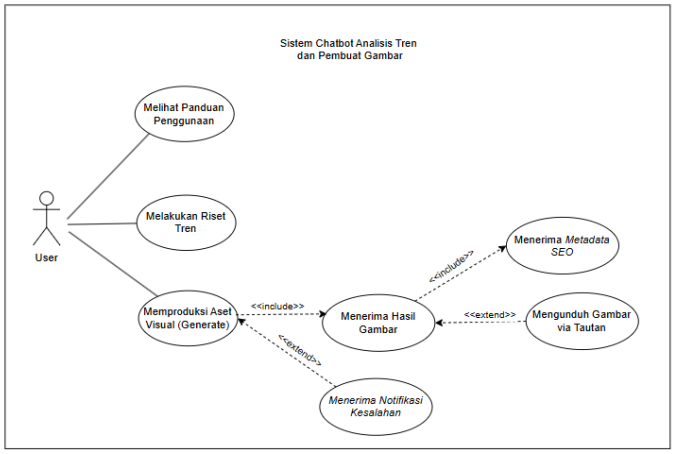
No	Nama *Node* pada Sistem	Jenis / Tipe *Node* n8n	Kategori Integrasi	Deskripsi Fungsi Utama
				Telegram pengguna.
30	*Error* Pelanggaran	Telegram (*sendMessage*)	*Native / Core Node*	Mengirimkan notifikasi pesan galat ke pengguna ketika *input prompt* melanggar kebijakan konten (*content policy*).
31	*Handling Error 2*	Telegram (*sendMessag*e)	*Native / Core Node*	Mengirimkan notifikasi pesan galat ke pengguna ketika terjadi kegagalan sistem, *timeout*, atau kendala jaringan pada API eksternal.

Berdasarkan Tabel 3.5, perancangan arsitektur *workflow* otomasi pada *platform* n8n melibatkan 31 komponen *nodes* yang saling terintegrasi. Secara keseluruhan, pemetaan komponen ini terdiri dari 20 *native nodes* bawaan n8n yang bertugas mengelola logika alur kerja, pengondisian sistem, pemrosesan format data, interaksi *bot* Telegram, serta penanganan notifikasi galat (*error handling*) akibat gangguan sistem maupun pelanggaran kebijakan konten. Selanjutnya terdapat 8 HTTP *request nodes* untuk menangani pertukaran data REST API dengan *platform* pihak ketiga, termasuk OpenRouter API untuk penajaman *prompt*, Replicate API memanfaatkan model FLUX 1.1 Pro (black-forest-labs/FLUX 1.1pro) untuk generasi gambar visual, serta model Real-ESRGAN (daanelson/real-esrgan) untuk peningkatan resolusi (*upscaling*), dan Supabase *storage* REST API untuk manajemen penyimpanan aset. Selain itu, terdapat 3 *external* API sub-*nodes* berprotokol OpenAI *chat model* untuk menyediakan pemrosesan model bahasa LLM Google Gemini (Gemini 2.5 Flash Lite) pada alur riset tren dan pembuatan *metadata*. Pembagian peran komponen yang terstruktur ini mendukung eksekusi seluruh alur kerja sistem, utama pada pembuatan aset visual secara otomatis yang mencakup generasi citra, *upscaling*, dan ekstraksi *metadata*, serta didukung oleh

fitur pendukung berupa panduan penggunaan dan riset tren *microstock*, hingga penanganan galat dan pengiriman hasil akhir ke pengguna secara efisien.

3.3.3 *Use Case Diagram*

Use case diagram digunakan untuk menggambarkan hubungan interaksi yang terjadi antara aktor (pengguna) dengan fungsi-fungsi utama yang disediakan di dalam sistem. Dalam sistem otomatisasi alur kerja ini, terdapat satu aktor utama, yaitu kontributor atau *user*. Rancangan diagram *use case* dari sistem ini dapat dilihat pada Gambar 3.3.



Gambar 3.3 *Use Case Diagram* Sistem

Berdasarkan visualisasi diagram *use case* pada Gambar 3.3, sistem memetakan interaksi langsung antara aktor *user* dengan fungsionalitas sistem melalui antarmuka Telegram *Bot. User* bertindak sebagai penggerak utama yang memicu tiga fungsi mendasar secara independen tanpa keharusan mengikuti urutan tertentu, yaitu melihat panduan penggunaan, melakukan riset tren, dan memproduksi aset visual (*generate*). Fungsi melihat panduan penggunaan diakses melalui perintah */start* untuk menampilkan instruksi dasar penggunaan sebagai langkah orientasi awal. Sementara itu, fungsi riset tren dirancang khusus untuk memfasilitasi analisis pasar industri *microstock*. Fitur ini memungkinkan pengguna memicu penelusuran topik populer menggunakan kata kunci berbasis pertanyaan seperti "Apa" atau "Bagaimana", sehingga pemantauan tren dapat dilakukan secara

instan tanpa harus terikat dengan alur pembuatan gambar.

Fungsi utama sistem berpusat pada proses pembuatan aset visual, di mana alur kerja akan terpicu secara otomatis ketika sistem mendeteksi adanya kata kunci tertentu (seperti kata "Buat/Buatkan") di dalam pesan instruksi teks yang dikirimkan oleh pengguna. Ketika fungsi ini berjalan, sistem secara otomatis melibatkan fungsi menerima hasil gambar dan fungsi menerima *metadata* SEO melalui relasi *include*, karena penyajian gambar final beserta data penunjang komersialnya merupakan satu kesatuan *output* utama yang wajib diberikan kepada pengguna setelah produksi berhasil. Sebagai perluasan alur kerja, fungsi menerima hasil gambar memiliki relasi *extend* dengan fungsi mengunduh gambar melalui tautan, karena tindakan mengunduh berkas sepenuhnya bersifat opsional tergantung kebutuhan pengguna. Di sisi lain, fungsi memproduksi aset visual (*generate*) juga terhubung dengan fungsi menerima notifikasi kesalahan melalui relasi *extend*, yang berarti respons galat tersebut hanya akan aktif secara kondisional apabila sistem mendeteksi hambatan teknis atau pelanggaran kebijakan konten di tengah proses eksekusi.

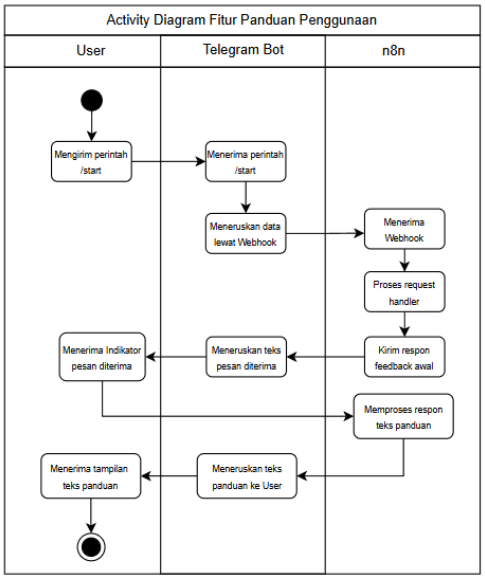
3.3.4 *Activity Diagram*

Activity diagram digunakan untuk menggambarkan alur kerja dan urutan aktivitas yang terjadi di dalam sistem dari awal hingga akhir. Pada penelitian ini, terdapat tiga *activity diagram* yang menggambarkan alur operasional sistem, yaitu alur fitur panduan penggunaan, alur fitur riset tren, dan alur modul utama pembuatan aset visual.

1. *Activity Diagram* Fitur Panduan Penggunaan

   Bagian ini menggambarkan rancangan aliran aktivitas pada fitur panduan penggunaan yang berfungsi sebagai menu panduan awal bagi pengguna. Perancangan ini melibatkan interaksi fungsional antara tiga entitas utama, yaitu pengguna, Telegram *bot,* dan mesin otomatisasi n8n yang dibagi ke dalam kolom aktivitas masing-masing. Alur kerja operasional secara menyeluruh mengenai bagaimana sistem memproses perintah awal dari antarmuka hingga menghasilkan respons balik teks panduan dapat dilihat

pada Gambar 3.4.



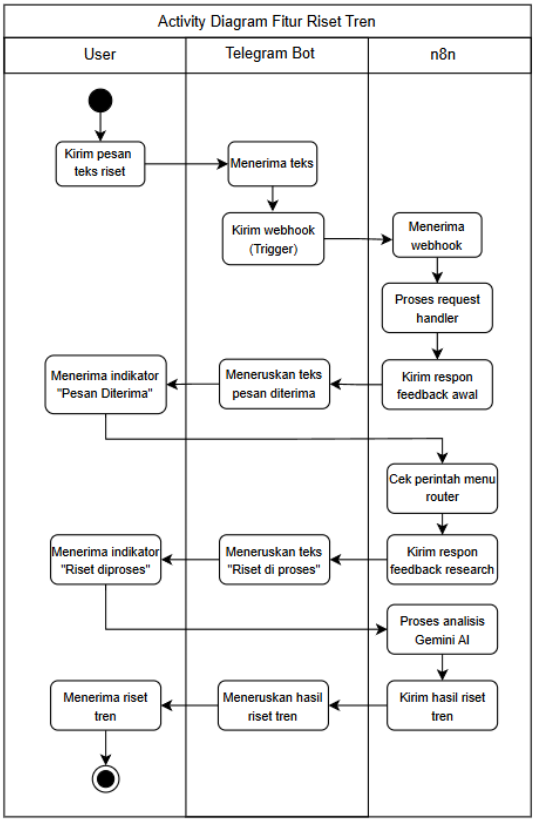
Gambar 3.4 *Activity Diagram* Fitur Panduan

Berdasarkan Gambar 3.4, alur kerja dimulai ketika pengguna mengirim perintah /*start* melalui antarmuka Telegram. Perintah tersebut diterima oleh Telegram *Bot*, yang kemudian meneruskan data pesan tersebut melalui *webhook* menuju sistem n8n. Setelah n8n menerima sinyal *webhook*, sistem menjalankan *proses request handler* dan mengirimkan respon *feedback* awal. Respon ini diteruskan oleh Telegram *Bot,* sehingga pengguna menerima indikator, bahwa pesan telah diterima. Selanjutnya, sistem n8n memproses respon teks panduan secara lengkap. Setelah teks panduan selesai diproses, n8n meneruskannya kembali ke Telegram *Bot* untuk dikirimkan kepada pengguna. Proses diakhiri ketika pengguna berhasil menerima dan melihat tampilan teks panduan penggunaan pada layar Telegram.

2. *Activity Diagram* Fitur Riset Tren

   Bagian ini menggambarkan aliran aktivitas pada fitur riset tren pasar untuk menganalisis topik populer kebutuhan industri. Alur operasional dari antarmuka pengguna hingga pengiriman hasil tren pasar, dapat dilihat pada

Gambar 3.5.



Gambar 3.5 *Activity Diagram* Fitur Riset Tren

Berdasarkan Gambar 3.5, alur kerja dimulai ketika pengguna mengirimkan pesan teks riset melalui antarmuka Telegram, yang terdeteksi melalui penggunaan kata kunci pertanyaan seperti kata “Apa”, “Bagaimana”, atau frasa tanya sejenisnya. Pesan tersebut diterima oleh Telegram *Bot*, yang kemudian mengirimkan *webhook* sebagai pemicu untuk meneruskan data pesan ke sistem n8n.

Setelah n8n menerima *webhook*, sistem langsung menjalankan proses *request handler* untuk mengolah data masukan dan mengirimkan respon *feedback* awal. Respon *feedback* awal ini meneruskan teks konfirmasi ke Telegram *Bot,* sehingga pengguna menerima indikator "Pesan Diterima" pada layar obrolan.

Selanjutnya, n8n melakukan pengecekan perintah menggunakan modul

menu router untuk memastikan jenis permintaan yang dikirimkan, yaitu dengan mendeteksi keberadaan kata kunci pertanyaan, seperti "Apa" atau "Bagaimana". Setelah terverifikasi sebagai permintaan riset, n8n mengirimkan respons balikan (*feedback research*). Respons ini kembali diteruskan oleh *bot* Telegram, sehingga pengguna menerima indikator status "Permintaan Riset Terdeteksi! Proses sedang berjalan, mohon tunggu sebentar..." sebagai pembaruan status sistem.

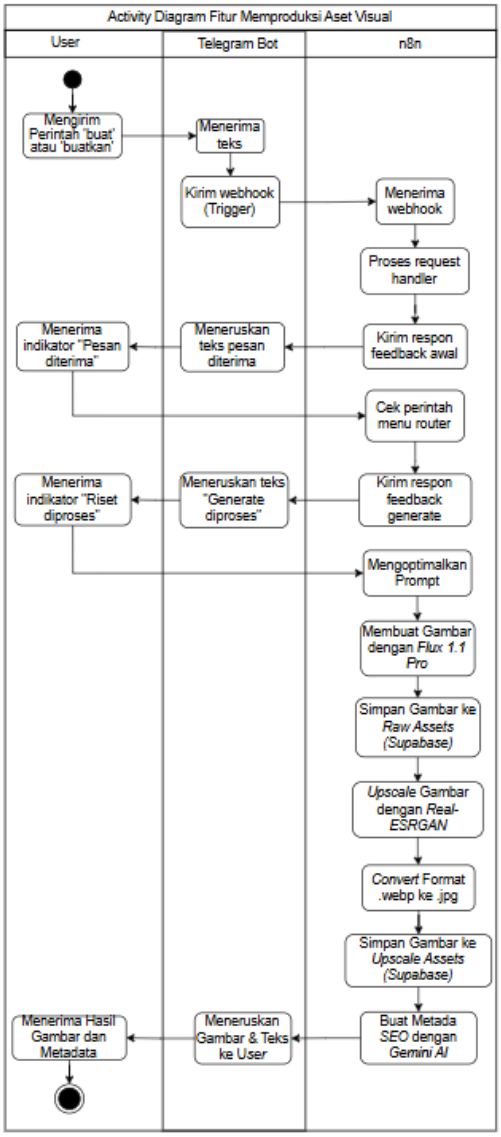
Setelah indikator tersebut diterima pengguna, sistem n8n melanjutkan alur pemrosesan inti menuju tahap proses analisis Gemini AI. Pada tahap ini, model kecerdasan buatan menganalisis topik dan tren yang diminta secara mendalam. Setelah pemrosesan analisis selesai, n8n mengirimkan hasil riset tren tersebut beserta format pesannya ke Telegram *Bot* untuk diteruskan kembali kepada pengguna. Seluruh alur proses diakhiri ketika pengguna berhasil menerima dan melihat tampilan hasil riset tren secara lengkap pada antarmuka Telegram.

3. *Activity Diagram* Fitur Memproduksi Aset Visual

   Bagian ini menggambarkan alur aktivitas pada fitur pembuatan aset visual sebagai modul utama dari sistem. Diagram ini memetakan seluruh alur pemrosesan berantai, mulai dari penerimaan perintah dari pengguna, pengoptimalan perintah, pembuatan dan peningkatan kualitas gambar, penulisan *metadata*, hingga penyimpanan akhir.

   Secara spesifik, proses dimulai ketika pengguna meng*input*kan teks instruksi (*prompt*) melalui antarmuka Telegram Bot. Sistem pada n8n kemudian menangkap *trigger* tersebut dan memberikan konfirmasi balik (*feedback*) kepada pengguna. Selanjutnya, *prompt* mentah dioptimalkan (*enhancement*) menggunakan OpenRouter API untuk memperjelas detail deskripsi visual sebelum dikirimkan ke model FLUX 1.1 Pro melalui Replicate API untuk tahap generasi gambar. Citra yang dihasilkan kemudian unggah ke penyimpanan *cloud* Supabase *bucket raw-assets* lalu diproses oleh modul Real-ESRGAN untuk peningkatan resolusi (*upscaling*), dikonversi ke format JPG, lalu diunggah ke penyimpanan *cloud*

Supabase *bucket upscale-assets*. Secara paralel, agen AI berbasis Gemini 2.5 Flash Lite melakukan ekstraksi *metadata* berupa *title* dan *keywords* komersial dari gambar tersebut. Alur kerja operasional yang terintegrasi antara pengguna, Telegram Bot, dan alur kerja n8n disajikan pada Gambar 3.6.



Gambar 3.6 *Activity Diagram* Fitur Produksi Aset Visual Gambar

Berdasarkan Gambar 3.6, alur kerja dimulai ketika pengguna mengirimkan perintah "Buat" atau "Buatkan" melalui antarmuka Telegram. Pesan tersebut diterima oleh Telegram *Bot*, yang kemudian mengirimkan *webhook*

sebagai pemicu untuk meneruskan data ke sistem n8n. Setelah n8n menerima *webhook*, sistem melakukan proses *request handler* dan mengirimkan respon *feedback* awal. Respon ini diteruskan oleh Telegram *Bot,* sehingga pengguna menerima indikator "Pesan diterima" pada antarmuka obrolan.

Selanjutnya, n8n melakukan pengecekan perintah melalui modul *menu router* untuk memastikan jenis permintaan yang dikirimkan, yaitu dengan mendeteksi keberadaan kata kunci perintah, seperti "Buat". Setelah terverifikasi sebagai permintaan pembuatan gambar, n8n mengirimkan respons balikan (*feedback generate*). Respons tersebut kembali diteruskan oleh *bot* Telegram, sehingga pengguna menerima indikator status "Permintaan Pembuatan Gambar Terdeteksi! Proses sedang berjalan, mohon tunggu sebentar..." sebagai pembaruan status sistem.

Setelah indikator diterima pengguna, n8n melanjutkan proses eksekusi berantai dengan mengoptimalkan *prompt* agar instruksi teks menjadi lebih presisi. Selanjutnya, n8n membuat gambar menggunakan FLUX 1.1 Pro dan menyimpan hasilnya ke direktori *raw assets* pada Supabase. Gambar yang telah tersimpan di dalam *bucket* tersebut kemudian diambil tautannya untuk diproses pada tahap *upscale* menggunakan model Real-ESRGAN, diubah formatnya dari .WEBP menjadi .JPG, lalu disimpan kembali ke direktori *upscale assets* pada Supabase. Tautan gambar hasil *upscale* tersebut selanjutnya digunakan oleh Gemini AI sebagai acuan untuk membuat *metadata* SEO secara otomatis.

Proses diakhiri ketika n8n mengirimkan hasil gambar beserta teks *metadata*nya ke Telegram *bot* untuk diteruskan kepada pengguna, hingga pengguna berhasil menerima dan melihat tampilan gambar, serta *metadata* tersebut pada Telegram.

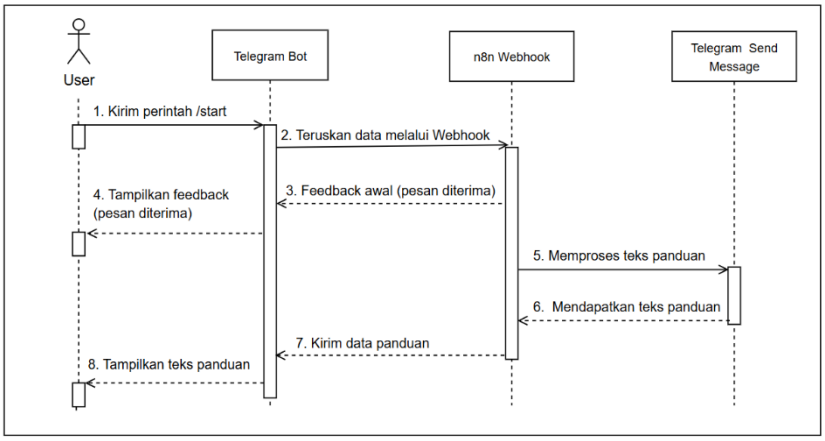
3.3.5 *Sequence Diagram*

Sequence diagram digunakan untuk menggambarkan interaksi antarkomponen di dalam sistem berdasarkan urutan waktu secara kronologis. Pada

penelitian ini, terdapat tiga *sequence diagram* yang menggambarkan alur komunikasi sistem, yaitu alur fitur panduan penggunaan, alur fitur riset tren, dan alur modul utama pembuatan aset visual.

1. *Sequence Diagram* Fitur Panduan Penggunaan

   Sequence diagram pada fitur ini menunjukkan urutan pengiriman pesan ketika pengguna mengaktifkan *chatbot* untuk pertama kali menggunakan perintah tertentu. Diagram ini melibatkan empat komponen utama yang berinteraksi dari atas ke bawah, yaitu *User*, Telegram *Bot,* n8n *Webhook*, dan Telegram *Send Message*. Proses dan urutan interaksi antarkomponen saat pengguna mengakses fitur panduan penggunaan dari awal hingga selesai, dapat dilihat pada Gambar 3.7.



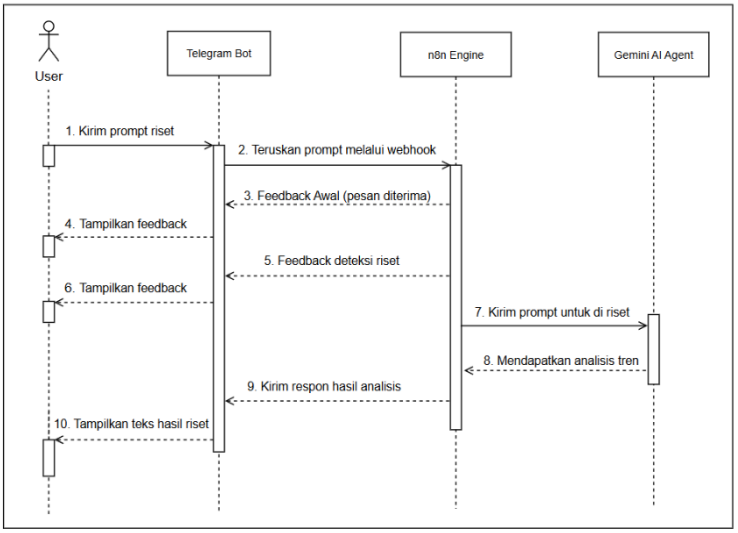
Gambar 3.7 *Sequence Diagram* Fitur Panduan

Berdasarkan Gambar 3.7, *sequence diagram* fitur panduan penggunaan menggambarkan interaksi antarkomponen di dalam sistem yang meliputi pengguna (*User*), Telegram *Bot*, n8n *Webhook*, dan Telegram *Send Message*. Alur proses diawali ketika pengguna mengirimkan perintah /*start* ke Telegram *Bot*, yang kemudian meneruskan data perintah tersebut melalui *webhook* ke n8n *Webhook*. Selanjutnya, n8n *Webhook* mengirimkan balasan *feedback* awal berupa konfirmasi pesan diterima, yang diteruskan oleh Telegram *bot,* sehingga pengguna dapat melihat indikator pesan diterima

pada layar obrolan. Setelah indikator dikirimkan, n8n *Webhook* memproses teks panduan ke komponen Telegram *send message*, lalu menerima kembali respon berupa teks panduan yang telah disiapkan. Data panduan tersebut kemudian dikirimkan kembali oleh n8n *Webhook* ke Telegram *Bot* hingga akhirnya Telegram *Bot* menampilkan teks panduan penggunaan secara utuh pada layar obrolan pengguna.

2. *Sequence Diagram* Fitur Riset Tren

   Sequence diagram pada fitur riset tren menunjukkan urutan interaksi kronologis saat pengguna melakukan analisis tren *microstock*. Proses ini melibatkan empat komponen utama, yaitu *User*, Telegram *Bot*, n8n *Engine*, dan Gemini *AI Agent*, sebagaimana disajikan pada Gambar 3.8.



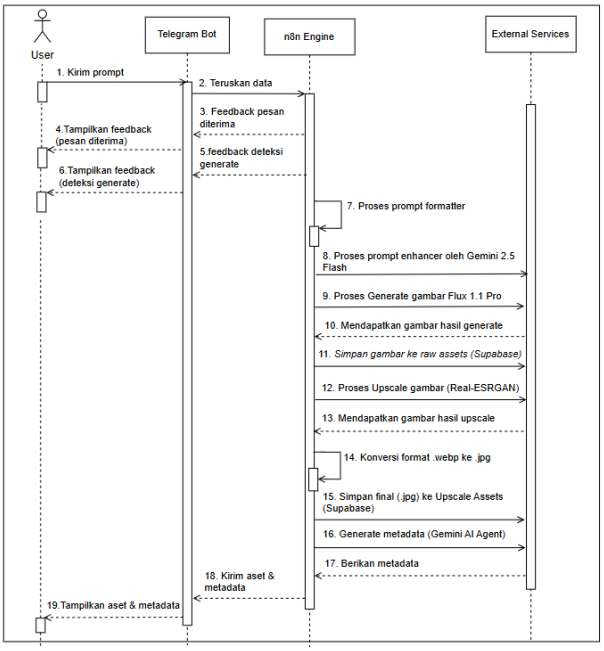
Gambar 3.8 *Sequence Diagram* Fitur Riset Tren

Berdasarkan Gambar 3.8, *Sequence diagram* fitur riset tren menggambarkan interaksi antarkomponen di dalam sistem yang meliputi *User*, Telegram *Bot*, n8n *Engine*, dan Gemini AI *Agent*. Alur proses diawali ketika pengguna mengirimkan *prompt* riset (menggunakan kata kunci seperti "Apa" atau "Bagaimana" atau frasa tanya sejenisnya) ke Telegram *Bot*, yang kemudian meneruskannya melalui *webhook* ke n8n *Engine*. Selanjutnya, n8n *Engine* memberikan *feedback* awal berupa konfirmasi pesan diterima, yang diteruskan oleh Telegram *Bot* sehingga pengguna melihat tampilan

feedback tersebut. n8n *Engine* kemudian mengirimkan *feedback* deteksi riset ke Telegram *Bot* untuk ditampilkan kembali kepada pengguna sebagai indikator pembaruan status pemrosesan. Setelah itu, n8n *Engine* mengirimkan *prompt* riset ke Gemini AI *Agent* untuk dianalisis, lalu mendapatkan kembali hasil analisis tren dari Gemini AI *Agent*. Hasil analisis tersebut selanjutnya dikirimkan oleh n8n *Engine* ke Telegram *Bot* hingga akhirnya Telegram *Bot* menampilkan teks hasil riset secara lengkap pada layar obrolan pengguna.

3. *Sequence Diagram* Fitur Produksi Aset Gambar

   Sequence diagram pada fitur ini menggambarkan urutan kronologis proses otomatisasi produksi aset visual mulai dari tahap pengguna melakukan *input prompt* hingga aset visual dan *metadata* yang sudah terstruktur dikirim kembali ke pengguna melalui Telegram. Proses komunikasi data ini melibatkan empat objek utama yang saling berinteraksi, yaitu *User*, Telegram *Bot, n8n Engine*, dan *External Services*. Proses interaksi antar komponen yang menggambarkan urutan waktu pengerjaan pada fitur produksi aset dari hulu ke hilir, dapat dilihat pada Gambar 3.9.



Gambar 3.9 *Sequence Diagram* Fitur Produksi Aset Gambar

Berdasarkan Gambar 3.9, *Sequence diagram* modul utama pembuatan aset visual menggambarkan interaksi antarkomponen di dalam sistem yang meliputi *User*, Telegram *Bot*, n8n *Engine*, dan *External Services*. Alur proses diawali ketika pengguna mengirimkan *prompt* pembuatan gambar (menggunakan kata kunci seperti "Buat" atau "Buatkan") ke Telegram *Bot*, yang kemudian meneruskannya ke n8n *Engine*. n8n *Engine* lalu mengirimkan dua *feedback* secara bertahap ke Telegram *Bot*, yaitu *feedback* pesan diterima dan *feedback* deteksi *generate*, untuk ditampilkan kepada pengguna sebagai pembaruan status awal.

Setelah itu, n8n *Engine* melakukan pemrosesan internal melalui *prompt formatter*, dilanjutkan dengan *prompt enhancer* menggunakan Gemini 2.5 Flash pada *External Services*, serta proses *generate* gambar menggunakan FLUX 1.1 Pro. Gambar hasil generasi yang diterima oleh n8n Engine terlebih dahulu disimpan ke *bucket raw assets* Supabase. Selanjutnya, proses *upscale* dilakukan dengan memanfaatkan tautan (*link*) gambar yang tersimpan di *bucket raw assets* tersebut. Setelah pemrosesan *upscale* selesai, n8n Engine melakukan konversi format gambar dari WEBP ke JPG, lalu menyimpan berkas *final* berformat .jpg tersebut ke dalam *bucket upscaled assets* Supabase. Selanjutnya, Gemini AI *Agent* mengakses tautan (*link*) gambar dari *bucket upscaled assets* tersebut untuk melakukan generasi *metadata*. Setelah *title* dan *keywords* berhasil dibuat, n8n Engine mengirimkan seluruh hasil aset gambar beserta *metadata*nya ke Telegram Bot untuk ditampilkan secara utuh pada layar obrolan pengguna.

3.3.6 Perancangan Penanganan Kesalahan

Perancangan penanganan kesalahan bertujuan untuk memastikan sistem tetap stabil dan dapat memberikan umpan balik (*feedback*) yang informatif kepada pengguna ketika terjadi kendala operasional. Rincian skenario validasi dan penanganan kesalahan pada sistem disajikan pada Tabel 3.5.

Tabel 3.5 Skenario Penanganan Kesalahan

No	Jenis Kesalahan / Validasi	Kondisi Pemicu (*Trigger*)	Tindakan Sistem
1	Validasi Format Perintah (*Invalid Command*)	Pengguna mengirimkan pesan atau perintah yang tidak dikenali/tidak sesuai dengan sintaks yang ditentukan oleh *bot*.	Sistem mengirimkan pesan balasan berupa panduan format yang benar serta menampilkan kembali menu panduan fitur.
2	Validasi Kebijakan Konten (*Content Policy Violation*)	*Prompt* atau instruksi pembuatan gambar dari pengguna terdeteksi mengandung kata/konten sensitif yang melanggar kebijakan.	Sistem membatalkan proses *generate* dan menampilkan notifikasi, bahwa *prompt* melanggar kebijakan konten.
3	Penanganan Kesalahan Jaringan & API (API / *Network Failure*)	Terjadi kegagalan koneksi jaringan, *timeout*, atau layanan pihak ketiga (Supabase, Telegram, Replicate, AI *Engine*) mengalami *downtime*.	Konfigurasi o*n error (Continue)* pada *node* diaktifkan untuk menangkap kegagalan (*exception*), lalu meneruskan alur ke *node* Telegram guna mengirimkan notifikasi pesan galat sistem ke pengguna.

Berdasarkan Tabel 3.5, perancangan penanganan kesalahan dikategorikan ke dalam tiga skenario utama untuk menjaga keandalan sistem. Pertama, validasi format perintah menangani masukan pengguna yang tidak sesuai sintaks dengan memberikan panduan penggunaan yang benar. Kedua, validasi kebijakan konten bertugas menghentikan proses generasi secara otomatis apabila *prompt* terdeteksi

melanggar aturan keamanan atau mengandung kata sensitif. Ketiga, penanganan kesalahan jaringan dan API memanfaatkan konfigurasi *On Error (Continue)* pada *workflow* untuk menangkap kegagalan koneksi maupun *downtime* layanan pihak ketiga, sehingga sistem tetap dapat mengirimkan notifikasi pesan galat yang informatif kepada pengguna tanpa menghentikan alur kerja secara mendadak.

3.4 Pembuatan Prototipe

Tahap pembuatan prototipe merupakan tahap inti dari model pengembangan *prototyping*. Pada tahap ini, rancangan sistem yang telah disusun diimplementasikan ke dalam bentuk *workflow* fungsional pada n8n yang dapat diuji dan disempurnakan secara iteratif. Proses pembuatan prototipe dilakukan melalui beberapa siklus iterasi hingga sistem dinilai memenuhi seluruh kebutuhan fungsional dan nonfungsional yang telah ditetapkan.

3.4.1 Proses Iterasi Prototipe

Sesuai dengan metode *prototyping*, proses pengembangan sistem dilakukan secara iteratif melalui siklus berulang yang mencakup pembangunan prototipe *workflow*, pengujian fungsionalitas, identifikasi kendala teknis, hingga tahap penyempurnaan. Setiap iterasi menghasilkan versi prototipe yang lebih stabil berdasarkan evaluasi komprehensif terhadap 31 komponen *nodes* yang terdaftar pada Tabel 3.5. Fokus utama penyesuaian berulang ini mencakup penyempurnaan alur logika dan pengondisian sistem pada 20 *native nodes*, optimasi *prompting* serta keluaran *metadata* berbahasa Inggris menggunakan Google Gemini 2.5 Flash Lite pada 3 *external* API *sub-nodes*, hingga otomatisasi eksekusi REST API untuk pembuatan gambar FLUX 1.1 Pro, *upscaling* Real-ESRGAN, dan penyimpanan Supabase *storage* pada 8 HTTP *Request nodes*.

Selain optimasi fungsi utama, iterasi juga difokuskan pada penerapan mekanisme *exception handling* dan validasi kesalahan untuk menangani potensi kegagalan pemanggilan API (*rate limit*), pelanggaran kebijakan konten (*content policy*), hingga ketidaksesuaian format atau instruksi pesan dari pengguna yang tidak sesuai ketentuan *prompt* sistem. Penyesuaian secara berulang ini sangat

krusial, karena keberhasilan otomatisasi latar belakang (*single-trigger*) dan kualitas luaran aset visual sangat bergantung pada ketepatan alur logika *workflow,* serta interaksi antar-*node*. Siklus iterasi dihentikan ketika seluruh komponen *nodes* pada Tabel 3.5 teruji mampu memenuhi seluruh kebutuhan fungsional secara konsisten dan stabil.

3.5 Uji Coba Sistem

Tahap uji coba dilakukan setelah prototipe dinyatakan stabil untuk memvalidasi performa *agentic workflow* secara menyeluruh. Pengujian ini berfokus pada tiga aspek utama, yaitu mengukur kualitas luaran (kesesuaian gambar dan *metadata*), menganalisis kinerja sistem (*latency*, *bottleneck*, dan *throughput*), serta mengukur efisiensi waktu dan usaha operasional (*effort*) yang berhasil dipangkas dibandingkan dengan alur pengerjaan secara manual.

Proses pengujian dieksekusi menggunakan 15 skenario *prompt* uji coba yang terbagi ke dalam tiga tingkat kompleksitas (*prompt* rendah, sedang, dan tinggi) untuk memastikan konsistensi serta keandalan sistem. Rincian mengenai pemetaan fokus pengujian, skenario, metrik yang diukur, serta bentuk luaran disajikan pada Tabel 3.6.

Tabel 3.6 Skenario Pengujian dan Evaluasi Sistem

No	Fokus Pengujian	Skenario Pengujian	*Output* Pengujian
1	Kualitas Luaran (*Output*)	Menguji eksekusi *prompt* berdasarkan variasi tingkat kompleksitas (rendah, sedang, tinggi).	Status relevansi (Sesuai/Tidak Sesuai) dan bukti fisik luaran.
2	Kinerja Sistem (*Performance*)	Mencatat durasi waktu pemrosesan pada tiap *node workflow* saat eksekusi berlangsung.	Evaluasi kinerja sistem berdasarkan durasi pemrosesan total (*latency*), jumlah permintaan per waktu (*throughput*), serta

No	Fokus Pengujian	Skenario Pengujian	*Output* Pengujian
			hambatan eksekusi (*bottleneck*).
3	Efisiensi Waktu & Beban Kerja	Membandingkan *baseline* alur manual (1 kali pemetaan) dengan sistem otomatisasi berdasarkan nilai rata-rata yang diperoleh dari 15 kali percobaan pengujian.	Perbandingan durasi waktu kerja total, serta jumlah interaksi fisik (*click*, *typing*).

Berdasarkan Tabel 3.6, perancangan skenario pengujian dan evaluasi sistem dibagi ke dalam tiga fokus utama. Pertama, pengujian kualitas luaran (output) dilakukan dengan mengeksekusi 15 skenario prompt pada berbagai tingkat kompleksitas untuk memvalidasi kesesuaian hasil gambar dan ketepatan metadata visual yang dihasilkan. Kedua, pengujian kinerja sistem (*performance*) mencakup pengukuran *latency* (durasi respons total dan per *node* dalam satuan waktu), *throughput* (kapasitas volume eksekusi tugas), serta bottleneck (identifikasi titik hambatan) pada node yang memiliki waktu proses paling lama. Ketiga, pengujian efisiensi waktu dan beban kerja dilakukan dengan membandingkan baseline alur pengerjaan secara manual terhadap sistem otomatisasi, berdasarkan nilai rata-rata dari 15 kali percobaan untuk mengukur penurunan waktu pemrosesan total dan pengurangan interaksi fisik pengguna (click, typing).

3.5.1 Pengujian Kualitas Luaran

Pengujian kualitas luaran dilakukan untuk mengevaluasi sejauh mana hasil generasi aset visual dan ekstrak *metadata* yang diproses oleh *agentic workflow* memenuhi kriteria kelayakan serta relevansi sesuai instruksi (*prompt*) yang diberikan. Pengujian ini terbagi menjadi dua fokus evaluasi utama, yaitu kualitas visual gambar dan ketepatan *metadata* (*title* dan *keywords*).

1. Pengujian Kualitas Gambar

   Rancangan struktur pencatatan data dan variabel yang dievaluasi pada pengujian kualitas visual gambar dijelaskan secara rinci pada Tabel 3.7.

Tabel 3.7 Variabel Evaluasi Kualitas Visual Gambar

No	Elemen yang Diuji	Fungsi Elemen	Kriteria Pengisian Data
1	*Prompt*	Instruksi teks masukan yang diberikan ke *workflow.*	Teks *prompt* masukan.
2	Tingkat	Klasifikasi kompleksitas *prompt.*	Tingkat kompleksitas *prompt* (rendah, sedang, atau tinggi).
3	Durasi Waktu (Detik)	Catatan durasi waktu eksekusi otomatis dalam satuan detik dari pemicu awal hingga respons dikirimkan ke pengguna.	Catatan durasi waktu eksekusi otomatis dalam satuan detik.
4	Bukti Fisik *Output*	Luaran visual yang dihasilkan oleh sistem.	Tampilan aset gambar hasil generasi yang telah melalui proses *upscaling.*
	Dimensi Gambar / Resolusi	Ukuran piksel berkas akhir hasil pemrosesan *upscaling* Real-ESRGAN.	Nilai panjang × lebar gambar dalam piksel (2048 × 2048 / 2K).
5	Status Relevansi	Derajat kesesuaian visual terhadap *prompt.*	Evaluasi kualitatif (Sesuai atau Tidak Sesuai).

Berdasarkan struktur pada Tabel 3.7, seluruh data dari 15 kali eksekusi *prompt* akan dicatat secara sistematis. Pencatatan tersebut mencakup pengelompokan *prompt* berdasarkan tingkat kompleksitasnya, pemuatan bukti fisik aset gambar yang dihasilkan, dimensi atau resolusi gambar, serta evaluasi status relevansi visual secara kualitatif.

2. Pengujian *Metadata* (Judul dan Kata Kunci)

   Rancangan struktur pencatatan data dan variabel yang dievaluasi pada pengujian *metadata* disajikan secara rinci pada Tabel 3.8.

Tabel 3.8 Variabel Evaluasi Akurasi *Metadata*

No	Elemen yang Diuji	Fungsi Elemen	Kriteria Pengisian Data
1	*Prompt*	Instruksi teks masukan yang diberikan ke *workflow.*	Teks *prompt* masukan.
2	Judul (*Title*)	Hasil penamaan judul deskriptif otomatis oleh sistem.	Teks judul berbahasa Inggris yang dihasilkan oleh LLM.
3	Kata Kunci (*Keywords*)	Hasil ekstrak daftar *tag* relevan oleh sistem.	Sampel 10 kata kunci utama berbahasa Inggris yang dipisahkan koma (catatan: tabel menampilkan 10 kata kunci teratas untuk keterbacaan, sedangkan luaran penuh berkisar 40–50 kata kunci disajikan pada Lampiran).
4	Status Relevansi	Derajat kesesuaian *metadata* terhadap objek gambar.	Evaluasi kualitatif (Sesuai atau Tidak Sesuai).

Berdasarkan struktur pada Tabel 3.8, seluruh data luaran *metadata* dari 15 kali eksekusi *prompt* akan dicatat secara sistematis. Pencatatan tersebut meliputi pemuatan teks *title*, pencantuman sampel *keywords* utama yang

relevan, serta evaluasi kualitatif terhadap ketepatan *metadata* yang dihasilkan oleh sistem.

3.5.2 Analisis Kinerja Sistem

Pengujian kinerja sistem bertujuan untuk mengukur efisiensi pemrosesan *agentic workflow* secara kuantitatif. Pengujian dilakukan dengan mengamati catatan durasi eksekusi (*execution history*) pada dasbor n8n dari 15 skenario *prompt* uji coba. Tiga metrik utama yang diukur mencakup rata-rata latensi, analisis *bottleneck*, dan *throughput* sistem.

1. Rata-Rata Latensi Total Sistem

   Latensi mengukur total durasi waktu yang dibutuhkan sistem mulai dari perintah pemicu dikirim hingga luaran akhir yang diterima kembali oleh pengguna. Formulasi perhitungan rata-rata latensi dirumuskan sebagai berikut:

$$\bar{L} = \frac{\sum_{i=1}^{n}(T_{\text{selesai},i} - T_{\text{mulai},i})}{n} \qquad (3.1)$$

   Keterangan:

   a. $\bar{L}$: Nilai rata-rata latensi total sistem (detik).
   b. $T_{\text{mulai,i}}$: Waktu awal pemicu Telegram *Bot* Aktif untuk aset ke-i.
   c. $T_{\text{selesai,i}}$: Waktu akhir saat hasil aset ke-i dikirim ke pengguna.
   d. n : Total sampel aset gambar.

2. Persentase Kontribusi Durasi *Node* (*Bottleneck*)

   Persentase kontribusi durasi mengukur besarnya proporsi waktu eksekusi dari suatu *node* atau fase spesifik terhadap total waktu latensi sistem. Perhitungan ini digunakan untuk mengidentifikasi fase yang menjadi titik hambat kerja (*bottleneck*) dalam alur kerja otomatisasi. Formulasi untuk menghitung persentase kontribusi durasi *node* dirumuskan sebagai berikut:

$$P_{node} = \frac{T_{node}}{\bar{L}} \times 100\% \qquad (3.2)$$

Keterangan:

a. P_{node} : Persentase kontribusi durasi waktu *node* (%).
b. T_{node} : Durasi waktu eksekusi rata-rata *node* spesifik.
c. $\bar{L}$: Nilai rata-rata latensi total sistem.

3. *Throughput* Sistem

Throughput mengukur estimasi kapasitas total aset gambar yang dapat diproduksi secara otomatis oleh sistem dalam satuan waktu satu jam. Formulasi untuk menghitung nilai *throughput* dirumuskan sebagai berikut:

$$H = \frac{3600}{\bar{L}} \quad (3.3)$$

Keterangan:

a. H : Nilai *throughput* sistem (aset/jam).
b. 3600 : Konstanta konversi waktu dalam detik (1 jam).
c. $\bar{L}$: Nilai rata-rata latensi total sistem (detik).

Selanjutnya, rancangan struktur pencatatan data dan parameter evaluasi performa waktu eksekusi untuk setiap fase alur kerja (*node*) dijelaskan secara rinci pada Tabel 3.9.

Tabel 3.9 Parameter Evaluasi Kinerja Waktu Sistem

No	Elemen yang Diuji	Fungsi Elemen	Kriteria Pengisian Data
1	Fase Alur Kerja Sistem	Tahapan/kelompok *node* yang dieksekusi oleh n8n.	Nama fase alur kerja (w*orkflow phase*).
2	Rata-Rata Latensi (Detik)	Durasi pemrosesan rata-rata per fase dari 15 kali uji coba.	Catatan waktu eksekusi dalam satuan detik.

No	Elemen yang Diuji	Fungsi Elemen	Kriteria Pengisian Data
3	Persentase	Proporsi durasi fase terhadap total latensi.	Nilai persentase kontribusi waktu (%).

Berdasarkan struktur pada Tabel 3.9, seluruh data pengukuran kinerja waktu dari 15 kali eksekusi akan dicatat secara sistematis untuk mengidentifikasi titik hambat (*bottleneck*), serta efisiensi pemrosesan sistem secara keseluruhan.

3.5.3 Analisis Efisiensi

Pengujian efisiensi bertujuan untuk mengukur tingkat efisiensi yang dihasilkan oleh *agentic workflow* dibandingkan alur kerja konvensional (manual). Seluruh skenario pengujian, baik metode manual maupun sistem otomatisasi, dilaksanakan secara konsisten pada lingkungan perangkat *desktop* (*PC/Laptop*). Penetapan lingkungan ini disesuaikan dengan salah satu perangkat lunak pada alur manual, yaitu VideoProc untuk proses peningkatan resolusi gambar, yang merupakan aplikasi berbasis *desktop*. Selain itu, pengujian efisiensi untuk pengukuran *effort* maupun pencatatan waktu dimulai saat seluruh situs *web* dan aplikasi pendukung telah dibuka. Pengujian efisiensi ini berfokus pada dua dimensi evaluasi utama, yaitu efisiensi waktu produksi dan efisiensi beban kerja operasional (*effort*).

1. Efisiensi Waktu Produksi

   Komponen waktu manual yang dihitung secara terpisah merujuk pada aktivitas dalam alur produksi aset, meliputi generasi gambar, peningkatan resolusi gambar, serta pembuatan *metadata*. Berlandaskan pada adanya variasi performa komputasi *antarplatform* yang memakan waktu dalam satuan menit dan detik, seluruh pencatatan durasi manual dikonversi secara seragam ke dalam satuan detik. Konversi standar satuan waktu tersebut dirumuskan sebagai berikut:

$$T_{\text{total_manual}} = T_{\text{Gemini}} + T_{\text{Kling}} + T_{\text{Videoproc}} \qquad (3.4)$$

Keterangan:

a. $T_{\text{total_manual}}$: Total durasi pengerjaan manual (detik).
b. T_{Gemini} : Durasi proses pada Gemini AI (detik).
c. T_{Kling} : Durasi proses pada Kling AI (detik).
d. $T_{Videoproc}$: Durasi proses pada VideoProc (detik).

Setelah total waktu dari kedua metode didapatkan, analisis tingkat efisiensi waktu komparatif dihitung dalam satuan persentase. Formulasi matematika yang digunakan untuk mengukur performa akselerasi sistem dinyatakan melalui rumus berikut:

$$E_t = \frac{T_{\text{total_manual}} - T_{workflow}}{T_{\text{total_manual}}} \times 100\% \qquad (3.5)$$

Keterangan:

a. E_t : Persentase tingkat efisiensi waktu (%).
b. $T_{workflow}$: Total durasi pengerjaan sistem otomatisasi (detik).
c. $T_{\text{total_manual}}$: Total durasi pengerjaan manual (detik).

2. Efisiensi Beban Kerja Operasional (*Effort*)

Pengujian efisiensi *effort* mengukur besarnya penghematan interaksi fisik pengguna seperti aktivitas klik dan ketik antara metode manual dan otomatis selama alur produksi aset berlangsung. Total estimasi interaksi fisik metode manual dihitung menggunakan persamaan berikut:

$$I_{total_manual} = I_{klik} + I_{ketik} \qquad (3.6)$$

Keterangan:

a. I_{total_manual} : Total interaksi fisik metode manual.
b. I_{klik} : Total aktivitas klik.

c. I_{ketik} : Total aktivitas ketik.

Selanjutnya, persentase penurunan beban kerja operasional (*effort*) dihitung menggunakan persamaan berikut:

$$E_e = \frac{N_{\text{total_manual}} - N_{\text{total_otomatis}}}{N_{\text{total_manual}}} \times 100\% \qquad (3.7)$$

Keterangan:

a. E_e : Persentase tingkat efisiensi beban kerja (%).

b. $N_{\text{total_otomatis}}$: Total interaksi fisik metode otomatis.

c. $N_{\text{total_manual}}$: Total interaksi fisik metode manual.

Selanjutnya, rancangan struktur pencatatan data dan variabel evaluasi efisiensi antara alur kerja manual dan otomatis disajikan secara rinci pada Tabel 3.10.

Tabel 3.10 Parameter Perbandingan Efisiensi Manual dan Otomatis

No	Dimensi Efisiensi	Metode Manual	Sistem Otomatisasi	Dampak Efisiensi
1	Waktu Produksi	Total waktu pengerjaan manual	Total waktu eksekusi otomatis	Persentase efisiensi waktu
2	Interaksi Pengguna	Total interaksi fisik	Total interaksi fisik	Persentase penurunan *effort*

Berdasarkan Tabel 3.10, evaluasi perbandingan efisiensi dipetakan ke dalam dua dimensi utama. Dimensi pertama adalah waktu produksi, yang membandingkan durasi pengerjaan manual terhadap total waktu eksekusi otomatisasi untuk menghitung persentase efisiensi waktu yang diperoleh. Dimensi kedua adalah interaksi pengguna, yang membandingkan jumlah interaksi fisik pada

metode manual dengan sistem otomatisasi guna mengukur persentase penurunan usaha operasional (*effort*).

3.6 Evaluasi Sistem

Tahap evaluasi dilakukan untuk memverifikasi fungsionalitas, kualitas luaran (*output*), kinerja, dan efisiensi dari alur *agentic workflow* agar sesuai dengan spesifikasi kebutuhan sistem yang telah dirancang. Proses evaluasi ini dirancang secara menyeluruh untuk memastikan sistem dapat bekerja secara optimal, stabil, serta memberikan manfaat yang terukur bagi pengguna.

Evaluasi fungsionalitas sistem dilaksanakan melalui pendekatan *black-box testing*, di mana respons dan luaran sistem diuji berdasarkan variasi instruksi masukan (*input*) tanpa melibatkan analisis terhadap struktur internal *node*. Rincian skenario pengujian tersebut disajikan pada Tabel 3.11.

Tabel 3.11 Skenario Pengujian *Black-Box Testing* Sistem

No	Fitur Yang Diuji	Skenario Pengujian	Hasil Yang Diharapkan
1	Notifikasi Konfirmasi Perintah	Pengguna mengirimkan pesan perintah atau instruksi melalui *bot* Telegram.	Sistem berhasil mengonfirmasi pesan, mendeteksi jenis permintaan (riset/generasi gambar), dan mengirimkan notifikasi bahwa proses sedang berjalan.
2	Riset Tren Pasar	Pengguna mengirimkan pesan instruksi riset tren pasar melalui *bot* Telegram.	AI Gemini 2.5 Flash Lite menganalisis tren pasar dan menghasilkan laporan riset tren pasar.mengekstrak

No	Fitur Yang Diuji	Skenario Pengujian	Hasil Yang Diharapkan
			metadata (Gemini), dan mengirim berkas final via Telegram.
3	Produksi Gambar, *Metadata*, Penyimpanan, dan Pengiriman	Pengguna mengirimkan perintah pembuatan gambar (*prompt*).	Sistem mengoptimalkan *prompt* (Gemini 2.5 Flash), menghasilkan gambar (FLUX 1.1 Pro), *upscale* ke 2K (Real-ESRGAN), mengekstraksi *metadata title* dan minimal 40 *keywords* (Gemini 2.5 Flash Lite), menyimpan file ke Supabase, serta mengirimkan hasil ke pengguna via Telegram.
4	Manajemen Menu panduan	Pengguna mengirimkan perintah /*start.*	Sistem berhasil memberikan respons konfirmasi, bahwa pesan telah diterima dan menampilkan pesan menu panduan.
5	Penanganan Kesalahan Format Perintah	Pengguna mengirimkan perintah salah format.	Sistem berhasil memberikan respons *feedback* berupa petunjuk instruksi yang benar.
6	Penanganan	Pengguna mengirimkan	Sistem berhasil

No	Fitur Yang Diuji	Skenario Pengujian	Hasil Yang Diharapkan
	Kesalahan Kebijakan *Prompt*	*prompt* melanggar kebijakan.	menolak membuat gambar dan memberikan respons *feedback* berupa notifikasi peringatan kebijakan.
7	Penanganan Kesalahan Akses API	Terjadi gangguan pada akses API eksternal.	Sistem berhasil memberikan respons *feedback* berupa notifikasi terjadinya kesalahan teknis.
8	Manajemen Kinerja dan Efisiensi	Pengguna memberikan satu tindakan pemicu (*single-trigger*).	Sistem berhasil mengeksekusi seluruh alur otomatisasi di latar belakang dan menghasilkan gambar, serta *metadata* berkualitas secara relevan.

Berdasarkan rincian kebutuhan sistem pada Tabel 3.11, selain *black-box testing*, evaluasi juga mencakup penilaian kualitatif luaran citra, pengujian kinerja kuantitatif, analisis efisiensi, evaluasi aspek etika dan hak cipta hingga kelebihan dan kekurangan pembuatan gambar AI.

4. HASIL DAN PEMBAHASAN

4.1 Implementasi Sistem

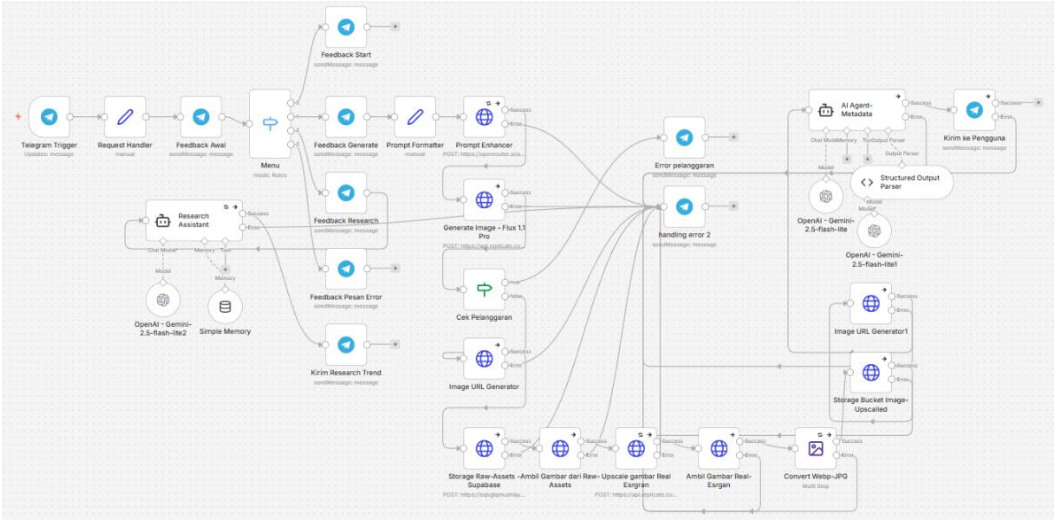
Pada tahap ini, rancangan arsitektur sistem, integrasi API model *generative* AI, modul *upscaling*, ekstraksi *metadata*, penyimpanan, serta antarmuka *bot* Telegram diimplementasikan menjadi sistem otomatisasi produksi aset *microstock* yang siap digunakan. Seluruh modul dihubungkan secara terintegrasi ke dalam alur kerja berbasis agen (*agentic workflow*) untuk memastikan proses berjalan secara mandiri dari awal hingga akhir. Implementasi ini bertujuan untuk merealisasikan sistem yang efisien dan responsif dalam merespons setiap instruksi dari pengguna.

Sistem ini dikembangkan menggunakan *platform workflow automation* n8n sebagai pusat kendali alur kerja, dengan Telegram *bot* API sebagai antarmuka interaksi utama pengguna. Sistem dirancang untuk menerima instruksi masukan dari pengguna melalui Telegram, mengeksekusi alur generasi gambar menggunakan model FLUX 1.1 Pro, melakukan peningkatan resolusi visual (*upscaling*) menggunakan model Real-ESRGAN, mengekstrak *metadata* berupa *title* dan *keywords* berbahasa Inggris secara otomatis menggunakan model Gemini 2.5 Flash Lite, serta mengembalikan berkas akhir ke pengguna.

Selain alur produksi gambar dan ekstraksi *metadata*, sistem ini juga dilengkapi dengan fitur pendukung berupa riset tren pasar secara otomatis berbasis analisis tren *microstock*, manajemen menu panduan utama melalui perintah /*start*, serta penanganan kesalahan (*error handling*) berbasis aturan (*rule-based feedback*). Fitur-fitur tersebut diimplementasikan untuk memangkas waktu pemrosesan manual, meminimalkan interaksi fisik (*effort*), meningkatkan keandalan sistem melalui penanganan kendala yang terstruktur, serta menjaga konsistensi kualitas aset sesuai standar *platform microstock* Adobe Stock.

Keseluruhan rancangan alur kerja otomatisasi tersebut diimplementasikan secara visual pada *canvas platform* n8n melalui penghubungan 31 *nodes* yang saling terintegrasi. Tampilan *canvas* ini memperlihatkan pemetaan alur data (*data flow*) secara menyeluruh, mulai dari penerimaan *trigger* pesan Telegram, pengondisian logika, pemanggilan REST API eksternal, hingga eksekusi *subnode*

kecerdasan buatan. Implementasi *canvas workflow* n8n secara utuh ditunjukkan pada Gambar 4.1.



Gambar 4.1 Arsitektur *Workflow* Otomatisasi pada n8n

Berdasarkan Gambar 4.1, arsitektur *workflow* otomatisasi pada *canvas* n8n mengintegrasikan seluruh komponen *nodes* ke dalam satu ekosistem yang saling terhubung secara terstruktur. Alur pemrosesan data (*data flow*) diawali dari *node* Telegram *trigger* yang menangkap pesan dari pengguna, lalu diteruskan ke *request handler* untuk memvalidasi variabel data dan *node feedback* awal untuk mengirim notifikasi konfirmasi penerimaan pesan. Selanjutnya, *node* menu bertipe *switch* mengevaluasi perintah pengguna dan membaginya ke beberapa cabang eksekusi. Jika pengguna mengirim perintah /*start*, alur mengaktifkan *node feedback start* untuk menampilkan panduan menu panduan, sedangkan perintah yang tidak *valid* akan diandalkan oleh *node feedback* pesan *error*.

Pada cabang riset tren, pesan diolah oleh *node feedback research* lalu dieksekusi oleh AI *agent research assistant* yang didukung sub-*node* LLM OpenAI - Gemini-2.5-*flash-lite2* serta *simple memory* untuk mengingat konteks percakapan. Hasil analisis berupa topik rekomendasi visual *microstock* kemudian dikirimkan kembali ke pengguna melalui *node* kirim *research trend*. Sementara itu, pada cabang generasi gambar, masukan pengguna dipicu oleh *node feedback generate*,

diformat oleh *prompt formatter*, dan diperkaya secara deskriptif oleh *node prompt enhancer* berbasis model Gemini 2.5 Flash melalui OpenRouter API sebelum dikirim ke *node generate image* - FLUX 1.1 Pro melalui Replicate *API* untuk menghasilkan gambar. Pemfilteran keamanan dan kebijakan konten dilakukan secara langsung oleh model FLUX 1.1 Pro pada saat proses generasi.

Selanjutnya, *node* cek pelanggaran bertugas menangkap status keluaran dari *node* FLUX 1.1 Pro tersebut. Apabila keluaran bernilai *failed*, sistem akan mengevaluasinya sebagai kondisi *true* (terjadi pelanggaran kebijakan). Sebaliknya, gambar yang berhasil lolos akan diekstrak tautannya oleh *image* URL *Generator* dan diunggah ke penyimpan *bucket* melalui *node storage raw-assets* - Supabase. Selanjutnya, *node* ambil gambar dari *bucket Raw-assets* mengambil tautan URL dari berkas yang tersimpan untuk kemudian diproses dalam peningkatan resolusi secara otomatis oleh *node upscale gambar* Real-ESRGAN berbasis Replicate API. Berkas hasil *upscale* kemudian diunduh melalui *node* ambil gambar Real-ESRGAN, dikonversi menjadi format JPG melalui *node convert* WEBP-JPG, dan disimpan ke *bucket* penyimpanan akhir menggunakan *node storage bucket image-upscaled*.

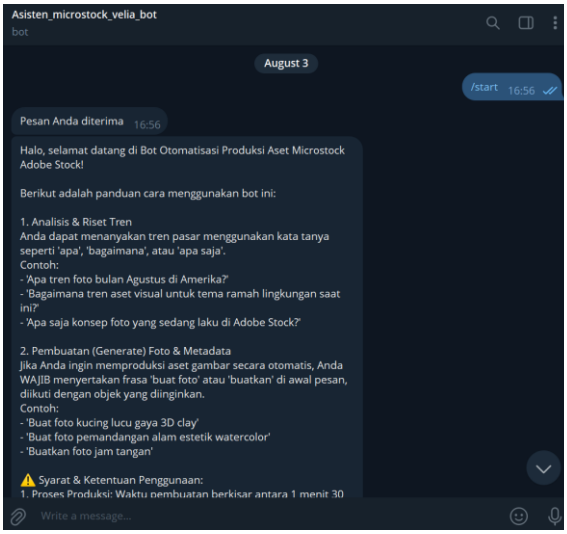
Pada tahap akhir, *node image* URL *generator1* mengekstrak tautan gambar JPG resolusi tinggi dari Supabase dan meneruskannya ke *node* AI *agent-metadata*. Agen AI ini terhubung dengan sub-*node* OpenAI - Gemini-2.5-flash-lite, serta *structured output parser* (yang didukung OpenAI - Gemini-2.5-flash-lite1). Kombinasi kedua modul ini bertugas menganalisis isi visual gambar, menyusun struktur format pesan yang akan dikirimkan kepada pengguna, serta menghasilkan *metadata* komersial berupa *title* dan *keywords* berbahasa Inggris dalam format JSON yang sudah rapih. Seluruh proses ditutup oleh *node* kirim ke pengguna yang menyatukan berkas gambar akhir, teks *metadata*, serta tautan (*link*) gambar ke percakapan Telegram pengguna untuk memudahkan peninjauan. Apabila terjadi kegagalan sistem atau pelanggaran konten di tengah jalan, *node* penanganan kesalahan seperti *error* pelanggaran dan *node handling error* 2 akan memutus alur dan mengirimkan pesan kegagalan yang informatif.

4.2 Implementasi Fitur *Bot* Telegram

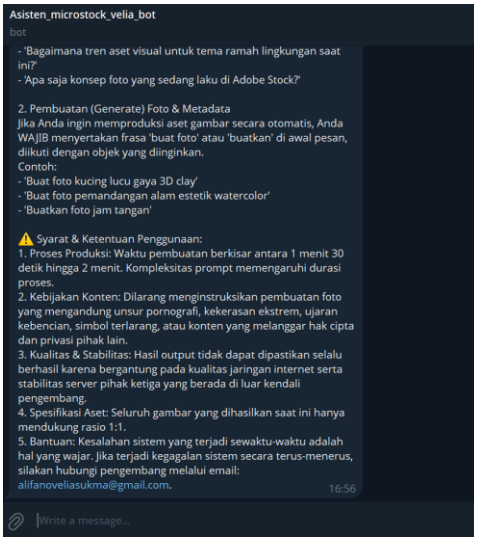
Implementasi antarmuka pengguna dirancang menggunakan Telegram *bot* API sebagai sarana interaksi utama. Pengguna dapat menjalankan seluruh alur produksi aset *microstock* cukup dengan mengirimkan perintah teks atau *prompt* ke akun *bot* Telegram. Berikut adalah bentuk implementasi dari setiap fitur yang ada pada sistem.

4.2.1 Fitur Menu panduan

Fitur ini diaktifkan ketika pengguna mengirimkan perintah /*start*. *Node* Telegram *trigger* akan menangkap perintah tersebut, lalu *node* menu memilah dan mengarahkannya ke *node feedback start*. Sistem memberikan pesan balasan berupa panduan penggunaan *bot*, daftar fungsi yang tersedia, serta format penulisan *prompt* yang disarankan. Tampilan antarmuka dari respon menu panduan ini dapat dilihat pada Gambar 4.2 dan Gambar 4.3.



Gambar 4.2 Antarmuka Menu panduan *Bot* (Bagian 1)



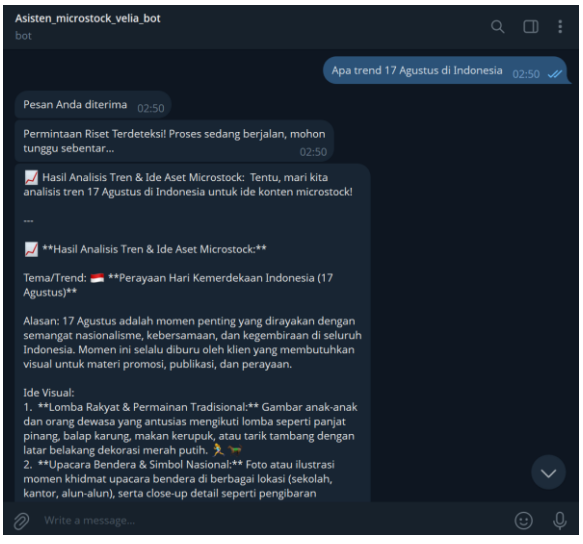
Gambar 4.3 Antarmuka Menu panduan *Bot* (Bagian 2)

Berdasarkan Gambar 4.2 dan Gambar 4.3, informasi yang disajikan mencakup panduan eksplorasi fitur riset tren pasar, tata cara penulisan instruksi produksi gambar berbasis frasa kunci, serta contoh format *prompt* yang *valid*.

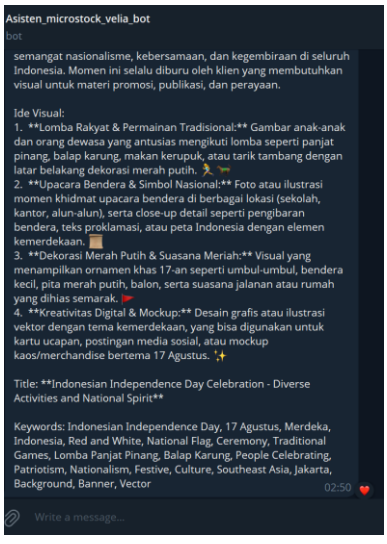
Selain itu, pesan ini memuat informasi penting terkait syarat dan ketentuan penggunaan, seperti estimasi waktu eksekusi sistem berkisar antara 45 detik hingga 2 menit dengan catatan bahwa tingkat kompleksitas *prompt* turut memengaruhi durasi pemrosesan. Sistem juga memberikan transparansi, bahwa keberhasilan luaran (*output*) tidak dapat dipastikan 100% selalu berhasil, karena sangat bergantung pada kualitas jaringan internet, serta stabilitas *server* API pihak ketiga yang berada di luar kendali pengembang. Ketentuan teknis lainnya mencakup batasan spesifikasi rasio gambar 1:1, kebijakan prapemrosesan konten (*safety filter*), hingga kontak bantuan pengembang jika terjadi kendala teknis.

4.2.2 Fitur Riset Tren Pasar *Microstock*

Fitur riset tren pasar berjalan saat pengguna mengirimkan kata kunci di Telegram, seperti "Apa tren 17 Agustus di Indonesia". Setelah pesan diterima, *node feedback* awal mengonfirmasi proses riset yang sedang berjalan. Pesan lalu diteruskan *node* menu ke *sub-workflow research assistant* berbasis Google Gemini 2.5 Flash Lite untuk menganalisis potensi pasar *microstock*. Hasil analisis tren tersebut ditunjukkan pada Gambar 4.4 dan Gambar 4.5.



Gambar 4.4 Antarmuka Menu Riset Tren (Bagian 1)



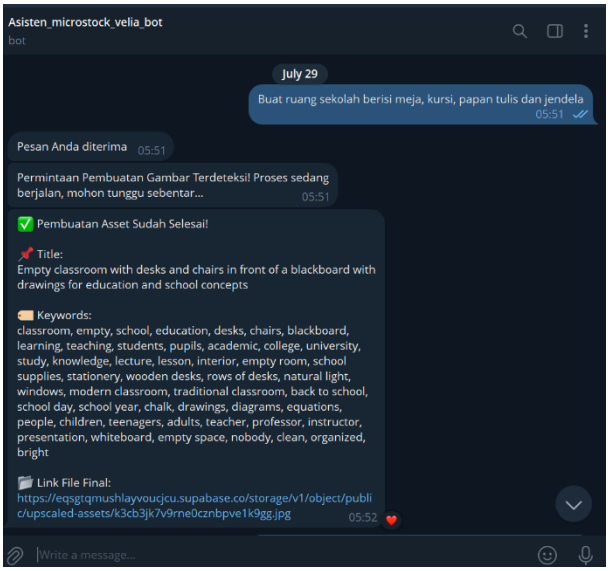
Gambar 4.5 Antarmuka Menu Riset Tren (Bagian 2)

Berdasarkan Gambar 4.4 dan Gambar 4.5, *bot* menyajikan laporan analisis

pasar komprehensif yang terstruktur secara sistematis. Informasi yang dihasilkan mencakup penetapan tema/tren (Perayaan Hari Kemerdekaan Indonesia), latar belakang alasan komersial kebutuhan aset, serta jabaran ide visual terperinci, seperti perlombaan tradisional, momen upacara bendera, dekorasi merah putih, hingga kreasi desain *digital*. Selain itu, sistem secara otomatis melengkapi laporan dengan rekomendasi judul berbahasa Inggris standar komersial (*"Indonesian Independence Day Celebration - Diverse Activities and National Spirit"*), serta daftar *keywords* relevan yang siap digunakan kontributor untuk optimasi pencarian aset pada *platform microstock* Adobe Stock.

4.2.3 Fitur Produksi Aset Visual

Fitur produksi aset diaktifkan saat pengguna mengirimkan perintah pembuatan gambar berbasis frasa kunci di awal pesan, seperti contoh instruksi "Buat ruang sekolah berisi meja, kursi, papan tulis dan jendela". Setelah pesan ditangkap oleh *node* Telegram *trigger*, *node feedback* awal dan *node feedback generate* memberikan respons notifikasi bertahap, bahwa permintaan pembuatan gambar terdeteksi dan alur otomatisasi sedang berjalan. Instruksi masukan pengguna kemudian diproses melalui *prompt formatter*, dan dioptimalkan oleh *prompt enhancer* berbasis OpenRouter API, sebelum dikirim ke model FLUX 1.1 Pro melalui Replicate API untuk proses generasi aset visual. Berkas visual yang dihasilkan selanjutnya ditingkatkan resolusinya menggunakan modul Real-ESRGAN, dikonversi ke format JPG, dan diunggah ke penyimpanan Supabase. Setelah itu, agen AI *agent-metadata* yang didukung model Google Gemini 2.5 Flash Lite menganalisis gambar akhir secara visual untuk mengekstrak *title* dan *keywords* komersial secara otomatis. Seluruh hasil produksi berupa tautan unduhan citra beresolusi tinggi beserta data *metadata* tersebut kemudian dikirimkan kembali ke ruang obrolan pengguna. Tampilan antarmuka hasil eksekusi alur produksi aset ini ditunjukkan pada Gambar 4.6.

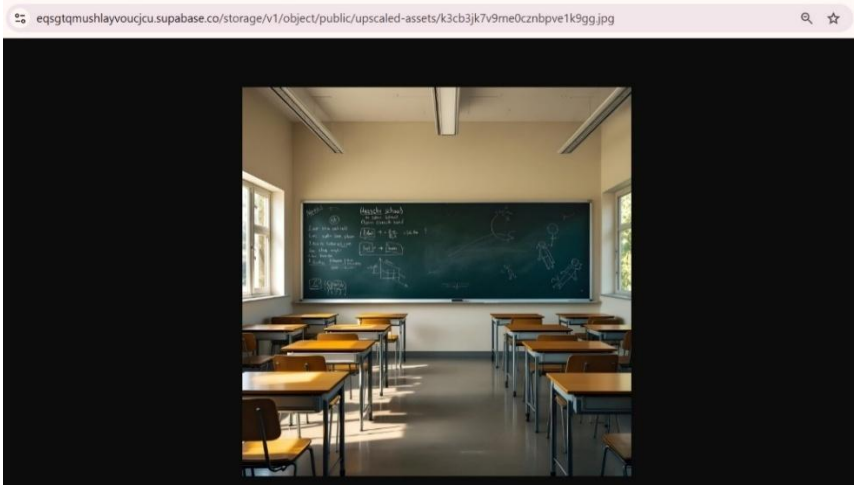


Gambar 4.6 Antarmuka Menu Produksi Aset

Berdasarkan Gambar 4.6, pesan balasan sistem menyajikan tiga luaran utama hasil produksi aset secara terstruktur, yaitu:

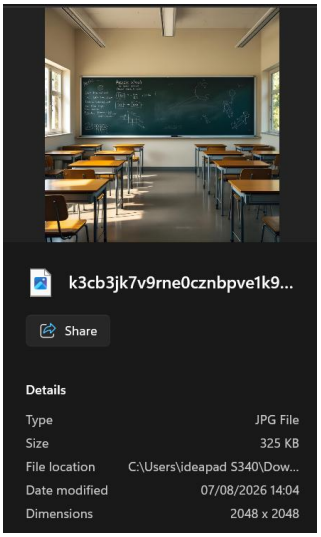
1. Judul deskriptif berbahasa Inggris standar industri *microstock* yang memuat deskripsi elemen visual secara detail (*"Empty classroom with desks and chairs in front of a blackboard with drawings for education and school concepts"*).
2. Puluhan kata kunci relevan berbahasa Inggris (*classroom*, *empty*, *school*, *education*, *desks*, *chairs*, *blackboard*, hingga *bright*) yang dipisahkan dengan tanda koma sehingga pengguna dapat langsung menyalinnya (*copy-paste*) ke *platform* Adobe Stock tanpa perlu penyuntingan manual satu persatu.
3. Tautan unduhan langsung (*direct* URL) yang memungkinkan pengguna mengunduh berkas gambar JPG hasil *upscale* beresolusi tinggi secara instan melalui peramban.

Ketika pengguna mengklik tautan *Link File Final* tersebut, peramban akan membuka dan menampilkan berkas gambar hasil *upscale* secara langsung. Tampilan pratinjau berkas gambar saat diakses melalui peramban ditunjukkan pada Gambar 4.7.



Gambar 4.7 Hasil Gambar

Berdasarkan Gambar 4.7, gambar ruang kelas hasil generasi dan *upscaling* dapat terbuka dengan sempurna di peramban *web* tanpa hambatan akses. Gambar tersebut selanjutnya diunduh ke dalam penyimpanan lokal perangkat untuk diperiksa rincian spesifikasi teknisnya. Rincian properti berkas gambar yang telah diunduh ditunjukkan pada Gambar 4.8.



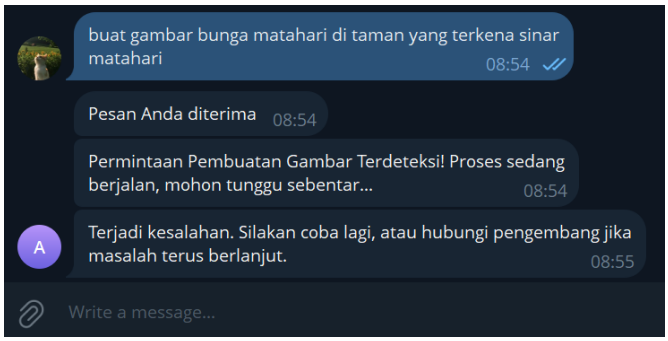
Gambar 4.8 *Detail* Berkas Gambar

Berdasarkan Gambar 4.8, hasil akhir dari alur otomatisasi produksi berupa berkas gambar ruang kelas terbukti berhasil diunduh dan tersimpan secara sempurna pada penyimpanan lokal perangkat. Berkas berformat JPG ini memiliki

spesifikasi teknis tinggi dengan resolusi 2048 x 2048 piksel dan ukuran berkas sebesar 325 Kb (*Kilobyte*), yang menampilkan *detail* visual jernih, tajam, serta bebas dari artefak kompresi, sehingga memenuhi standar kualitas komersial untuk langsung diunggah ke *platform* Adobe Stock. Penamaan berkas menggunakan string acak yang bersumber dari nama unik JSON. Langkah ini diterapkan sebagai skema keamanan untuk mencegah akses tak berizin (*direct file access*) dari pengguna tidak bertanggung jawab, mengingat keterbatasan kapasitas penyimpanan sistem yang masih tersentralisasi pada satu tempat.

4.2.4 *Error* Interupsi Jaringan atau Keterbatasan Akses API

Penerapan penanganan kesalahan pada bagian ini berfungsi untuk mengantisipasi adanya interupsi jaringan internal maupun ketika akses API pihak ketiga (seperti FLUX 1.1 Pro, Real-ESRGAN, atau Gemini AI) mengalami kegagalan respons atau batasan kuota (*rate limit*). Mekanisme ini memastikan sistem tidak mengalami *crash* saat terjadi masalah teknis di latar belakang. Bentuk respons umpan balik penanganan kesalahan yang ditampilkan kepada pengguna ditunjukkan pada Gambar 4.9.



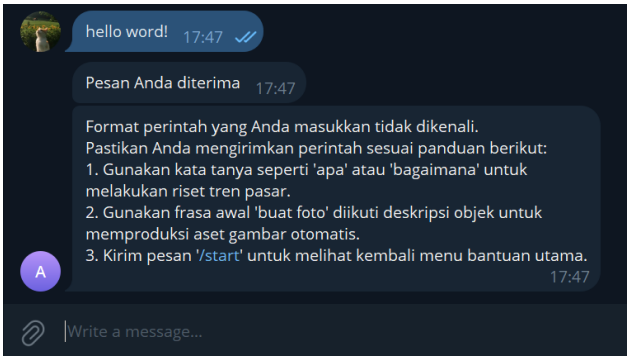
Gambar 4.9 Respon Kesalahan Interupsi Jaringan

Sebagaimana ditunjukkan pada Gambar 4.9, ketika proses riset tren pasar maupun produksi aset visual mengalami kendala eksekusi, *workflow* secara otomatis menangkap *error* (*catch error*) dan langsung mengirimkan pesan respons kepada pengguna: "Terjadi kesalahan. Silakan coba lagi, atau hubungi pengembang, jika masalah terus berlanjut." Hal ini memberikan umpan balik

(*feedback*) yang jelas pada setiap layanan, sehingga pengguna tidak menunggu tanpa kepastian status proses.

4.2.5 *Error* Kesalahan Format Perintah Pengguna

Penanganan kesalahan ini diaktifkan apabila instruksi atau pesan yang dikirimkan oleh pengguna tidak memenuhi struktur sintaks maupun format perintah yang dikenali oleh *bot* Telegram. Mekanisme ini memastikan *bot* tidak memproses instruksi yang ambigu atau tidak *valid*, sebagaimana ditunjukkan pada Gambar 4.10.

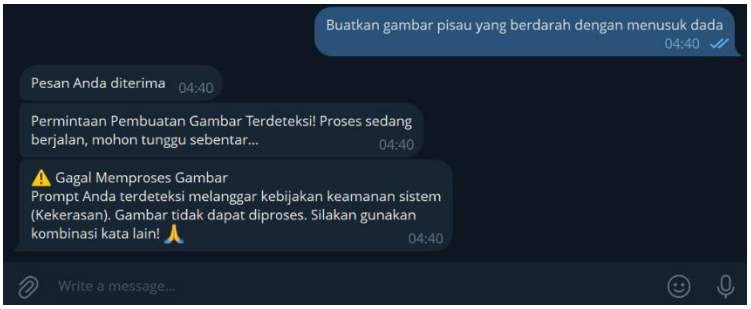


Gambar 4.10 Respon Kesalahan Format Perintah

Sebagaimana ditunjukkan pada Gambar 4.10, ketika pengguna mengirimkan pesan yang tidak sesuai dengan ketentuan instruksi, *workflow* akan secara otomatis menolak permintaan tersebut dan mengembalikan pesan notifikasi berupa panduan format yang benar. Umpan balik (*feedback*) ini membantu pengguna untuk memahami dan memperbaiki struktur instruksi yang dikirimkan.

4.2.6 *Error Filter* Keamanan Gambar

Fitur ini berfungsi untuk mengantisipasi dan menangani instruksi (*prompt*) dari pengguna yang terdeteksi mengandung kata kunci sensitif atau melanggar kebijakan keamanan konten pada model generatif AI (seperti FLUX 1.1 Pro), sebagaimana ditunjukkan pada Gambar 4.11.



Gambar 4.11 Respon Kesalahan Kebijakan Konten

Sebagaimana ditunjukkan pada Gambar 4.11, ketika *prompt* yang diajukan terdeteksi melanggar batas kebijakan keamanan, sistem akan menghentikan alur pemrosesan gambar secara aman sebelum dieksekusi lebih lanjut. Sistem kemudian memberi tahu pengguna melalui pesan notifikasi bahwa *prompt* tersebut tidak dapat diproses, karena batasan kebijakan konten.

4.3 Implementasi dan Pengujian Layanan Eksternal

Pada bagian ini disajikan proses pengujian integrasi *endpoint* REST API untuk setiap layanan eksternal yang digunakan dalam alur kerja sistem, mencakup model AI dan *storage*. Pengujian dilakukan secara independen menggunakan perangkat lunak Postman untuk memvalidasi metode pengiriman data (*POST*) dari masing-masing penyedia layanan.

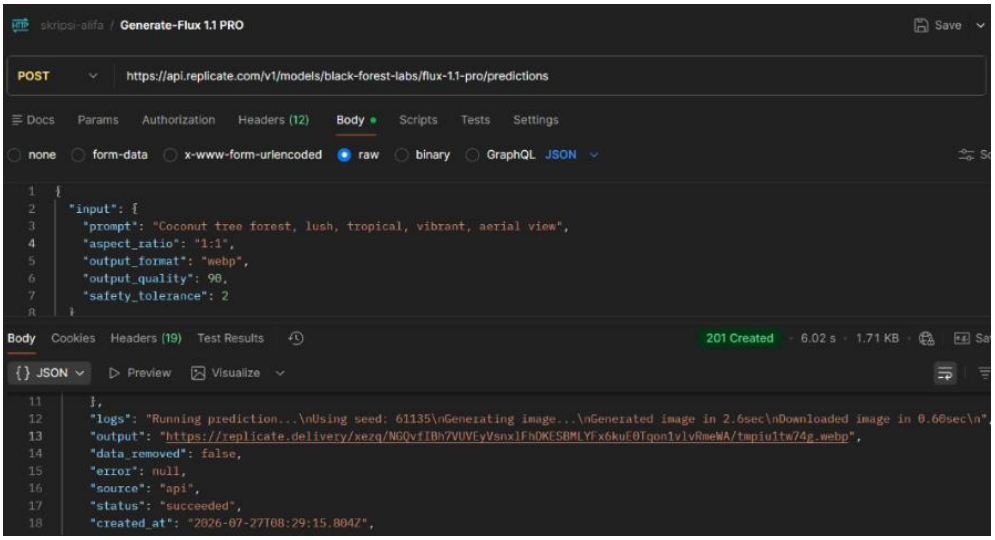
4.3.1 Pengujian REST API Menggunakan Postman

Pengujian *endpoint* REST API menggunakan perangkat lunak Postman dilakukan sebagai bukti implementasi teknis dalam memverifikasi alur komunikasi data, serta status keberhasilan respons dari setiap model AI. Pengujian ini menampilkan secara langsung bentuk panggilan API dan luaran JSON yang dihasilkan oleh masing-masing *provider*.

1. Implementasi API Model FLUX 1.1 Pro

   Pengujian model FLUX 1.1 Pro dilakukan melalui Replicate API dengan metode *POST* untuk menjalankan fungsi generasi gambar (*text-to-image*). Pengujian ini mengirimkan parameter masukan berupa *prompt* deskriptif

dan kunci otentikasi API (*bearer token*). Hasil pengujian *endpoint* ini ditunjukkan pada Gambar 4.12.

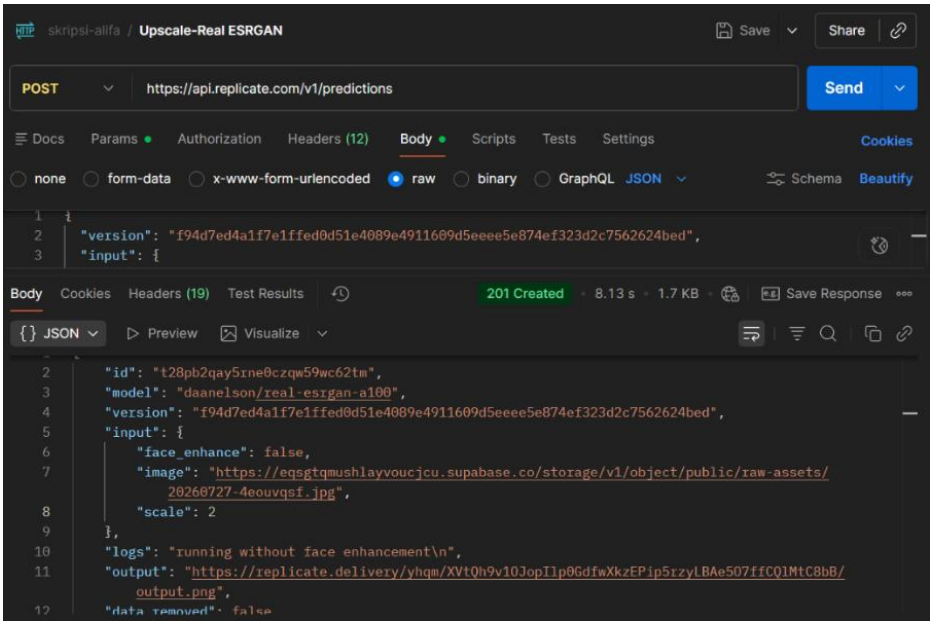


Gambar 4.12 Hasil Pengujian API FLUX 1.1 Pro

Berdasarkan hasil eksekusi pada Gambar 4.12, pengujian *endpoint* API FLUX 1.1 Pro menggunakan Postman berhasil mendapatkan respons status 201 *Created* dengan waktu pemrosesan selama 6,02 detik. Luaran respons JSON mengembalikan status "*succeeded*" beserta tautan URL berkas pada atribut "*output*" yang berisi citra hasil generasi yang sesuai dengan *prompt* masukan yang diberikan dengan format WEBP.

2. Implementasi API Model Real-ESRGAN

   Pengujian model Real-ESRGAN dilakukan melalui Replicate API dengan metode *POST* untuk fungsi peningkatan resolusi gambar (*super-resolution/upscaling*). Parameter masukan yang dikirimkan mencakup ID versi model, URL gambar awal dari Supabase *storage*, faktor perbesaran skala (scale: 2), serta kunci otentikasi API (*bearer token*). Tampilan hasil pengujian API ini disajikan pada Gambar 4.13.

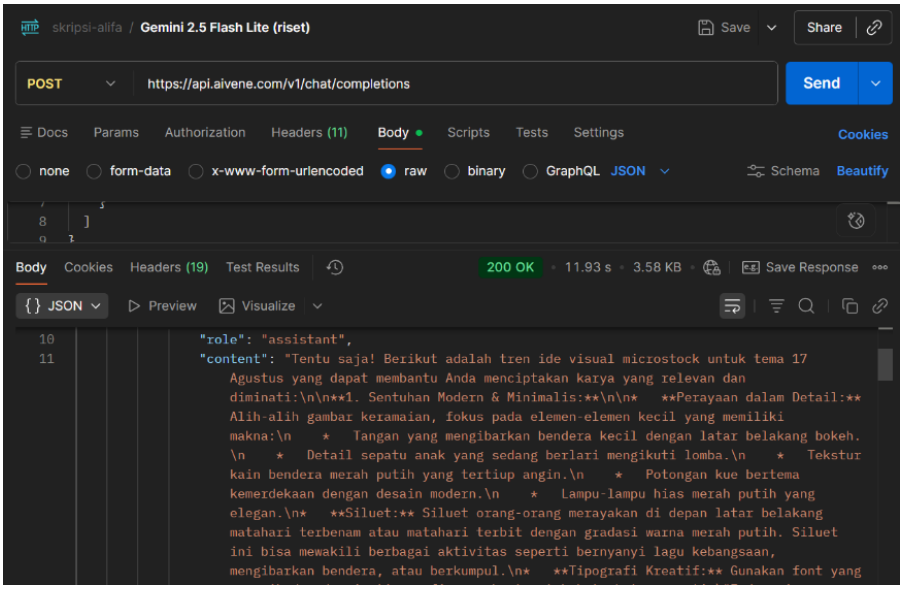


Gambar 4.13 Hasil Pengujian API Real-ESRGAN

Berdasarkan hasil eksekusi pada Gambar 4.13, pengujian *endpoint* API Real-ESRGAN menggunakan Postman berhasil mendapatkan respons status 201 *Created* dengan waktu pemrosesan selama 8,13 detik. Luaran respons JSON mengembalikan tautan URL berkas citra berformat PNG pada atribut "*output*" yang telah ditingkatkan skala resolusinya sebesar 2x (scale: 2) berdasarkan berkas gambar masukan awal yang bersumber dari Supabase *storage*.

3. Implementasi API Model Gemini 2.5 Flash Lite

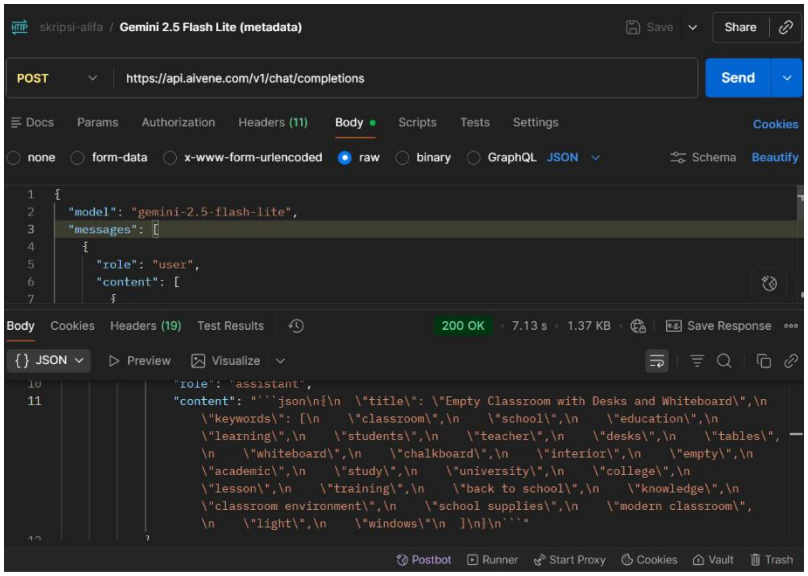
   Pengujian model Gemini 2.5 Flash Lite dilakukan melalui API *gateway* Aivene berbasis OpenAI-*Compatible* dengan metode *POST*. Pengujian ini difokuskan pada analisis riset tren pasar, ekstraksi *metadata* visual (*title* dan *keywords* berbahasa Inggris) berbasis masukan berkas citra, serta pembentukan struktur luaran pesan berformat JSON murni (*structured output*). Bukti eksekusi *endpoint* ini diperlihatkan pada Gambar 4.14.



Gambar 4.14 Hasil Pengujian API Gemini 2.5 Flash Lite (Riset)

Berdasarkan hasil eksekusi pada Gambar 4.14, pengujian *endpoint* API Gemini 2.5 Flash Lite untuk fungsi riset tren pasar menggunakan Postman berhasil mendapatkan respons status 200 OK dengan waktu pemrosesan selama 11,93 detik. Luaran respons JSON mengembalikan analisis ide konsep visual *microstock* yang berfokus pada tema perayaan Hari Kemerdekaan Indonesia (17 Agustus) sesuai dengan perintah (*prompt*) masukan yang diberikan. Penjelasan rinci mencakup rekomendasi sudut pandang visual, elemen-elemen spesifik yang diminati pasar, hingga potensi klasifikasi ide, seperti permainan tradisional dan dekorasi suasana. Kecepatan serta kelengkapan struktur data JSON ini membuktikan, bahwa *endpoint* tersebut dapat diintegrasikan dengan stabil ke dalam alur otomatisasi sistem tanpa mengalami kendala *timeout*.

Sementara itu, bukti eksekusi *endpoint* API Gemini 2.5 Flash Lite untuk fungsi ekstraksi *metadata* visual diperlihatkan pada Gambar 4.15.

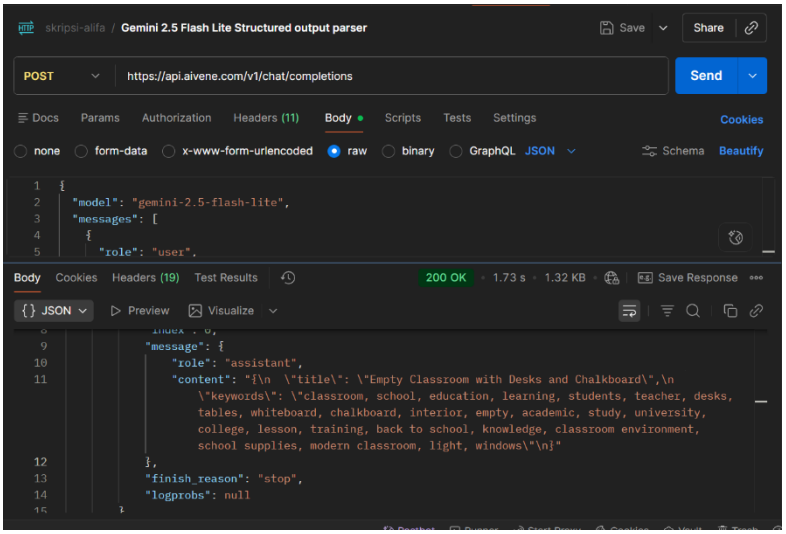


Gambar 4.15 Hasil Pengujian API Gemini 2.5 Flash Lite (*Metadata*)

Berdasarkan hasil eksekusi pada Gambar 4.15, pengujian *endpoint* API Gemini 2.5 Flash Lite untuk fungsi ekstraksi *metadata* menggunakan Postman berhasil mendapatkan respons status 200 OK dengan waktu pemrosesan selama 8,79 detik. Luaran respons JSON mengembalikan hasil analisis atribut visual citra dari Supabase *storage* berupa *title,* serta susunan kata kunci (*keywords*) berbahasa Inggris yang relevan untuk kebutuhan penamaan aset *microstock*. Hasil ekstraksi ini menunjukkan kelengkapan elemen deskriptif yang akurat, seperti penyebutan objek utama ruangan, perabot, hingga suasana kelas secara rinci. Akurasi penamaan dan pemilihan kata kunci ini sangat krusial dalam domain *microstock* untuk mengoptimalkan visibilitas pencarian, serta meningkatkan indeksasi aset pada *platform* agensi *global*. Selain itu, kecepatan respons di bawah 10 detik ini menegaskan efisiensi skema pemrosesan *multimodal* dalam menangani inferensi data berbasis citra secara langsung melalui tautan *storage*.

Selanjutnya, pengujian *endpoint* API Gemini 2.5 Flash Lite untuk fungsi *structured output parser*. Pengujian ini bertujuan untuk menguji keandalan sistem dalam merapikan format luaran mentah (*raw output*) yang sebelumnya masih memuat karakter berlebih (seperti *array* atau *escape characters*), menjadi objek JSON tunggal yang bersih, ringkas, dan

terstandarisasi tanpa mengubah substansi kata kunci, sebagaimana diperlihatkan pada Gambar 4.16.



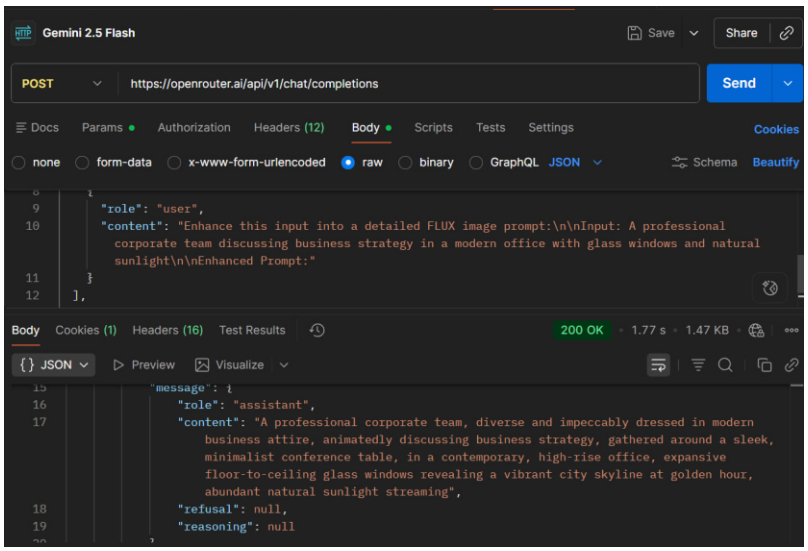
Gambar 4.16 Hasil Pengujian API Gemini 2.5 Flash Lite (Structured *Output*)

Berdasarkan hasil eksekusi pada Gambar 4.16, pengujian fungsi *structured output parser* melalui Postman berhasil mendapatkan respons status 200 OK. Pengujian ini membuktikan keandalan sistem dalam mengonversi teks *metadata* mentah (*raw string output*) hasil ekstraksi AI menjadi struktur JSON murni yang divalidasi sesuai skema. Dengan hilangnya karakter pembungkus (*escape characters*) dan pemformatan yang tidak standar, atribut *title* serta *keywords* kini tersusun secara konsisten dan terstruktur. Kepastian format data yang rapi dan terstandarisasi ini sangat penting untuk menjamin validitas data sebelum dikirimkan secara langsung oleh *node* Telegram kepada pengguna.

4. Implementasi API Model Gemini 2.5 Flash

   Pengujian model Gemini 2.5 Flash dilakukan melalui OpenRouter API *gateway* dengan metode *POST* untuk fungsi penajaman perintah (*prompt enhancement*). Pengujian ini mengirimkan parameter masukan berupa instruksi dasar pengguna, aturan format *prompt*, serta kunci otentikasi API (*bearer token*). Pada alur ini, teks instruksi awal yang sederhana dari

pengguna diperkaya dengan menambahkan *detail* spesifik, seperti pencahayaan, komposisi visual, suasana objek, serta gaya artistik agar sesuai dengan karakteristik generasi gambar pada model FLUX 1.1 Pro. Proses penajaman ini bertujuan untuk memastikan bahwa luaran gambar yang dihasilkan memiliki detail estetika yang optimal dan relevan dengan standar *microstock*. Hasil pengujian *endpoint* ini ditunjukkan pada Gambar 4.17.



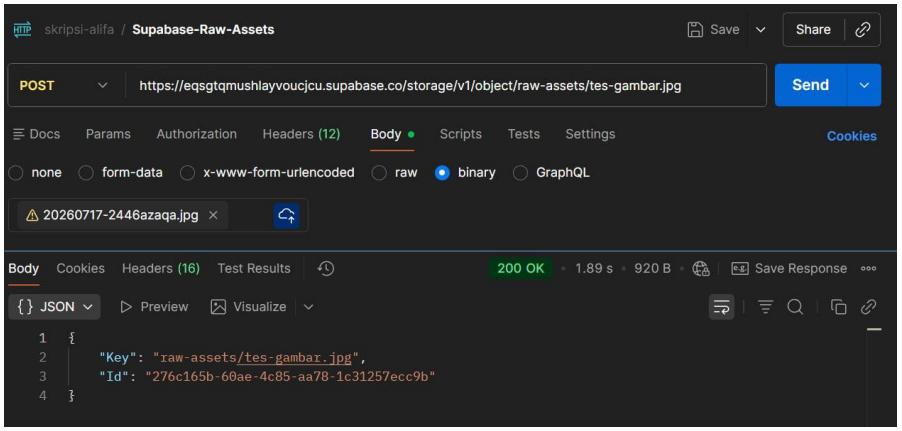
Gambar 4.17 Hasil Pengujian API Gemini 2.5 Flash (*Prompt Enhancer*)

Berdasarkan hasil eksekusi pada Gambar 4.17, pengujian *endpoint* API Gemini 2.5 Flash menggunakan Postman berhasil mendapatkan respons status 200 OK dengan waktu pemrosesan selama 1,77 detik. Luaran respons JSON mengembalikan deskripsi *prompt* visual yang berhasil ditingkatkan kualitas *detail*nya pada atribut "*content*", yang kemudian akan digunakan sebagai *prompt* masukan pada pembuatan gambar menggunakan model Flux.

5. Implementasi API Supabase *Storage*

   Pengujian API Supabase *storage* dilakukan melalui Supabase REST API *gateway* menggunakan metode *POST* untuk fungsi penyimpanan media (*cloud storage*). Pengujian ini dilakukan pada dua *bucket* utama, yaitu *raw*-

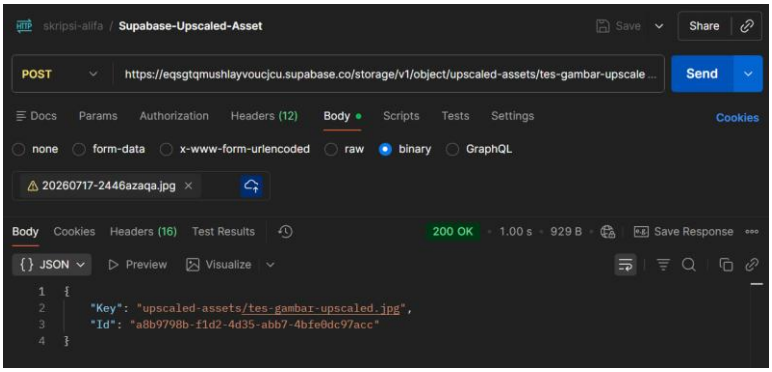
assets untuk menampung gambar hasil generasi awal model FLUX 1.1 Pro dan *upscaled-assets* untuk menyimpan gambar yang telah ditingkatkan resolusinya oleh model Real-ESRGAN dan berformat JPG. Parameter yang dikirimkan mencakup nama berkas tujuan, biner citra (*binary file*), serta kunci otentikasi API (*apikey* dan *bearer token*), sebagaimana ditunjukkan pada Gambar 4.18.



Gambar 4.18 Hasil Pengujian API Supabase *Bucket Raw-assets*

Berdasarkan hasil eksekusi pada Gambar 4.18, pengujian *endpoint* Supabase *storage* pada *bucket raw-assets*, pengujian menggunakan Postman berhasil mendapatkan respons status 200 OK dengan waktu pemrosesan selama 1,89 detik. Luaran respons JSON mengembalikan atribut "*Key*" dengan nilai "*raw-assets*/tes-gambar.jpg", yang menunjukkan berkas citra telah berhasil tersimpan pada wadah penampung awal.

Sementara itu, bukti eksekusi pengujian *endpoint* Supabase *storage* untuk *bucket upscaled-assets* diperlihatkan pada Gambar 4.19.



Gambar 4.19 Hasil Pengujian API Supabase *Bucket Upscaled-Assets*

Berdasarkan hasil eksekusi pada Gambar 4.19, pengujian *endpoint* Supabase *storage* pada *bucket upscaled-assets*, pengujian menggunakan Postman berhasil mendapatkan respons status 200 OK dengan waktu pemrosesan selama 1,00 detik. Luaran respons JSON mengembalikan atribut "*Key*" dengan nilai "*upscaled-assets/tes-gambar-upscaled.jpg*", yang menandakan bahwa berkas citra bernilai resolusi tinggi siap digunakan untuk tahap ekstraksi *metadata* oleh Gemini 2.5 Flash Lite.

4.3.2 Rekapitulasi Hasil Pengujian API

Berdasarkan seluruh pengujian *endpoint* REST API yang telah dilakukan melalui Postman, seluruh layanan eksternal terbukti dapat berkomunikasi secara *valid* dan mengembalikan kode status respons HTTP yang sesuai. Rekapitulasi status pengujian untuk masing-masing penyedia layanan dirangkum pada Tabel 4.1.

Tabel 4.1 Rekapitulasi Hasil Pengujian API dengan Postman

Model	Proses	*Method*	Status Code	Status
FLUX 1.1 Pro	Generasi Gambar (*Text-to-Image*).	*POST*	201 *Created*	*Pass*
Real-ESRGAN	Peningkatan Resolusi (*Upscaling*).	*POST*	201 *Created*	*Pass*
Gemini 2.5	Riset tren, ekstraksi	*POST*	200 *OK*	*Pass*

Model	Proses	*Method*	Status Code	Status
Flash Lite	*metadata* (*title* & *keywords*), serta pemformatan struktur luaran pesan.			
Gemini 2.5 Flash	Penajam Perintah (*Prompt Enhancement).*	*POST*	200 *OK*	*Pass*
Supabase *Storage*	Pengunggahan Berkas Citra (*Bucket raw-assets* dan *upscaled-assets*).	*POST*	200 *OK*	*Pass*

Berdasarkan hasil yang tercantum pada Tabel 4.1, seluruh layanan eksternal yang meliputi model AI (FLUX 1.1 Pro, Real-ESRGAN, Gemini 2.5 Flash Lite, dan Gemini 2.5 Flash), serta media *storage* menunjukkan status eksekusi berhasil dengan metode komunikasi *POST,* serta mengembalikan kode status *respons* HTTP yang benar (201 *Created* dan 200 OK).

4.4 Hasil Uji Coba Sistem

Tahap uji coba dilakukan untuk mengevaluasi fungsionalitas, kinerja, serta keandalan sistem otomatisasi produksi aset *microstock* yang telah dibangun. Pengujian ini bertujuan untuk memastikan bahwa seluruh *node* dan integrasi API pada n8n dapat berjalan dengan stabil, serta mampu menghasilkan luaran aset visual dan *metadata* yang memenuhi standar kualitas *platform* Adobe Stock. Proses pengujian dilaksanakan melalui tiga pendekatan utama, yaitu penilaian kualitas luaran (*output*), analisis kinerja sistem, serta analisis efisiensi.

4.4.1 Hasil Penilaian Kualitas Luaran

Penilaian kualitas luaran dilakukan untuk menguji sejauh mana sistem mampu mengeksekusi instruksi pengguna (*prompt*) menjadi aset visual dan *metadata* yang relevan, yang mencakup dua fokus evaluasi utama yaitu hasil generasi gambar berdasarkan tingkat kompleksitas instruksi beserta durasi pemrosesannya, serta keakuratan pengekstrakan *metadata* pendukung berupa judul (*title*) dan kata kunci (*keywords*) komersial untuk kebutuhan *platform* Adobe Stock.

1. Hasil Evaluasi Generasi Gambar

   Pengujian generasi gambar dilakukan untuk menguji kemampuan sistem dalam mengeksekusi *prompt* masukan menjadi aset visual berdasarkan tingkat kompleksitas instruksi (rendah, sedang, dan tinggi), serta mengukur durasi pemrosesannya. Tampilan rekapitulasi pengujian kualitas dan performa luaran gambar ditunjukkan pada Tabel 4. 2.

Tabel 4.2 Hasil Pengujian Kualitas Visual Gambar

No	*Prompt*	Tingkat	Durasi (Detik)	Bukti Fisik *Output*	Dimensi	Status
1	Buat Apel merah dengan *background* putih.	Rendah	45,60		2048 × 2048 (2K)	Sesuai
2	Buat cangkir kopi dengan *background* kuning.	Rendah	49,60		2048 × 2048 (2K)	Sesuai

No	*Prompt*	Tingkat	Durasi (Detik)	Bukti Fisik *Output*	Dimensi	Status
3	Buat gambar irisan pisang dengan *background* putih.	Rendah	46,81		2048 × 2048 (2K)	Sesuai
4	Buat cangkir kopi dengan *background* hijau.	Rendah	47,48		2048 × 2048 (2K)	Sesuai
5	Buat dompet kulit dengan *background* putih.	Rendah	49,40		2048 × 2048 (2K)	Sesuai
6	Buat kamera jadul di atas meja kayu.	Sedang	52,12		2048 × 2048 (2K)	Sesuai
7	Buat lampu tidur di atas meja kayu.	Sedang	51,24		2048 × 2048 (2K)	Sesuai

No	*Prompt*	Tingkat	Durasi (Detik)	Bukti Fisik *Output*	Dimensi	Status
8	Buat Mangkuk kaca berisi buah stroberi.	Sedang	50,43		2048 × 2048 (2K)	Sesuai
9	Buat *notebook* di atas meja marmer pot di sudut jendela kamar tidur.	Sedang	45,27		2048 × 2048 (2K)	Sesuai
10	Buat seiris kue cokelat di atas piring.	Sedang	54,37		2048 × 2048 (2K)	Sesuai
11	Buat gambar ruangan apartemen terdiri dari meja, sofa, hiasan meja, jendela.	Tinggi	86,44		2048 × 2048 (2K)	Sesuai
12	Buat sofa, bantal, dan meja di ruang tamu	Tinggi	52,41		2048 × 2048 (2K)	Sesuai

No	*Prompt*	Tingkat	Durasi (Detik)	Bukti Fisik *Output*	Dimensi	Status
13	Buat gambar jus jeruk, buku catatan dan lampu di meja kerja.	Tinggi	72,52		2048 × 2048 (2K)	Sesuai
14	Buat gambar ruangan rumah sakit, terdapat kasur, meja, kursi, tirai, selimut dengan lantai putih dan dinding putih	Tinggi	61,11		2048 × 2048 (2K)	Sesuai
15	Buat ruang sekolah berisi meja, kursi, papan tulis dan jendela.	Tinggi	76,51		2048 × 2048 (2K)	Sesuai

Berdasarkan Tabel 4.2, hasil pengujian kualitas dan performa luaran (*output*) menunjukkan bahwa seluruh sampel dari berbagai tingkat kompleksitas *prompt* memiliki status relevansi yang sesuai terhadap instruksi pengguna. Dari segi kualitas fisik citra, hasil pembacaan properti berkas membuktikan bahwa seluruh *output* gambar berhasil ditingkatkan

dimensinya secara otomatis oleh *node* Real-ESRGAN menjadi 2048 × 2048 piksel (resolusi 2K) tanpa mengalami kerusakan visual (*artifacting*). Selain itu, hasil pengujian menunjukkan bahwa tingkat kompleksitas instruksi berbanding lurus dengan durasi pemrosesan sistem secara *end-to-end*.

Pada kelompok *prompt* dengan tingkat kompleksitas rendah (lima percobaan), sistem memerlukan waktu pemrosesan antara 45,60 detik hingga 49,60 detik dengan rata-rata durasi sekitar 47,76 detik. Waktu ini tergolong cepat, karena elemen visual yang diolah cukup sederhana, seperti objek tunggal dengan latar belakang polos.

Pada kelompok *prompt* dengan tingkat kompleksitas Sedang (lima percobaan), durasi pemrosesan berada pada rentang 45,27 detik hingga 54,37 detik dengan rata-rata sekitar 51,89 detik. Peningkatan durasi waktu ini dipengaruhi oleh penambahan *detail* objek, interaksi antar elemen, serta pencahayaan yang lebih terperinci pada instruksi masukan.

Sementara itu, pada kelompok *prompt* dengan tingkat kompleksitas tinggi (lima percobaan), terjadi peningkatan durasi pemrosesan yang signifikan, yaitu berkisar antara 52,41 detik hingga 86,44 detik dengan rata-rata mencapai 69,80 detik. Peningkatan durasi waktu pemrosesan pada kategori ini disebabkan oleh kerumitan komposisi visual, banyaknya objek dalam satu adegan, serta deskripsi detail latar belakang yang membutuhkan waktu pengayaan *prompt* oleh Gemini AI, generasi gambar oleh FLUX 1.1 Pro, hingga pemrosesan peningkatan resolusi (*upscaling*) 2K yang lebih intensif pada *server*.

Meskipun secara keseluruhan relevansi gambar dan peningkatan resolusi telah memenuhi standar 2K (2048 × 2048 piksel), hasil pengujian kualitatif menunjukkan adanya variasi pada hasil komposisi visual. Tanpa adanya instruksi tata letak yang spesifik pada *prompt*, penempatan objek oleh model AI bersifat acak (*stochastic*). Sebagian besar sampel berhasil menghasilkan objek yang presisi di tengah bingkai, namun pada sampel nomor 5 (*prompt*: 'Buat dompet kulit dengan *background* putih'), terjadi kekurangan berupa bagian tepi objek yang sedikit terpotong. Kekurangan ini murni disebabkan

oleh variasi generasi dari model AI itu sendiri ketika tidak diberikan batasan tata letak (*composition constraint*) yang *detail.*

2. Hasil Evaluasi *Metadata* Pendukung

Pengujian *metadata* pendukung dilakukan untuk mengukur tingkat relevansi judul (*title*) dan kata kunci (*keywords*) yang diekstrak secara otomatis oleh agen Google Gemini berdasarkan analisis visual gambar. *Metadata* yang dihasilkan disesuaikan dengan standar komersial *platform* Adobe Stock menggunakan bahasa Inggris deskriptif guna mengoptimalkan keterpenemuan aset. Tampilan rekapitulasi pengujian relevansi *metadata* ditunjukkan pada Tabel 4.3.

Tabel 4.3 Hasil Evaluasi Relevansi Judul dan Kata Kunci Gambar

No	***Prompt***	Judul	Kata Kunci	Status Relevansi
1	Buat apel merah dengan *background* putih.	*Red apple on a white background for healthy eating and fruit concepts.*	*apple, red apple, fruit, healthy eating, food, fresh, single, isolated, white background, studio shot* +37 lainnya.	Sesuai
2	Buat cangkir kopi dengan *background* kuning.	*White coffee cup with steam on a saucer on a yellow background.*	*coffee, cup, mug, hot, drink, beverage, steam, smoke, aroma, caffeine* +40 lainnya	Sesuai
3	Buat gambar irisan pisang dengan *background* putih.	*Slices of ripe banana on a white background for healthy*	*banana, bananas, slice, slices, sliced, fruit, tropical, yellow, food, healthy* +35 lainnya.	Sesuai

No	*Prompt*	Judul	Kata Kunci	Status Relevansi
		eating and food concepts.		
4	Buat cangkir kopi dengan *background* hijau.	*Top view of a white cup of coffee with dark specks on a bright green surface with a shadow, for beverage and morning concepts.*	*coffee, cup, white cup, coffee cup, hot beverage, caffeine, morning, breakfast, drink, beverage* +43 lainnya.	Sesuai
5	Buat dompet kulit dengan *background* putih.	*Close-up of a brown leather wallet on a white background for personal finance and accessory concepts.*	*leather wallet, brown wallet, mens wallet, womens wallet, leather goods, accessory, fashion accessory, wallet, close up, detail* +40 lainnya.	Sesuai
6	Buat kamera jadul di atas meja kayu.	*Vintage camera on a wooden surface with natural light and shadows for*	*vintage camera, retro camera, film camera, old camera, antique camera, analog camera, camera, photography, photograhy equipment, camera accessory* +39	Sesuai

No	*Prompt*	Judul	Kata Kunci	Status Relevansi
		photography and retro concepts.	lainnya.	
7	Buat Lampu tidur di atas meja kayu.	*Modern desk lamp on a nightstand in a bedroom for home decor and lighting concepts.*	*desk lamp, table lamp, bedside lamp, nightstand lamp, modern lamp, contemporary lamp, home decor, interior design, lighting, bedroom* +37 lainnya.	Sesuai
8	Buat mangkuk kaca berisi buah stroberi.	*Fresh strawberries in a clear bowl on a wooden surface for healthy eating and summer fruit concepts.*	*strawberry, strawberries, fresh, fruit, food, red, healthy, organic, ripe, sweet* +35 lainnya.	Sesuai
9	Buat *notebook* di atas meja marmer.	*Flat lay of a notebook and lit candle on a marble surface for journaling and cozy home concepts.*	*notebook, journal, candle, lit candle, flame, marble, marble surface, desk, table, flat lay* +37 lainnya.	
10	Buat seiris kue cokelat di atas	*Decadent chocolate cake*	*chocolate cake, cake slice, dessert, sweet,*	Sesuai

No	*Prompt*	Judul	Kata Kunci	Status Relevansi
	piring.	*slice on a white plate with a gold fork for dessert and baking concepts.*	*food, baked goods, confectionery, indulgence, gourmet, rich* +36 lainnya.	
11	Buat gambar ruangan apartemen terdiri dari meja, sofa, hiasan meja, jendela.	*Modern grey sofa with a bouquet of flowers on a coffee table in a bright living room interior for home decor and lifestyle concepts..*	*sofa, couch, living room, modern, interior design, home decor, furniture, lifestyle, comfortable, cozy* +39 lainnya.	Sesuai
12	Buat sofa, bantal, dan meja di ruang tamu.	*Minimalist living room with a neutral-toned sofa, decorative pillows, and a blanket on a wooden coffee table in front of a blank frame on the wall for*	*living room, interior design, home decor, sofa, couch, neutral tones, beige, grey, pillows, cushions* +37 lainnya.	Sesuai

No	*Prompt*	Judul	Kata Kunci	Status Relevansi
		interior design and home decor concepts		
13	Buat gambar jus jeruk, buku catatan dan lampu di meja kerja.	*Glass of orange juice on a notebook on a wooden desk with plants in the background for a healthy lifestyle and work-from-home concept*	*orange juice, glass, drink, beverage, refreshment, citrus, fruit, healthy, vitamin C, wellness* +38 lainnya.	Sesuai
14	Buat gambar ruangan rumah sakit, terdapat kasur, meja, kursi, tirai, selimut dengan lantai putih dan dinding putih.	*Minimalist hospital room with medical bed and curtains for healthcare and interior design concepts*	*hospital, room, medical, bed, white, clean, modern, interior, design, healthcare* + 36 lainnya.	Sesuai
15	Buat ruang sekolah berisi meja, kursi, papan tulis dan jendela.	*Empty classroom with desks and chairs in front of a blackboard*	*classroom, empty, school, education, desks, chairs, blackboard, learning, teaching, students* +37 lainnya.	Sesuai

No	*Prompt*	Judul	Kata Kunci	Status Relevansi
		with drawings for education and school concepts		

Berdasarkan Tabel 4.3, hasil evaluasi dari 15 kali pengujian menunjukkan bahwa agen AI Gemini 2.5 Flash Lite mampu menghasilkan *metadata* yang sangat akurat dan relevan terhadap subjek visual gambar. Pada skenario pengujian pertama dengan perintah *"*Buat apel merah dengan *background* putih*"*, sistem berhasil meregenerasi judul komersial *"Red apple on a white background",* serta mengekstrak rangkaian kata kunci relevan, seperti *apple*, *red apple*, *fruit*, *healthy eating*, *food*, *fresh*, hingga *single*. Secara keseluruhan, sampel pengujian menghasilkan keluaran berkisar antara 45 hingga 53 *keywords* per aset. Meskipun *platform* Adobe Stock memiliki batasan maksimal 49 *keywords*, perolehan hasil ini tetap menjadi nilai tambah yang baik, karena memberikan pengguna keleluasaan untuk memilih kata kunci yang paling presisi dan optimal untuk kebutuhan SEO sebelum diunggah.

4.4.2 Hasil Analisis Kinerja Sistem

Bagian ini memaparkan data hasil pengujian sistem otomatisasi produksi aset visual yang telah dilakukan melalui skenario eksperimen terkontrol sebanyak 15 kali pengujian. Seluruh tahapan proses, mulai dari aktivasi pemicu hingga pengiriman hasil akhir, dipantau melalui rekam jejak eksekusi sistem untuk menjamin objektivitas data. Analisis kinerja ini diuraikan secara komprehensif dengan mengorelasikan tiga dimensi utama, yaitu akumulasi waktu tunda (latensi), kapasitas produksi sistem (*throughput*), serta identifikasi titik hambat kerja (*bottleneck*).

Berdasarkan pengujian eksperimental tersebut, kinerja sistem diukur secara

mendalam pada setiap fase alur kerja otomatisasi. Untuk mempermudah proses analisis, rangkaian *node* di dalam sistem dikelompokkan ke dalam 6 fase utama berdasarkan fungsi spesifiknya, sebagaimana struktur parameter evaluasi yang telah dirancang pada Tabel 3.9. Rata-rata durasi waktu eksekusi, serta persentase kontribusi waktu dari masing-masing fase disajikan pada Tabel 4.4.

Tabel 4.4 Hasil Pengujian Latensi Sistem

No	Fase Alur Kerja Sistem	Rata-rata Latensi (Detik)	Persentase
1	Inisialisasi & Penerimaan Pesan.	2,00	3,70%
2	Peningkatan *Prompt* (*Prompt Enhancer*).	1,39	2,60%
3	Generasi Gambar (*Image Generation*).	5,90	11,00%
4	Pengolahan & Peningkatan Kualitas Gambar (*Upscaling*).	36,47	67,90%
5	Analisis *Metadata* AI (*AI Agent*).	6,90	12,80%
6	Pengiriman Hasil ke Pengguna (*Delivery*).	1,03	1,90%
Total	Rata-rata Latensi Total Sistem ($\bar{L}$).	53,68	100%

Berdasarkan data yang disajikan pada Tabel 4.4, diperoleh nilai rata-rata latensi total sistem sebesar 53,68 detik. Nilai ini merupakan hasil perhitungan rata-rata latensi total seluruh sampel pengujian (n) yang mengacu pada Persamaan (3.1), sekaligus merupakan akumulasi durasi dari enam fase alur kerja di dalam sistem. Melalui representasi data per fase tersebut, kontribusi durasi waktu dari masing-masing proses dapat dianalisis secara rinci.

Hasil pengujian menunjukkan bahwa Fase 4 (Pengolahan & Peningkatan Kualitas Gambar / *Upscaling*) menjadi kontributor waktu terbesar dalam alur kerja sistem. Berdasarkan rumus perhitungan persentase kontribusi pada Persamaan (3.2), fase ini membutuhkan waktu rata-rata selama 36,47 detik dengan persentase dominan mencapai 67,90 % dari total keseluruhan latensi. Tingginya durasi pada fase ini disebabkan oleh beban komputasi yang berat saat melakukan rekonstruksi piksel untuk meningkatkan resolusi gambar secara *detail*. Dominasi waktu yang signifikan pada Fase 4 ini secara otomatis menetapkan proses *upscaling* sebagai titik hambat utama (*bottleneck*) di dalam alur kerja sistem.

Sebaliknya, Fase 6 (Pengiriman Hasil ke Pengguna / *Delivery*) merupakan proses paling efisien dengan rata-rata waktu eksekusi tersingkat, yaitu hanya 1,03 detik (1,90 %). Sementara itu, fase-fase pendukung lainnya, seperti inisialisasi & penerimaan Pesan (3,70 %), *prompt enhancer* (2,60 %), generasi gambar (11,00 %), dan analisis *metadata* AI (12,80 %) berjalan relatif stabil dengan total akumulasi waktu di bawah 15 detik.

Berdasarkan nilai rata-rata latensi total sebesar 53,68 detik tersebut, kapasitas performa sistem dalam memproses data dalam satu satuan waktu tertentu (*throughput*) dapat diketahui. Dengan mengonversi waktu standar satu jam (3600 detik) dan dihitung berdasarkan Persamaan (3.3), diperoleh nilai *throughput* sistem sebesar 67,06 proses per jam. Artinya, dalam kondisi operasional yang berjalan secara kontinu dan normal, sistem ini secara rata-rata mampu memproduksi serta mengirimkan sekitar 67 aset visual siap pakai kepada pengguna di setiap jamnya.

Secara keseluruhan, integrasi analisis latensi, titik hambat (*bottleneck*), dan kapasitas produksi (*throughput*) memberikan gambaran objektif mengenai kualitas kinerja sistem otomatisasi yang telah dibangun. Keberlanjutan stabilitas performa sistem ini sangat bergantung pada responsivitas API pihak ketiga, di mana potensi gangguan jaringan atau batasan akses telah diantisipasi melalui mekanisme penanganan kesalahan (*error handling*). Guna mengoptimalkan nilai *throughput* secara lebih maksimal di masa mendatang, pengembangan arsitektur dapat difokuskan pada peningkatan komputasi, khususnya pada Fase 4 yang menjadi pusat hambatan kerja.

4.4.3 Hasil Analisis Efisiensi Sistem

Pengujian efisiensi dilakukan untuk mengukur sejauh mana penerapan *agentic workflow* berbasis otomatisasi mampu mengoptimalkan alur kerja produksi aset *microstock* dibandingkan dengan metode konvensional (manual). Evaluasi ini ditinjau dari dua dimensi utama, yaitu efisiensi waktu produksi dan efisiensi beban kerja operasional (*effort*).

1. Analisis Efisiensi Waktu Produksi

   Efisiensi waktu diukur dengan membandingkan total durasi pengerjaan secara manual pada beberapa *platform* terpisah terhadap total durasi eksekusi oleh sistem otomatisasi. Berdasarkan proses pengujian dan pengukuran langsung yang dilakukan oleh peneliti, produksi satu aset visual menggunakan metode manual membutuhkan tiga tahapan utama:

   a. Generasi gambar pada *web-browser* Kling AI dengan durasi rata-rata selama 29,06 detik.
   b. Peningkatan kualitas dan resolusi gambar (*upscaling*) menggunakan perangkat lunak lokal VideoProc dengan durasi rata-rata selama 8 menit (480 detik).
   c. Analisis dan pemrosesan *metadata* pada *web-browser* Google Gemini AI dengan durasi rata-rata selama 22 detik.

   Melalui pengujian eksperimental tersebut dan dihitung berdasarkan Persamaan (3.4), total durasi pengerjaan secara manual tercatat sebesar 531,06 detik. Sementara itu, total durasi pengerjaan rata-rata menggunakan sistem otomatisasi hanya membutuhkan waktu sebesar 53,68 detik.

   Berdasarkan perhitungan menggunakan Persamaan (3.5), diperoleh tingkat efisiensi waktu sebesar 89,89%. Hasil pengujian ini membuktikan bahwa penerapan sistem otomatisasi berbasis *agentic workflow* berhasil memangkas waktu produksi aset secara signifikan atau hampir 10 kali lebih cepat dibandingkan metode manual.

2. Efisiensi Beban Kerja Operasional (*Effort*)

   Efisiensi beban kerja operasional mengukur penghematan interaksi fisik pengguna seperti aktivitas klik (*click*) dan ketik (*type*). Rekapitualisasi

perbandingan interaksi fisik pada metode manual disajikan pada Tabel 4.5.

Tabel 4.5 Rincian Aktivitas dan Beban Kerja Operasional Manual

No	Fase Produksi	Jenis Interaksi Fisik	Deskripsi Aktivitas Operasional	Estimasi Beban
1	Generasi Gambar (Kling AI)	Klik & Ketik	Memilih fitur generasi gambar, membuka *setting image*, mengklik rasio, mengklik jumlah yang ingin dihasilkan, menutup *setting image*, mengklik tombol *input*, mengetik perintah, dan mengklik tombol kirim.	± 7x Klik 1x Ketik
2	Peningkatan Kualitas (VideoProc)	Klik &	Mengklik fitur *upscale*, mengklik tombol memasukkan gambar, mengklik berkas gambar, mengklik *open*, mengklik tombol *export setting*, mengklik *dropdown* format keluaran, mengklik format (JPG), dan mengklik *run*.	± 8x Klik
3	Riset *Metadata* (Google Gemini)	Klik & Ketik	Mengklik ikon tambah, mengklik pilihan *upload file*, mengklik gambar, mengklik *open*, mengetik *prompt*, dan mengklik	± 5x Klik 1x Ketik

No	Fase Produksi	Jenis Interaksi Fisik	Deskripsi Aktivitas Operasional	Estimasi Beban
			kirim.	
Total	3 Fase / 3 *Platform*	*Multi-Trigger*	Interaksi fisik berulang secara manual.	± 22 Aktivitas Operasional (20x Klik, 2x Ketik)

Berdasarkan rincian aktivitas pada Tabel 4.5, pelaksanaan alur kerja operasional secara manual memerlukan proses yang relatif panjang dan berulang. Pada fase pertama generasi gambar menggunakan Kling AI, pengguna harus melakukan 7 kali aksi klik dan 1 kali aksi peng*et*ikan untuk mengonfigurasi parameter gambar hingga mengirimkan instruksi. Selanjutnya, pada fase kedua peningkatan kualitas menggunakan VideoProc, dibutuhkan 8 kali aksi klik untuk memproses pengunggahan, pengaturan format, hingga ekspor berkas. Pada fase ketiga riset *metadata* menggunakan Gemini AI, pengguna kembali melakukan 5 kali aksi klik dan 1 kali peng*et*ikan untuk mengunggah aset dan mengeksekusi perintah pembuatan *metadata*.

Secara keseluruhan, metode manual membutuhkan total 22 aktivitas operasional yang terdiri dari 20 kali interaksi klik dan 2 kali peng*et*ikan. Kerumitan ini timbul akibat sifat alur kerja manual yang *multi-trigger* dan mengharuskan pengguna melakukan navigasi berulang di antara tiga *platform* terpisah.

Untuk mengevaluasi penyederhanaan beban kerja fisik (*effort*) setelah diterapkannya sistem otomatisasi berbasis Telegram *bot*, dilakukan perbandingan interaksi operasional pada lingkungan perangkat yang sama. Penilaian perbandingan beban kerja operasional disajikan pada Tabel 4.6.

Tabel 4.6 Perbandingan Beban Kerja Operasional (*Desktop Environment*)

Metode Pengujian	**Rincian Interaksi Fisik Operasional**	**Total Beban Kerja (*Effort*)**
Metode Manual	20x Klik + 2x Ketik	22 Aktivitas
Sistem Otomatisasi *(Telegram Bot)*	1x Ketik (*prompt*) + 1x Klik (tombol kirim)	2 Aktivitas

Berdasarkan perbandingan pada Tabel 4.6, pengujian pada sistem otomatisasi berbasis Telegram *bot* menunjukkan penyederhanaan alur kerja yang sangat signifikan. Pada lingkungan Telegram *Desktop*, kolom *input* pesan secara bawaan sudah berada dalam kondisi fokus/aktif saat aplikasi terbuka. Kondisi ini membuat pengguna hanya memerlukan 2 langkah interaksi fisik, yaitu 1 kali peng*et*ikan *prompt* dan 1 kali aksi klik pada tombol kirim.

Melalui pengujian eksperimental tersebut dan dihitung berdasarkan Persamaan (3.7), penerapan sistem otomatisasi berbasis *agentic workflow* ini terbukti berhasil memberikan tingkat efisiensi beban kerja operasional (*effort*) sebesar 90,91%. Penghematan ini dicapai, karena sistem berhasil memangkas kerumitan navigasi *multi-platform*, pemrosesan manual *upscaling*, serta pengunggahan aset berulang menjadi satu pintu interaksi berbasis pesan.

Ringkasan perbandingan hasil evaluasi efisiensi antara metode manual dan sistem otomatisasi disajikan pada Tabel 4. 7.

Tabel 4.7 Perbandingan Efisiensi Metode Manual dan Otomatisasi

No	**Dimensi Efisiensi**	**Metode Manual**	**Sistem Otomatisasi**	**Dampak Efisiensi**
1	Waktu Produksi	531,06 detik (± 8,8 menit)	53,68 detik (± 1 menit)	Efisiensi waktu sebesar 89,89%

No	Dimensi Efisiensi	Metode Manual	Sistem Otomatisasi	Dampak Efisiensi
2	Interaksi Pengguna (*effort*)	22 aktivitas operasional (*multi-trigger*)	2 langkah interaksi fisik (*single-interface*)	Penurunan *effort* sebesar 90,91%

Berdasarkan rekapitulasi data pada Tabel 4.7, dapat disimpulkan bahwa penerapan *agentic workflow* berbasis otomatisasi memberikan dampak signifikan terhadap peningkatan efisiensi kinerja produksi aset visual. Sistem tidak hanya mampu memangkas durasi waktu produksi hingga 89,89%, tetapi juga berhasil menurunkan beban kerja operasional pengguna (*effort*) sebesar 90,91% melalui penyederhanaan alur kerja dari *multi-trigger* pada beberapa *platform* terpisah menjadi satu pintu interaksi (*single-interface*).

4.5 Evaluasi

Evaluasi sistem dilakukan untuk memverifikasi fungsionalitas, kualitas luaran (*output*), kinerja, dan efisiensi dari alur *agentic workflow* yang telah dibangun. Pengujian fungsionalitas dilaksanakan menggunakan metode *black box testing* guna memastikan seluruh komponen, integrasi antar-*node*, dan fitur aplikasi berfungsi sesuai skenario. Pengujian dinyatakan berhasil dan diberi status *pass* apabila luaran sistem telah sesuai dengan hasil yang diharapkan. Hasil pengujian fungsionalitas sistem secara rinci disajikan pada Tabel 4.8.

Tabel 4.8 Hasil *Black Box Testing*

No	Fitur yang Diuji	Skenario Uji	Ekspektasi *Output*	Hasil Aktual	Status
1	Notifikasi Konfirmasi Perintah	Pengguna mengirimkan perintah/instruksi	Sistem mengonfirmasi pesan telah	Sistem berhasil mengonfirmasi pesan,	*Pass*

No	Fitur yang Diuji	Skenario Uji	Ekspektasi *Output*	Hasil Aktual	Status
		melalui *bot* Telegram.	diterima, mendeteksi jenis permintaan yang masuk (seperti pembuatan gambar atau riset), dan mengirimkan notifikasi bahwa proses sedang berjalan.	mendeteksi jenis permintaan (riset/generasi gambar), dan mengirimkan notifikasi bahwa proses sedang berjalan.	
2	Riset Tren Pasar	Pengguna mengirimkan perintah riset tren pasar via *bot* Telegram.	AI Gemini 2.5 Flash Lite menganalisis tren pasar dan menghasilkan laporan riset tren pasar.	AI Gemini 2.5 Flash Lite berhasil menganalisis dan menghasilkan laporan riset tren pasar.	*Pass*
3	Produksi Aset Visual, *Metadata*, Penyimpanan, dan Pengiriman	Pengguna mengirimkan perintah pembuatan gambar (*prompt*).	Sistem mengoptimalkan *prompt* (Gemini 2.5 Flash), memproduksi gambar (FLUX 1.1 Pro), *upscale* ke 2K (Real-ESRGAN), mengekstraksi	Sistem berhasil mengoptimalkan *prompt*, memproduksi gambar, *upscale* ke 2K, mengekstraksi *metadata* (*title* dan > 40 *keywords*),	*Pass*

No	Fitur yang Diuji	Skenario Uji	Ekspektasi *Output*	Hasil Aktual	Status
			metadata title dan minimal 40 *keywords* (Gemini 2.5 Flash Lite), menyimpan *file* ke Supabase, serta mengirimkan hasil ke pengguna via Telegram.	menyimpan *file* di Supabase, dan mengirimkan hasil ke Telegram.	
4	Manajemen Menu panduan	Pengguna mengirimkan perintah /*start*.	Sistem memberikan respons konfirmasi bahwa pesan telah diterima dan menampilkan pesan menu panduan.	Sistem berhasil memberikan respons konfirmasi bahwa pesan telah diterima dan menampilkan pesan menu panduan.	*Pass*
5	Penanganan Kesalahan Format Perintah	Pengguna mengirimkan perintah salah format.	Sistem memberikan respons *feedback* berupa petunjuk instruksi yang benar.	Sistem berhasil memberikan respons *feedback* berupa petunjuk instruksi yang benar.	*Pass*
6	Penanganan	Pengguna	Sistem menolak	Sistem berhasil	*Pass*

No	Fitur yang Diuji	Skenario Uji	Ekspektasi *Output*	Hasil Aktual	Status
	Kesalahan Kebijakan *Prompt*	mengirimkan *prompt* melanggar kebijakan	membuat gambar dan memberikan respons *feedback* berupa notifikasi peringatan kebijakan.	menolak membuat gambar dan memberikan respons *feedback* berupa notifikasi peringatan kebijakan.	
7	Penanganan Kesalahan Akses API	Terjadi gangguan pada akses API eksternal	Sistem memberikan respons *feedback* berupa notifikasi terjadinya kesalahan teknis.	Sistem berhasil memberikan respons *feedback* berupa notifikasi terjadinya kesalahan teknis.	*Pass*
8	Manajemen Kinerja dan Efisiensi	Pengguna memberikan satu tindakan pemicu (*single-trigger*)	Sistem mengeksekusi seluruh alur otomatisasi di latar belakang dan menghasilkan gambar serta *metadata* berkualitas secara relevan.	Sistem berhasil mengeksekusi seluruh alur otomatisasi di latar belakang dan menghasilkan gambar serta *metadata* berkualitas secara relevan.	*Pass*

Berdasarkan hasil pengujian *black box testing* yang disajikan pada Tabel 4.8, seluruh skenario pengujian dari tujuh cakupan fitur utama dinyatakan lulus (100% *pass*). Sistem terbukti mampu memproses setiap instruksi masukan (*input*) melalui Telegram *bot* secara konsisten, mulai dari riset tren pasar, produksi gambar beserta

metadata dan pengirimannya, manajemen menu panduan, penanganan berbagai kondisi kesalahan seperti format perintah, kebijakan *prompt*, dan akses API, hingga manajemen kinerja serta efisiensi, tanpa mengalami kegagalan fungsi.

Selain pengujian fungsionalitas melalui *black box testing*, evaluasi sistem secara menyeluruh dilakukan dengan mengintegrasikan hasil analisis kinerja kuantitatif dan evaluasi kualitatif terhadap luaran (*output*) yang dihasilkan. Berdasarkan seluruh rangkaian eksperimen yang telah dilakukan, ringkasan capaian kinerja dan catatan evaluasi sistem dapat dirinci sebagai berikut:

1. Kualitas Fisik dan Keterbatasan Komposisi Visual

   Sistem berhasil menghasilkan luaran visual berresolusi 2K tanpa cacat (*artifacting*). Dari gambar yang terbentuk, sistem secara otomatis mengekstraksi judul komersial (*title*) serta rangkaian kata kunci (*keywords*) yang akurat dan relevan, dengan perolehan berkisar antara 45 hingga 53 *keywords* per aset.

   Hasil pengujian juga menunjukkan bahwa kompleksitas *prompt* berbanding lurus dengan durasi pemrosesan *end-to-end*. Pada *prompt* kompleksitas rendah (objek tunggal, *background* polos), pemrosesan membutuhkan waktu rata-rata 47,76 detik (45,60–49,60 detik). Pada kompleksitas sedang, durasi meningkat menjadi rata-rata 51,89 detik (45,27–54,37 detik) seiring bertambahnya detail objek. Sementara pada kompleksitas tinggi, durasi mencapai rata-rata 69,80 detik (52,41–86,44 detik) akibat kerumitan visual, serta pemrosesan *upscaling* yang lebih intensif.

   Di samping itu, pengujian kualitatif mencatat sifat stokastik model AI dalam menentukan tata letak objek tanpa *composition constraint* yang spesifik. Terlihat pada Sampel Nomor 5 (*prompt*: ″Buat dompet kulit dengan background putih″), objek dompet terlalu bergeser ke tepi bingkai (*frame*) hingga sedikit terpotong. Kekurangan ini murni dipengaruhi oleh variasi generasi bawaan model AI akibat ketiadaan instruksi tata letak yang *detail*.

2. Kinerja Waktu dan Latensi Sistem

   Sistem berbasis *agentic workflow* mencatat nilai rata-rata latensi sebesar 53,68 detik per siklus produksi. Fase *upscaling* (Real-ESRGAN 2K)

teridentifikasi sebagai titik hambat utama (*bottleneck*) dengan durasi rata-rata 36,47 detik atau menyumbang 67,90% dari total waktu pemrosesan. Meskipun demikian, kapasitas keluaran (*throughput*) sistem berada pada angka 67,06 proses per jam (sekitar 67 aset visual/jam) dalam kondisi operasional normal.

3. Efisiensi Waktu dan Beban Kerja (*Effort*)

   Penerapan sistem otomatisasi berhasil memberikan efisiensi waktu produksi sebesar 89,89% jika dibandingkan dengan metode manual (531,06 detik vs 53,68 detik). Dari dimensi interaksi pengguna (*effort*), sistem memangkas kerumitan alur kerja dari 22 aktivitas operasional manual multi-*platform* menjadi hanya 2 langkah interaksi fisik (mengetik *prompt* dan mengirim pesan) pada Telegram *bot*, yang menghasilkan penurunan beban kerja operasional sebesar 90,91%%.

4.5.1 Evaluasi Aspek Etika dan Hak Cipta

Adapun dalam proses generasi dan pemanfaatan aset visual berbasis AI, sistem ini turut memperhatikan prinsip-prinsip etika *digital,* serta ketentuan hak cipta. Mengutip pedoman etika penggunaan gambar AI dari Dinas Komunikasi, Informatika dan Statistik (DKIS) Kota Cirebon (2025), penerapan etika pada sistem ini mencakup beberapa poin utama:

1. Transparansi dan Lisensi Komersial (*Commercial Licensing*): Penggunaan model FLUX.1 Pro dari Black Forest Labs secara resmi mencantumkan lisensi penggunaan komersial (*commercial use*). Hal ini memastikan bahwa seluruh gambar luaran yang diproduksi oleh sistem memiliki legalitas penggunaan yang sah untuk diperjualbelikan maupun diunggah ke *platform microstock,* seperti Adobe Stock, selama memenuhi ketentuan deklarasi konten *Generative* AI dan hak kekayaan intelektual *platform* tar*get*.
2. Pengakuan Sumber dan Kepatuhan Data Pelatihan (*Data Sourcing & Compliance*): Pemilihan *generator* AI mempertimbangkan keterbukaan sumber data pelatihan. Pengembang model FLUX (Black Forest Labs) secara publik menyediakan dokumentasi keterbukaan data (*Training Data*

Disclosure) sesuai regulasi California AB 2013 yang dapat diakses melalui halaman transparansi resmi mereka (Black Forest Labs, 2026). Dokumentasi ini mengonfirmasi, bahwa seluruh ekosistem model FLUX dilatih menggunakan kombinasi data terkurasi dari pihak ketiga, data sintetis, serta data internal yang telah melalui proses pembersihan (*deduplication*) dan deidentifikasi guna memenuhi standar keterbukaan serta regulasi AI internasional.

3. Pertimbangan Sosial dan Keamanan Konten (*Social Considerations & Content Safety*): Model FLUX 1.1 Pro memiliki *filter* keamanan bawaan yang secara otomatis menolak atau menyaring instruksi yang tidak pantas. Hal ini memastikan sistem terhindar dari menghasilkan gambar yang memperkuat stereotip, bias, misrepresentasi kelompok tertentu, maupun konten sensitif yang melanggar standar etika.

4.5.2 Kelemahan dan Kelebihan Gambar AI

Penggunaan model *Generative* AI dalam proses pembuatan aset visual menawarkan keunggulan signifikan dari segi efisiensi dan kecepatan produksi. AI mampu menghasilkan gambar berkualitas tinggi dalam hitungan detik berdasarkan deskripsi teks (*prompt*), yang secara substansial memangkas waktu kerja manual. Selain itu, sistem ini memberikan fleksibilitas eksplorasi gaya visual yang luas, mempermudah proses iterasi ide, serta memungkinkan otomatisasi skala besar bagi kebutuhan industri kreatif.

Di sisi lain, teknologi ini masih memiliki keterbatasan mendasar. Meskipun dalam pengujian terhadap 15 *prompt* yang dilakukan tidak ditemukan adanya cacat visual yang signifikan, hal tersebut tidak menutup kemungkinan terjadinya fenomena halusinasi AI pada pengujian skala yang lebih luas. Halusinasi visual ini umumnya berupa ketidakakuratan anatomi, timbulnya artefak *rendering* yang janggal, hingga kegagalan model dalam menerjemahkan logika spasial instruksi. Keterbatasan ini terjadi, karena model generatif bekerja berdasarkan analisis pola statistik *probabilistik* dari *dataset* pelatihan, bukan atas dasar pemahaman konteks visual secara nyata. Oleh karena itu, hasil yang diperoleh tetap memerlukan

pengawasan manusia (*human-in-the-loop*) guna memastikan kualitas akhir aset.

5. PENUTUP

5.1 Kesimpulan

Berdasarkan hasil penelitian, perancangan, pengujian, dan evaluasi yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Penelitian ini berhasil merancang dan mengimplementasikan *agentic workflow* berbasis Google Gemini dan n8n pada *bot* Telegram untuk otomatisasi produksi aset visual *microstock*. Sistem terbukti mampu mengintegrasikan seluruh rantai proses secara otomatis mulai dari riset tren pasar sebagai fitur pendukung, eksekusi generasi gambar berbasis AI, otomatisasi *metadata* (*title* dan *keywords*), peningkatan resolusi gambar menjadi (2048×2048) / 2K, hingga pengiriman berkas akhir ke Telegram *bot* pengguna dengan tingkat keandalan ideal berkat adanya mekanisme *error handling* pada tiap *node* integrasi API.
2. Pemangkasan alur kerja produksi aset visual berhasil diwujudkan melalui skenario interaksi perintah tunggal (*single-trigger*) berbasis *cloud*. Integrasi antarmuka berbasis Telegram *bot* dan infrastruktur *cloud* terbukti menyederhanakan proses kerja yang semula terdiri dari 22 langkah operasional manual menjadi 2 langkah interaksi, sekaligus mengalihkan seluruh beban komputasi berat dari perangkat keras lokal ke *server cloud* tanpa ketergantungan pada spesifikasi perangkat pengguna.
3. Hasil analisis menunjukkan bahwa sistem yang dibangun unggul dari segi kualitas luaran, kinerja, maupun efisiensi jika dibandingkan dengan pengerjaan secara manual. Secara kualitatif, sistem konsisten menghasilkan aset gambar beresolusi 2K dan *metadata* yang relevan. Secara kuantitatif, sistem mencatatkan rata-rata waktu pemrosesan sebesar 53,68 detik per siklus produksi dan *throughput* sebesar 67,06 proses per jam. Implementasi ini terbukti memberikan tingkat efisiensi waktu sebesar 89,89% serta memangkas beban kerja operasional pengguna sebesar 90,91%.

5.2 Saran

Berdasarkan hasil penelitian yang telah dilakukan, terdapat beberapa saran yang dapat diterapkan untuk pengembangan penelitian selanjutnya agar sistem otomatisasi alur kerja dapat memiliki performa yang lebih optimal:

1. Memperluas fleksibilitas pembuatan gambar dengan mendukung berbagai pilihan rasio aspek (*aspect ratio*), seperti 16:9, 9:16, atau 4:3, guna memenuhi kebutuhan spesifikasi aset visual pada berbagai *platform microstock* yang saat ini masih terbatas pada rasio 1:1.
2. Mengembangkan kapabilitas sistem dengan mengintegrasikan fitur generasi video berbasis AI (*text-to-video* atau *image-to-video*) untuk memperkaya jenis portofolio aset *digital* yang dapat dihasilkan dan didistribusikan.
3. Meningkatkan kapasitas komputasi peningkatan resolusi menggunakan API berbasis *cloud* yang lebih masif untuk memangkas pusat hambatan kerja pada sistem.
4. Mengeksplorasi penggunaan model generasi gambar yang lebih efisien atau menerapkan teknik akselerasi model guna mempercepat waktu eksekusi tanpa menurunkan kualitas visual aset.
5. Menambahkan variasi model kecerdasan buatan sebagai API cadangan untuk menjaga kelancaran operasional sistem saat layanan utama mengalami gangguan.
6. Mengimplementasikan pembaruan status proses secara *real-time* pada Telegram agar pengguna dapat memantau tahapan alur kerja *detail* yang sedang berjalan.
7. Mengembangkan fitur integrasi multi-*user* pada Telegram *Bot* untuk pengelolaan ruang penyimpanan berkas.

DAFTAR PUSTAKA

Abednego, A & Julianto, E 2021, 'Pengembangan Aplikasi Layanan Multiguna Menggunakan Low-Code Platform (Studi Kasus: Astra Credit Companies)', *Jurnal Informatika Atma Jogja*, vol. 2, no. 1, pp. 10-19, viewed 9 August 2026, https://ojs.uajy.ac.id/index.php/jiaj/article/view/5463.

Abimanyu, A 2026, 'Pengaruh harga dan kualitas produk terhadap keputusan pembelian pada platform *microstock* Shutterstock di komunitas digital marketing', *PENG: Jurnal Ekonomi dan Manajemen*, vol. 3, no. 1, pp. 919-932, DOI: 10.62710/v07pnr30.

Adrian, F & Sahono, MF 2026, 'Pemanfaatan Generative Artificial Intelligence (AI) dalam efisiensi production konten media sosial untuk pemasaran digital UMKM', *Jurnal Media Akademik (JMA),* vol. 4, no. 3, DOI: 10.62281/b8xd5j13.

Affandi, E & Syahputra, T 2018, 'Pemodelan UML manajeman sistem inventory', *Jurnal Teknologi Sistem Informasi dan Sistem Komputer TGD*, vol. 1, no. 2, pp. 14-25. https://ejurnal.umri.ac.id/index.php/JST/article/view/7986.

Akbar, R, Weriana, W, Siroj, RA & Afgani, MW 2023, 'Experimental research dalam metodologi pendidikan', *Jurnal Ilmiah Wahana Pendidikan*, vol. 9, no. 2, pp. 465-474. https://jurnal.peneliti.net/index.php/JIWP/article/view/3165.

Andini, W, Masitoh, RD, Harati, CC, Sari, NNK & Pranatawijaya, VH 2024, 'Implementasi Sistem Jual Beli Kopi Berbasis Mobile Android Dengan Penggunaan Api Openai Untuk Generate Deskripsi Produk Kopi', *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 8, no. 4, pp. 7604-7607, viewed 9 August 2026, https://doi.org/10.36040/jati.v8i4.9830.

Arif, MF 2020, 'Implementasi enkripsi URL pada website menggunakan metode Base64 dan Rotation13', *SPIRIT*, vol. 12, no. 1,.https://repository.unmerpas.ac.id/123/.

Artanto, H & Nurdiyansyah, F 2017, 'Penerapan SEO (Search Engine Optimization) untuk meningkatkan penjualan produk', *Journal of Information Technology and Computer Science*, vol. 2, no. 1, p. 224632. https://doi.org/10.31328/jointecs.v2i1.409.

Beyan, E. V. P., & Rossy, A. G. C. (2023). A review of AI *image generator*: Influences, challenges, and future prospects for architectural field. *Journal of Artificial Intelligence in Architecture*, 2(1), 53–65. https://doi.org/10.24002/jarina.v2i1.6662.

Black Forest Labs, 2026. *Transparency: Training Data Summaries*. Tersedia di: https://bfl.ai/transparency?tab=training-data-summaries [Diakses 10 Agustus 2026].

Briliansah, MD, Sari, NP & Susanto, E 2026, 'ANALISIS EFEKTIVITAS GENERATIVE AI DALAM PROSES KREATIF PRODUKSI KONTEN MULTIMEDIA DIGITAL', *Jurnal Media Akademik (JMA)*, vol. 4, no. 4, viewed 9 August 2026, https://doi.org/10.62281/dkdjt653.

Casaclang, H, Ionescu, B, Kim, Y & Jo, JY 2026, 'Self-hosted workflow automation for AI-based cybersecurity operation', *Journal of The Colloquium for Information Systems Security Education*, vol. 13, no. 1, pp. 21-21, DOI: 10.53735/cisse.v13i1.241.

Çetin, ŞB 2023, 'Real-ESRGAN: A deep learning approach for general *image* restoration and its application to aerial *image*s', *Advanced Remote Sensing*, vol. 3, no. 2, pp. 90-99, viewed 9 August 2026, https://www.researchgate.net/*public*ation/376398493_Real-ESRGAN_A_deep_learning_approach_for_general_*image*_restoration_and_its_application_to_aerial_*image*s_Advanced_Remote_Sensing_Journal_Vol_3_No_2_2023_90-99.

Chaudhry, GI, Choukse, E, Qiu, H, Goiri, Í, Fonseca, R, Belay, A & Bianchini, R 2025, 'Murakkab: Resource-efficient *agentic* workflow orchestration in cloud platforms', *arXiv preprint* arXiv:2508.18298, viewed 31 July 2026,https://www.usenix.org/conference/osdi26/presentation/chaudhry.

Digiarty 2025, 'AI video *upscale*r for Mac True 4K enhancement made easy', *Macworld*. https://www.macworld.com/article/2806334/ai-video-*upscale*r-for-mac-true-4k-enhancement-made-easy.html.

Dinas Komunikasi, Informatika dan Statistik Kota Cirebon, 2025. *Jangan Asal Pakai Gambar AI, Ini Etika dan Aturan Hak Cipta yang Harus Kamu Tahu*. Tersedia di: https://dkis.cirebonkota.go.id/artikel/jangan-asal-pakai-gambar-ai-ini-etika-dan-aturan-hak-cipta-yang-harus-kamu-tahu [Diakses 10 Agustus 2026].

Dwiyatno, S, Rakhmat, E & Gustiawan, O 2020, 'Implementasi virtualisasi server berbasis docker container', *Jurnal PROSISKO*, vol. 7, no. 2. https://doi.org/10.30656/prosisko.v7i2.2520.

Falaq, FI & Pradana, MIY 2025, 'Pengembangan Knowledge Mangement System (KMS) menggunakan *Agentic* AI dengan Pendekatan Vibe Coding', *Jurnal Prima Manajemen*, vol. 1, no. 2, pp. 351-364, viewed 9 August 2026, https://journal.al-afif.org/index.php/jpm/article/view/624.

Farros, MFZ & Pramana, AL 2026, 'Rancang bangun buku tamu digital menggunakan otomasi n8n dengan metode agile', *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 10, no. 2, pp. 3106-3113. https://doi.org/10.36040/jati.v10i2.17927.

Goodman, J 2022, Digital *image*s, EBSCO Research *Start*ers, viewed 24 June 2026. https://www.ebsco.com/research-*start*ers/computer-science/digital-*image*s.

Hanafi, A, Sukarsa, IM & Wiranatha, AAKAC 2017, 'Pertukaran data antar database dengan menggunakan teknologi API', *Lontar Komputer*, vol. 8, no. 1, pp. 22-30. https://scispace.com/pdf/pertukaran-data-antar-database-dengan-menggunakan-teknologi-3py2a7f8zj.pdf.

Hariyadi, INR & Soraya, DU 2025, 'Implementasi *chat*bot AI berbasis n8n untuk layanan informasi administrasi akademik Departemen Teknik Elektro dan Informatika', *JEPIN (Jurnal Edukasi dan Penelitian Informatika),* vol. 11, no. 3, pp. 442-447, DOI: 10.26418/jp.v11i3.101635.

Herdiyatmoko, HF 2022, 'Desain sistem backend berbasis REST API menggunakan Framework Laravel 7', *SKANIKA: Sistem Komputer dan Teknik Informatika*, vol. 5, no. 2, pp. 136-144. https://doi.org/10.36080/skanika.v5i2.2947.

Imran, M & Almusharraf, N 2024, 'Google Gemini as a next generation AI educational tool: a review of emerging educational technology', *Smart Learning Environments*, vol. 11, no. 1, p. 22, viewed 9 August 2026, https://doi.org/10.1186/s40561-024-00308-3.

Indusface n.d., 'What is latency?', *Indusface Learning*. https://www.indusface.com/learning/what-is-latency/.

Insani, F & Patria, AS 2023, 'Ilustrasi sebagai aset visual Wawa Food and Cookies', Jurnal Desain Grafis Aplikatif, vol. 1, no. 1, pp. 15-27, DOI: 10.26740/desgrafia.v1i1.54484.

Insani, T, Fadholi, A, Mutaqin, I, Zein, R & Saputra, DIS 2020, 'Industri *microstock* sebagai peluang peningkatan ekonomi kreatif di tengah pandemi Covid-19', *JAMI: Jurnal Ahli Muda Indonesia*, vol. 1, no. 1, pp. 46-54, DOI: 10.46510/jami.v1i1.12.

Jeharu, VAN 2021, 'Digital creative labour: prosumsi desainer grafis kontributor dalam platform *microstock* Freepik', *Retorik: Jurnal Ilmu Humaniora*, vol. 9, no. 2, pp. 133-156, DOI: 10.24071/ret.v9i2.3491.

Jiang, YH, Lu, Y, Dai, L, Wang, J, Li, R & Jiang, B 2025, '*Agentic* Workflow for Education: Concepts and Applications', *arXiv preprint*, https://doi.org/10.48550/arXiv.2509.01517.

Khesya, N 2021, 'Mengenal flowchart dan pseudocode dalam algoritma dan pemrograman', *Preprints*, pp. 1-15. https://doi.org/10.31219/osf.io/syjn5.

Ko, HK, Park, G, Jeon, H, Jo, J, Kim, J & Seo, J 2023, 'Large-scale text-to-*image* generation models for visual artists' creative works', *Proceedings of the 28th International Conference on Intelligent User Interfaces*, pp. 919-933. https://doi.org/10.1145/3581641.3584078.

Kustanto, P, Khalil, RB & Noe'man, A 2024, 'Penerapan metode prototype dalam perancangan media pembelajaran interaktif', *Journal of Students' Research in Computer Science*, vol. 5, no. 1, pp. 83-94. https://doi.org/10.31599/6x0dfz47.

Laipaka, R 2017, 'Penerapan teknologi informasi pembelajaran e-learning menggunakan ADDIE model', *JUPITER (Jurnal Penelitian Ilmu dan Teknologi Komputer)*, vol. 9, no. 1, pp. 1-12. https://doi.org/10.5281/zenodo.3411828.

Luthfansa, ZM & Rosiani, UD 2021, 'Pemanfaatan Wireshark untuk sniffing komunikasi data berprotokol HTTP pada jaringan internet', *JIEET (Journal of Information Engineering and Educational Technology)*, vol. 5, no. 1, pp. 34-39. https://doi.org/10.26740/jieet.v5n1.p34-39.

Martadireja, S 2018, 'Pengaruh *Microstock* Terhadap Kesadaran Hak Kekayaan Intelektual Desainer Grafis', *Journal of Urban Society's Arts*, vol. 5, no. 1, pp. 19-28.

Masripah, S & Ramayanti, L 2019, 'Pengujian black box pada sistem informasi penerimaan siswa baru berbasis web', *Information System For Educators And Professionals: Journal of Information System*, vol. 4, no. 1, pp. 1-10. https://ejournal-binainsani.ac.id/index.php/ISBI/article/view/1181.

Muhajir, MDA, Prastiti, N & Koeshardianto, M 2025, 'Implementasi *Chat*bot Menggunakan Framework Langchain Berbasis Llm Gpt', *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 9, no. 2, pp. 2151-2158, viewed 9 August 2026, https://doi.org/10.36040/jati.v9i2.13003.

Nandal, P, Pahal, S, Khanna, A & Pinheiro, PR 2024, 'Super-resolution of medical *image*s using real ESRGAN', *IEEE Access*, vol. 12, pp. 176155-176170, https://doi.org/10.1109/ACCESS.2024.3497002.

Narulita, S, Nugroho, A & Abdillah, MZ 2024, 'Diagram Unified Modelling Language (UML) untuk perancangan sistem informasi manajemen penelitian dan pengabdian masyarakat (SIMLITABMAS)', *Bridge: Jurnal Publikasi Sistem Informasi dan Telekomunikasi*, vol. 2, no. 3, pp. 244-256. https://doi.org/10.62951/bridge.v2i3.174.

Niu, B, Song, Y, Lian, K, Shen, Y, Yao, Y, Zhang, K & Liu, T 2025, 'Flow: modularized *agentic* workflow automation', *arXiv preprint* arXiv:2501.07834, DOI: 10.48550/arXiv.2501.07834.

Nurfauzi, Y, Taime, H, Hanafiah, H, Yusuf, M & Asir, M 2023, 'Literature review: analysis of factors influencing purchasing decisions, product quality and competitive pricing', *Management Studies and Entrepreneurship Journal*, vol. 4, no. 1, pp. 183-188, DOI: 10.37385/msej.v4i1.1246.

OWASP Foundation 2026, 'Improper error *handling*', *OWASP Community*. https://owasp.org/www-community/Improper_Error_*Handling*.

Parlika, R, Wijaya, DCM, Nisaa, TA & Rahmawati, S 2021, 'Sistem integrasi BOT register terhadap website pengolah data menggunakan akses NGROK', *Jurnal Ilmiah Sinus (JIS)*, vol. 19, no. 2. https://ejournal.tsu.ac.id/index.php/e-jurnal_SINUS/article/view/531.

Pitaloka, LK & Kardoyo, K 2023, 'Membuka pintu pasar digital: pelatihan pemasaran digital untuk mendorong pertumbuhan UMKM olahan pangan di Salatiga', *Madaniya*, vol. 4, no. 4, pp. 1368-1380, DOI: 10.53696/27214834.559.

Purwanto, E, Sandjoko, SPM & Syahputra, SA 2025, 'Integrasi N8N dan algoritma generative *chat*bot untuk transformasi digital penerimaan mahasiswa baru', *SEMASTER: Seminar Nasional Teknologi Informasi & Ilmu Komputer*, vol. 4, no. 1, pp. 19-25. https://journal.unilak.ac.id/index.php/Semaster/article/view/30961.

Raharja, MP & Wijayanto, H 2023, 'Pengembangan Prototype Back-End Website Pelaporan Barang Hilang di PT. Presentologics Dicoding Academy Indonesia', *Jurnal Begawe Teknologi Informasi (JBegaTI)*, vol. 4, no. 2. https://doi.org/10.29303/jbegati.v4i2.1078.

Rashidi, J, Brata, KC & Brata, AH 2022, 'Pembangunan aplikasi program membership wahana go-kart menggunakan Flutter dengan pustaka Deepface (Studi kasus CV Indi Karya Teknik)', *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, vol. 6, no. 7, pp. 3054-3061. https://j-ptiik.ub.ac.id/index.php/j-ptiik/article/view/11265.

Replicate 2024, *How does Replicate work?*, Replicate Docs, viewed 9 August 2026, https://replicate.com/docs/reference/how-does-replicate-work.

Rifky, S., Kharisma, L.P.I., Afendi, H.A.R., Napitupulu, S., Ulina, M., Lestari, W.S. dan Rizal, A.A. (2024) *Artificial intelligence: Teori dan penerapan AI di berbagai bidang*. PT. Sonpedia Publishing Indonesia.

Saputra, AC & Saragih, AS 2020, 'Implementasi algoritma Rijndael dalam enkripsi dan dekripsi gambar digital berbasis web', Jurnal Teknologi Informasi: Jurnal Keilmuan dan Aplikasi Bidang Teknik Informatika, vol. 14, no. 1, pp. 52-63, DOI: 10.47111/jti.v14i1.609.

ScienceDirect n.d., *Server Architecture*, ScienceDirect Topics, viewed 6 August 2026, https://www.sciencedirect.com/topics/computer-science/server-architecture.

Sianturi, D, Munir, M & Sunny, AG 2023, 'Tantangan penggunaan artificial intelligence dalam hybrid warfare', *Innovative: Journal Of Social Science Research*, vol. 3, no. 5, pp. 9984-9996, viewed 9 August 2026, https://j-innovative.org/index.php/Innovative/article/view/6080.

Singh, A, Ehtesham, A, Kumar, S & Khoei, TT 2024, 'Enhancing AI systems with *agentic* workflows patterns in large language model', 2024 *IEEE World AI IoT Congress (AIIoT), IEEE*, pp. 527-532, DOI: 10.1109/AIIoT61789.2024.10578990.

Sugiono, S 2020, 'Industri konten digital dalam perspektif society 5.0 (digital content industry in society 5.0 perspective)', *JURNAL IPTEKKOM (Jurnal Ilmu Pengetahuan & Teknologi Informasi)*, vol. 22, no. 2, pp. 175-191, DOI: 10.17933/iptekkom.22.2.2020.175-191.

Susilawati, E 2022, 'Pemanfaatan JSON untuk menampilkan data realtime Covid-19 dengan Model View Presenter', *Jurnal Teknoinfo*, vol. 16, no. 1. https://doi.org/10.33365/jti.v16i1.780.

Syahputra, F, Sabrina, E, Ilmi, MB, Chesio, MR, Handoko, MRF & Siregar, HA 2026, 'RANCANG BANGUN SISTEM TEXT-TO-*IMAGE* AI BERBASIS GOOGLE GEMINI PADA SERVERLESS', *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 10, no. 1, pp. 7-14, viewed 9 August 2026, https://doi.org/10.36040/jati.v10i1.16574.

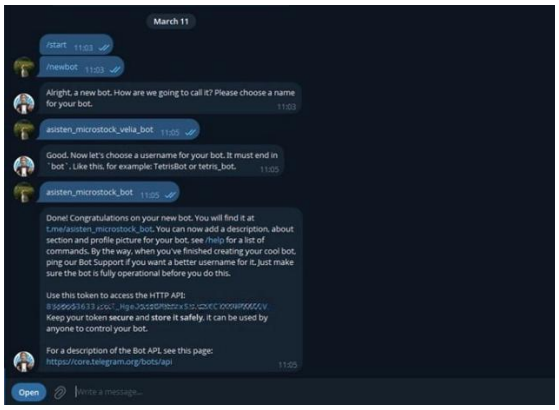
Wardhana, I & Assegaff, S 2017, 'Perancangan dan Penerapan Arsitektur Cloud Storage Pada Iain STS Jambi', *Jurnal Manajemen Sistem Informasi*, vol. 2, no. 1, pp. 244-259. http://dx.doi.org/10.11591/jurnalmsi.v12i4.xxxx.

Wayahdi, MR & Ruziq, F 2023, 'Pemodelan sistem penerimaan anggota baru dengan unified modeling language (UML)(Studi kasus: Programmer Association of Battuta)', *Jurnal Minfo Polgan*, vol. 12, no. 1, pp. 1514-1521. https://doi.org/10.33395/jmp.v12i1.12870.

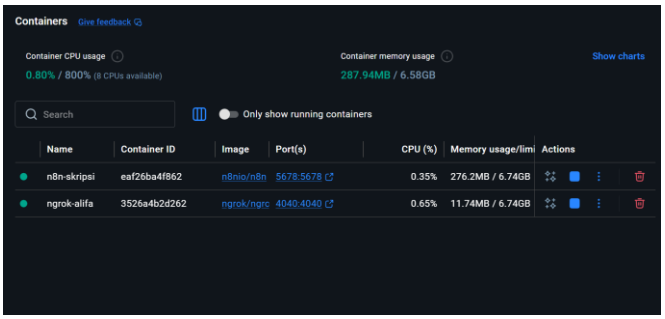
Wikipedia Contributors 2026, 'Kling AI', *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Kling_AI.

LAMPIRAN

Konfigurasi Lingkungan Sistem

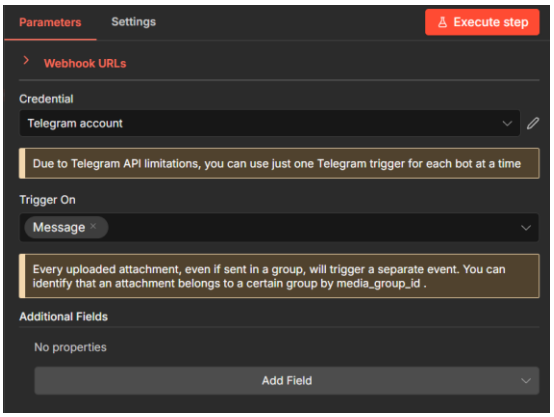


Lampiran 1: Pembuatan *Bot* Telegram

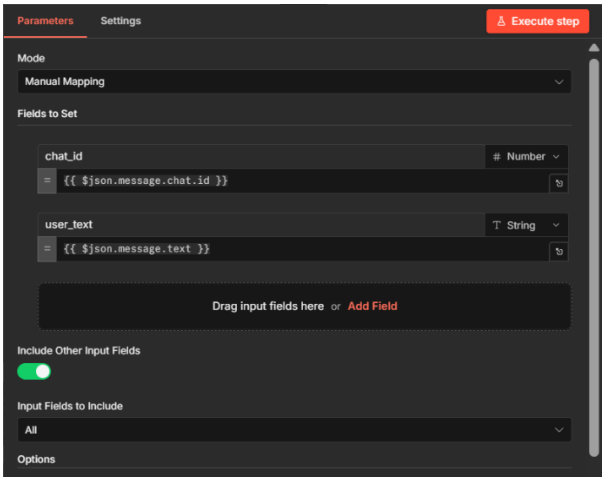


Lampiran 2: Konfigurasi *Container* Docker

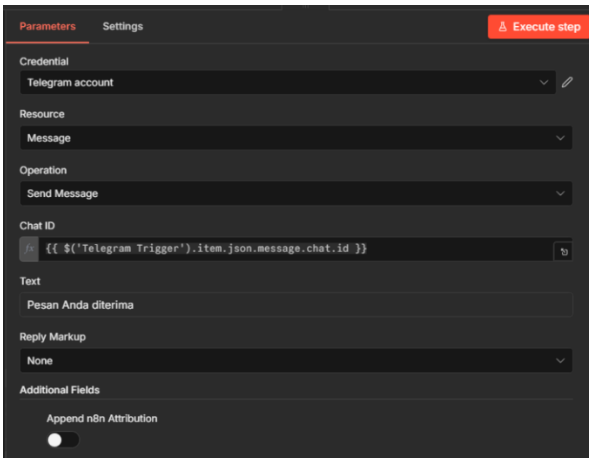
Konfigurasi *Node* Canvas n8n



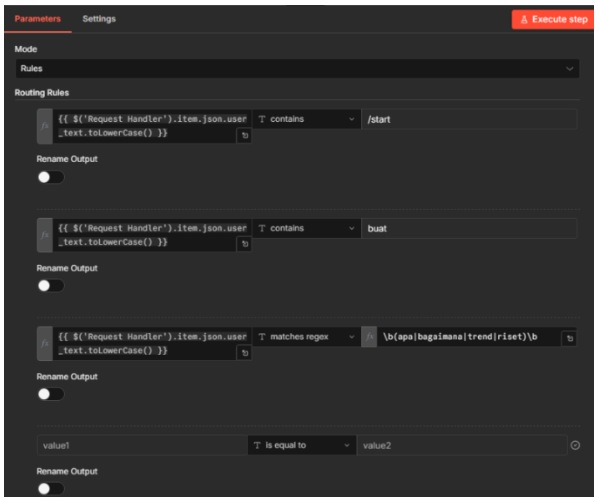
Lampiran 3: Konfigurasi *Node* Telegram *Trigger*



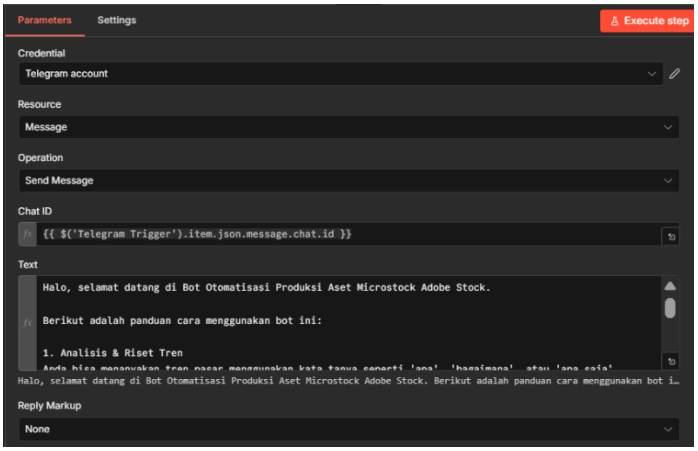
Lampiran 4: Konfigurasi *Node Request Handler*



Lampiran 5: Konfigurasi *Node Feedback* Awal



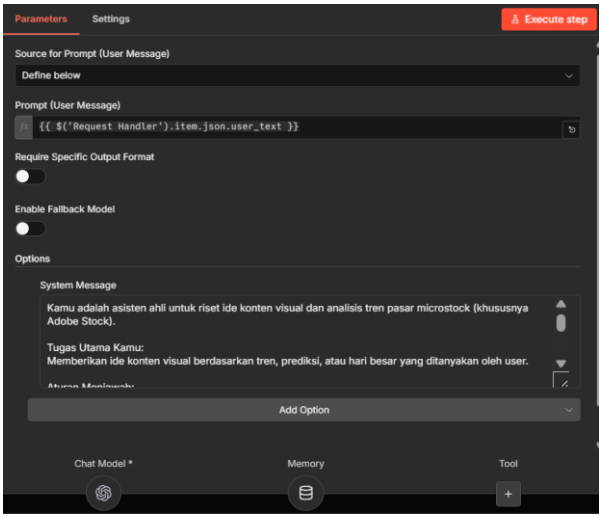
Lampiran 6: Konfigurasi *Node Switch* Menu *Routing*



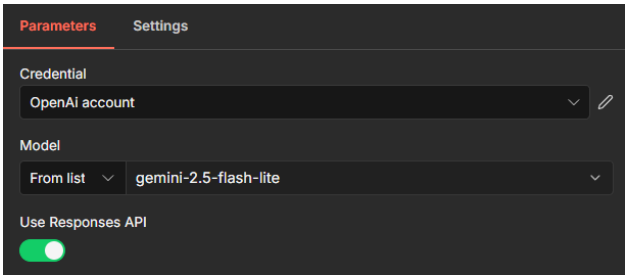
Lampiran 7: Konfigurasi *Node Feedback Prompt Start*



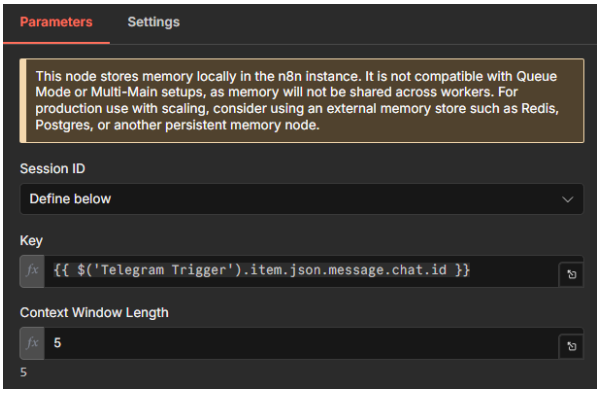
Lampiran 8: Konfigurasi *Node Feedback* Deteksi Riset



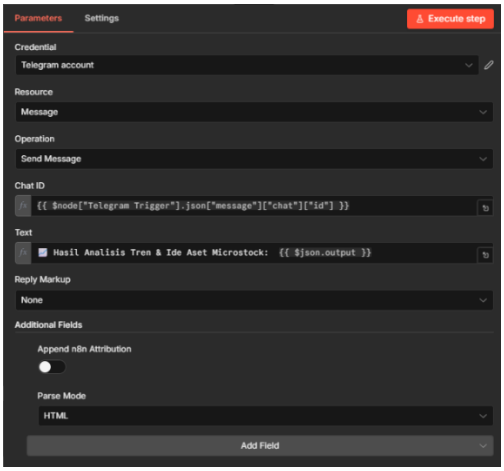
Lampiran 9: Konfigurasi *Node* AI *Agent* Riset Tren



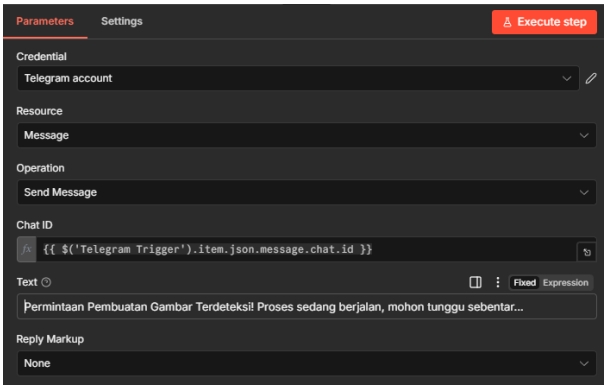
Lampiran 10: Konfigurasi *Node* Model *Chat* AI *Agent* Riset



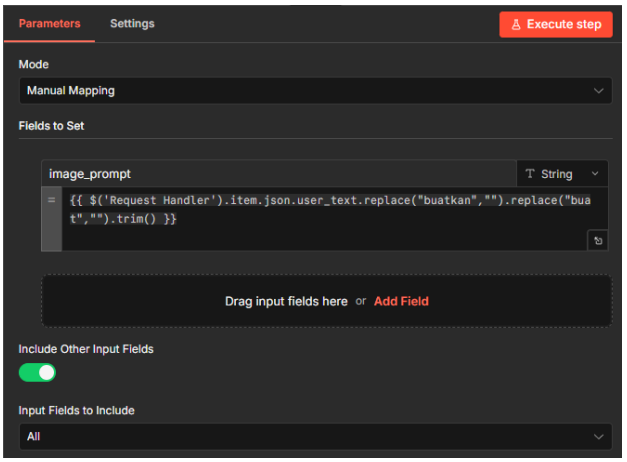
Lampiran 11: Konfigurasi *Window Buffer Memory*



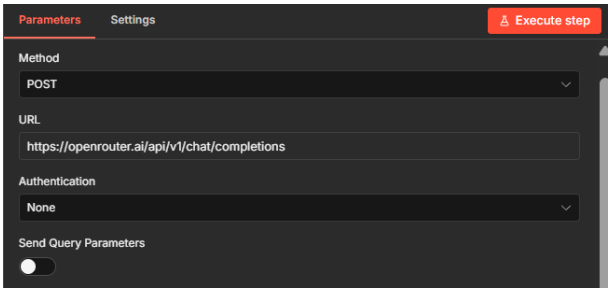
Lampiran 12: Konfigurasi *Node* Telegram *Feedback* Riset



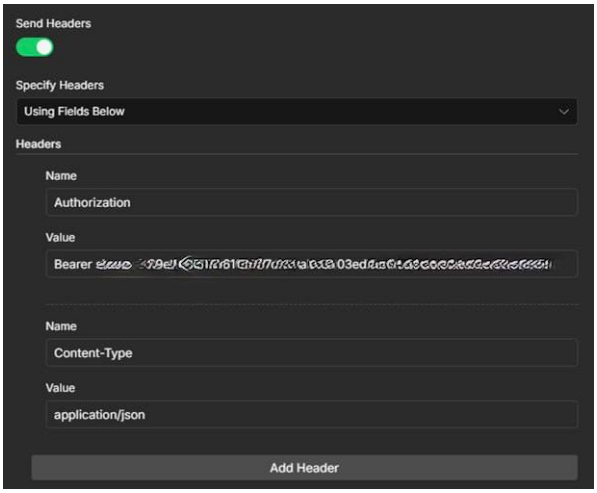
Lampiran 13: Konfigurasi *Node Feedback* Deteksi Pembuatan Aset



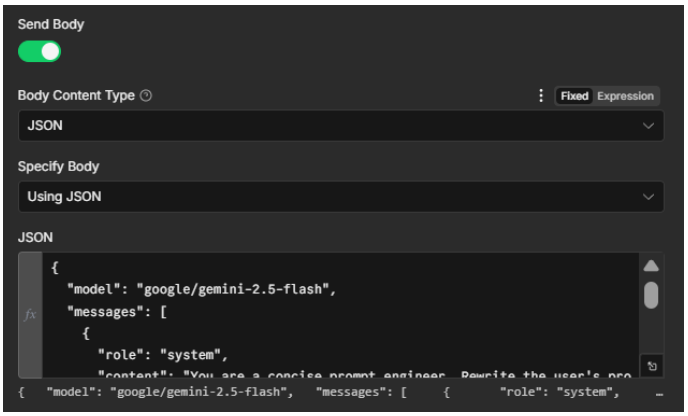
Lampiran 14: Konfigurasi *Node Formatter*



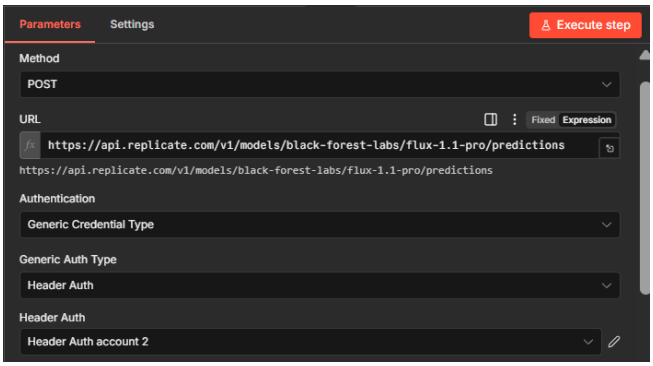
Lampiran 15: Konfigurasi *Node Prompt Enhancer*



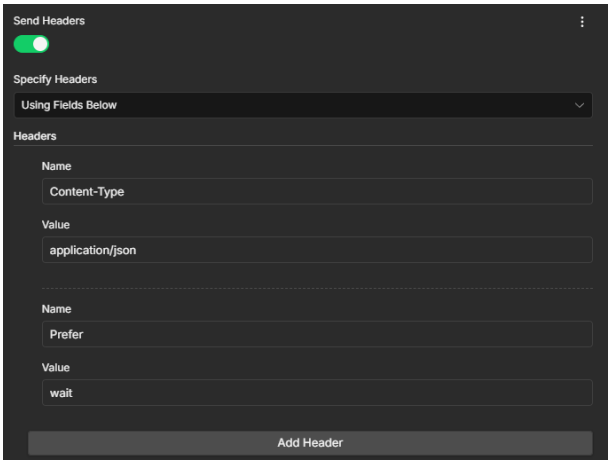
Lampiran 16: Konfigurasi *Headers Node Enhancer*



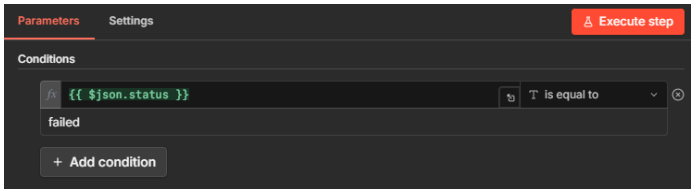
Lampiran 17: Konfigurasi *Body* JSON *Node Enhancer*



Lampiran 18: Konfigurasi *Node HTTP Request* Replicate FLUX 1.1 Pro



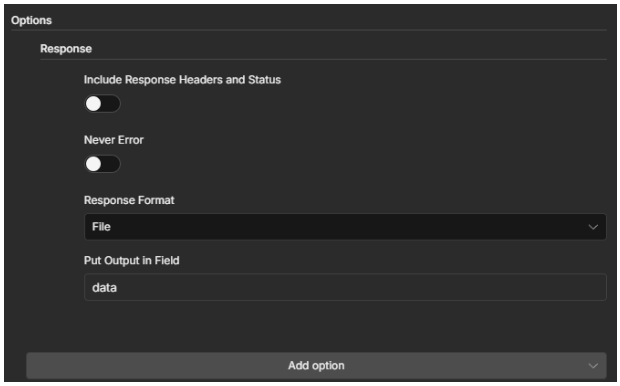
Lampiran 19: Konfigurasi *Headers* HTTP *Request*



Lampiran 20: Konfigurasi *Node* Cek Status Pelanggaran Konten



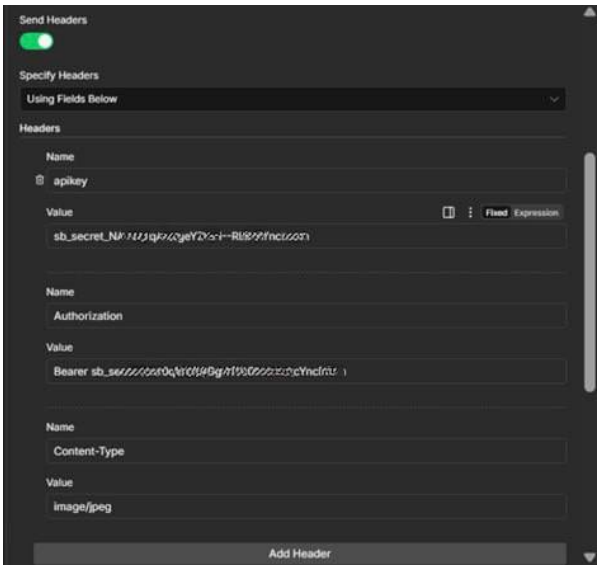
Lampiran 21: Konfigurasi *Node Image URL Generator*



Lampiran 22: Konfigurasi *Options Node Image Url Generator*



Lampiran 23: Konfigurasi *Method & URL Node Raw-assets*



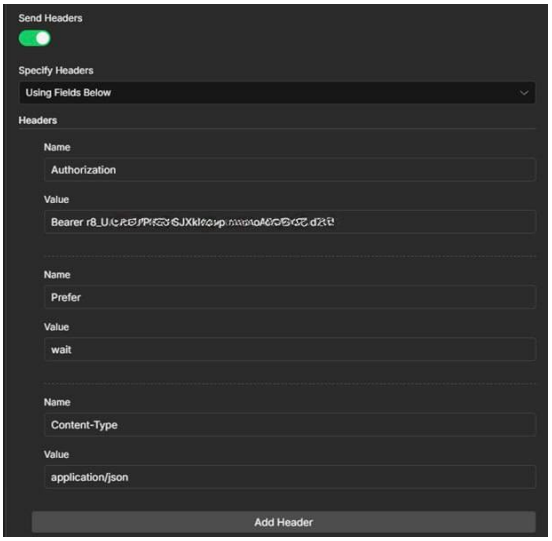
Lampiran 24: Konfigurasi *Headers Node Raw-assets*



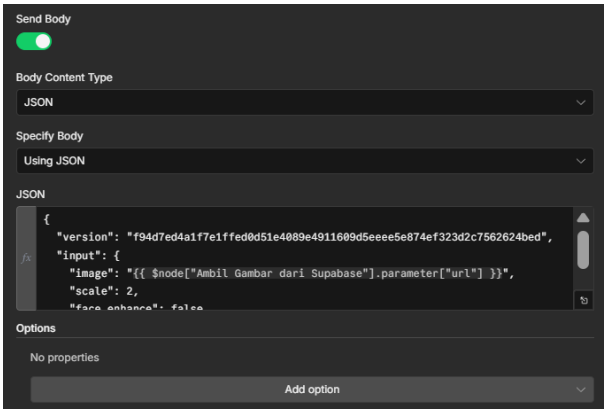
Lampiran 25: Konfigurasi *Node GET Public* URL Supabase



Lampiran 26: Konfigurasi *Method* dan URL pada *Node Upscale* Gambar



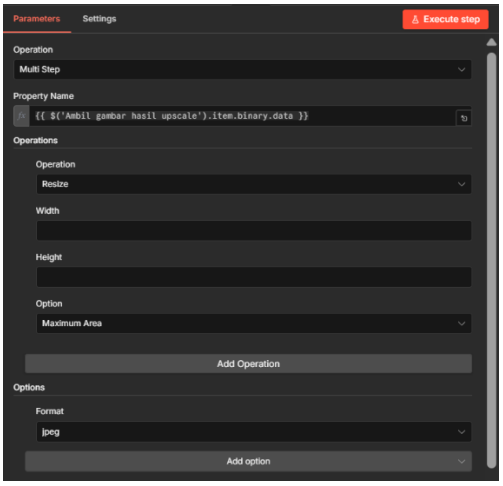
Lampiran 27: Konfigurasi *Send Headers* pada *Node Upscale*



Lampiran 28: Konfigurasi *Send Body* pada *Node Upscale*



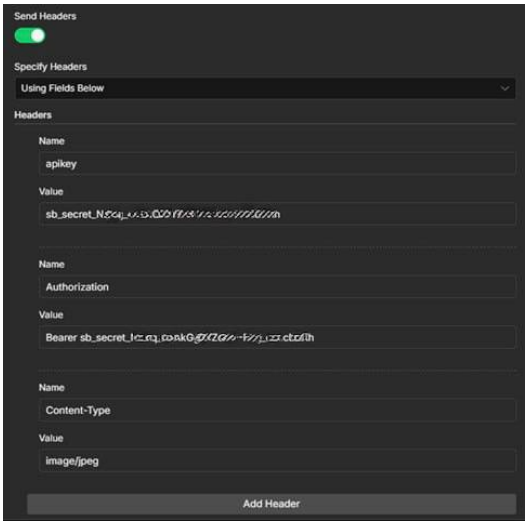
Lampiran 29: Konfigurasi *Node Get* Gambar *Upscale*



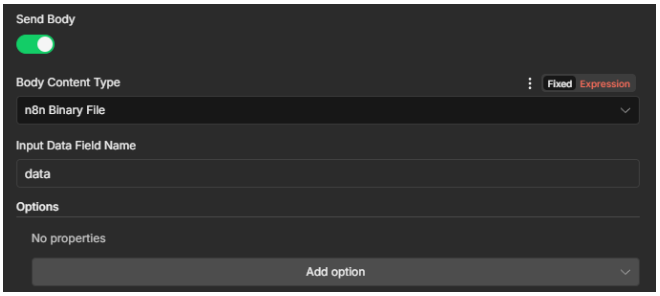
Lampiran 30: Pengaturan *Convert* Format pada *Node Convert*



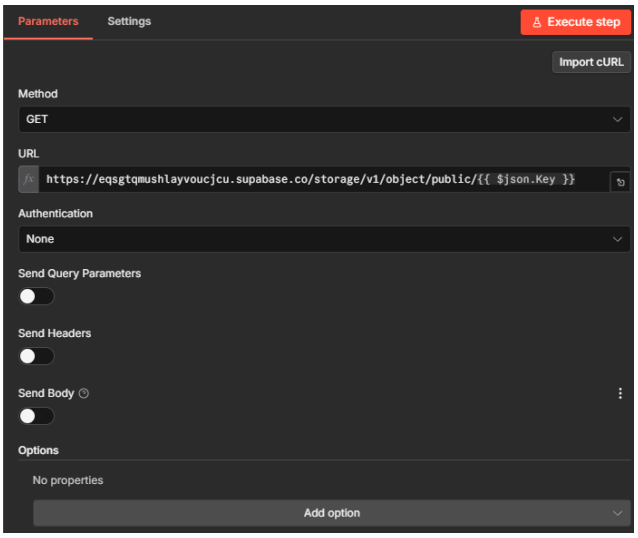
Lampiran 31: Konfigurasi *Method* dan *URL Node Bucket Upscaled-Assets*



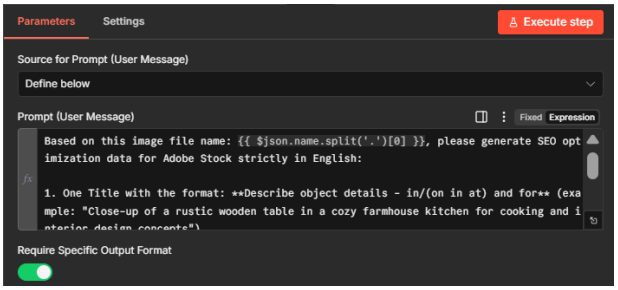
Lampiran 32: Konfigurasi *Headers Node Bucket Upscaled-Assets*



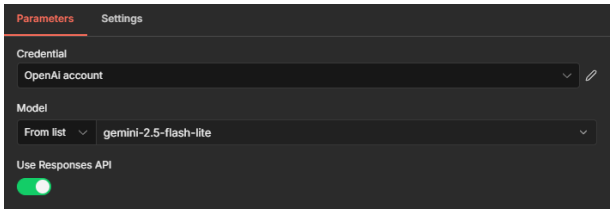
Lampiran 33: Konfigurasi *Body Node Bucket Upscaled-Assets*



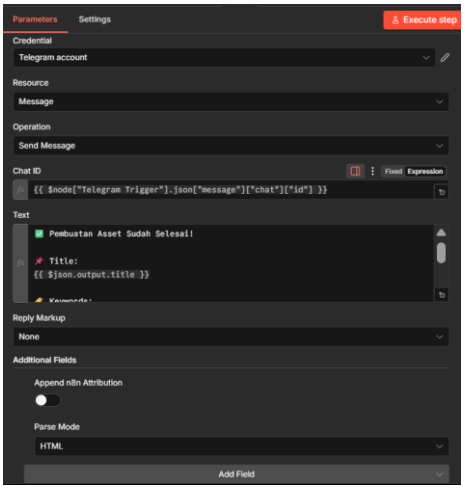
Lampiran 34: Konfigurasi *Node Image* URL *Generator* 1



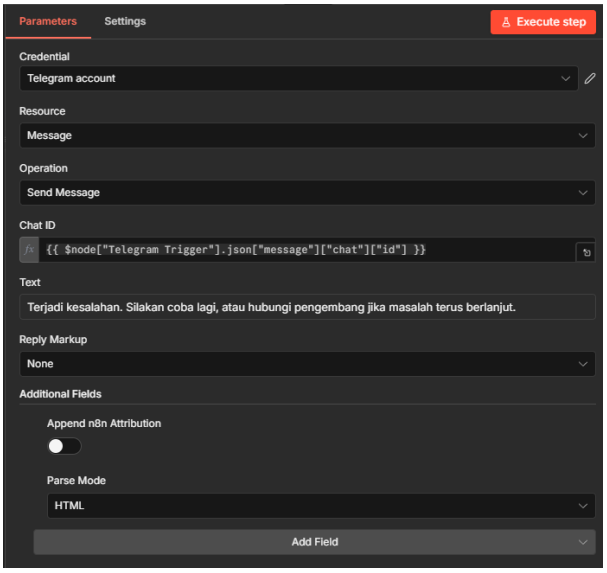
Lampiran 35: Konfigurasi *Prompt* dan *Output* Format *Metadata*



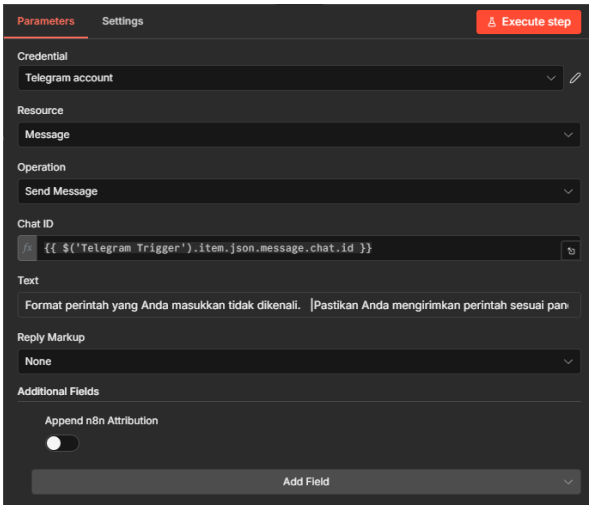
Lampiran 36: Konfigurasi Model *Node* AI *Agent Metadata*



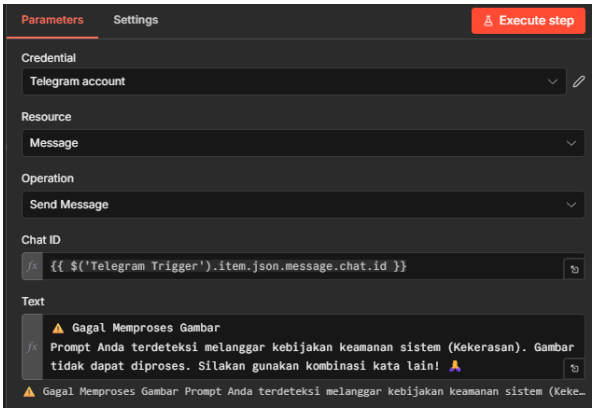
Lampiran 37: Konfigurasi *Node* Telegram Hasil Akhir



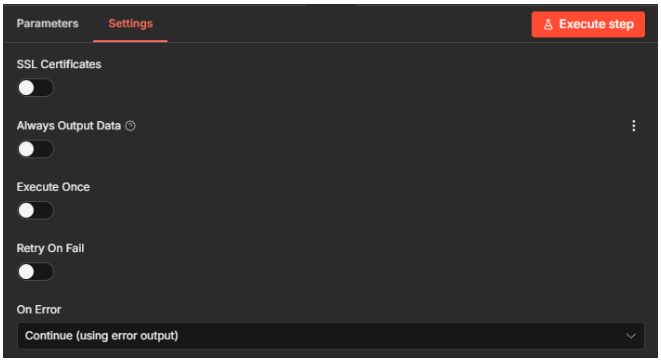
Lampiran 38: Konfigurasi Pesan Penanganan Kesalahan Jaringan



Lampiran 39: Konfigurasi Pesan Penanganan Kesalahan Format *Prompt*



Lampiran 40: Konfigurasi Pesan Penanganan Kesalahan Konten Kebijakan



Lampiran 41: Konfigurasi Penanganan Kesalahan Setiap *Node*

Hasil *Metadata* dari 15 kali Uji Percobaan

apple, red apple, fruit, healthy eating, food, fresh, single, isolated, white background, studio shot, juicy, ripe, organic, natural, sweet, delicious, nutrition, vitamin, raw, whole, closeup, healthy lifestyle, dieting, vegetarian, vegan, ingredient, cooking, baking, dessert, snack, crisp, shiny, perfect, simple, minimalist, clean, vibrant, color, autumn, harvest, orchard, tree, nature, garden, freshness, wellness, refreshment, diet food

Lampiran 37: Hasil *Output Keywords* Percobaan ke-1

coffee, cup, mug, hot, drink, beverage, steam, smoke, aroma, caffeine, morning, breakfast, energy, break, cafe, restaurant, kitchen, table, ceramic, porcelain, white, yellow, background, simple, minimal, clean, graphic, illustration, vector, drawing, art, modern, contemporary, isolated, object, food, refreshment, warm, cozy, design, concept, beverage, beverage industry, cafe culture, hot beverage, aromatic, invigorating, drinkware, saucer

Lampiran 38: Hasil *Output Keywords* Percobaan ke-2

banana, bananas, slice, slices, sliced, fruit, tropical, yellow, food, healthy, organic, fresh, natural, nutrition, diet, sweet, dessert, ingredient, raw, whole, ripe, cut, close-up, macro, top view, studio shot, isolated, white background, simple, minimalist, clean, vibrant, juicy, texture, pattern, exotic, vegetarian, vegan, wellness, refreshment, breakfast, snack, sweet food, delicious, natural food, copy space, health, vitamin

Lampiran 39: Hasil *Output Keywords* Percobaan ke-3

coffee, cup, white cup, coffee cup, hot beverage, caffeine, morning, breakfast, drink, beverage, dark specks, top view, overhead view, flat lay, bright green, vibrant, shadow, isolated, studio shot, minimalist, simple, clean, refreshing, energy, refreshment, cafe, aroma, taste, delicious, nobody, indoors, domestic life, lifestyle, food and drink, close-up, macro, detail, creative, conceptual, modern, graphic, graphic design, presentation, background, copy space, art, illustration, digital art

Lampiran 40: Hasil *Output Keywords* Percobaan ke-4

leather wallet, brown wallet, mens wallet, womens wallet, leather goods, accessory, fashion accessory, wallet, close up, detail, texture, stitching, craftsmanship, luxury, high end, genuine leather, full grain leather, handmade, premium, stylish, elegant, modern, minimalist, essential, everyday carry, EDC, personal finance, money, cash, cards, credit cards, business cards, identity, security, protection, storage, organization, man, woman, unisex, isolated, white background, studio shot, professional photography, commercial use, retail, shopping, gift idea, wealth

Lampiran 41: Hasil *Output Keywords* Percobaan ke-5

vintage camera, retro camera, film camera, old camera, antique camera, analog camera, camera, photography, photograhy equipment, camera accessory, collectible, collectible camera, camera gear, camera lover, photography enthusiast, shutter, lens, aperture, light meter, viewfinder, film roll, nostalgia, memory, past, history, wooden, wood, table, surface, texture, background, natural light, sunlight, shadows, contrast, detail, close-up, high resolution, professional, creative, artistic, conceptual, design, decor, retro style, vintage style, classic, timeless, isolated

Lampiran 42: Hasil *Output Keywords* Percobaan ke-6

desk lamp, table lamp, bedside lamp, nightstand lamp, modern lamp, contemporary lamp, home decor, interior design, lighting, bedroom, furniture, wood, nightstand, bed, pillow, light, illumination, warm light, cozy, ambiance, evening, reading light, study, office, workspace, residential, domestic, indoor, object, technology, electricity, switch, cable, clean, simple, minimalist, neutral, close-up, detail, lifestyle, comfort, relaxation, illuminated, glowing, bright, shine, study lamp, focus light, bedside table

Lampiran 43: Hasil *Output Keywords* Percobaan ke-7

strawberry, strawberries, fresh, fruit, food, red, healthy, organic, ripe, sweet, delicious, natural, agriculture, garden, harvest, summer, dessert, vitamin, nutrition, juicy, freshness, closeup, macro, bowl, clear, glass, wood, surface, rustic, kitchen, background, vibrant, color, taste, ingredient, raw, vegan, vegetarian, dieting, gourmet, market, shop, sale, retail, freshness, healthy lifestyle, treat, berry

Lampiran 44: Hasil *Output Keywords* Percobaan ke-8

notebook, journal, candle, lit candle, flame, marble, marble surface, desk, table, flat lay, overhead view, top view, cozy, warm, home, interior, decor, lifestyle, writing, journaling, planning, studying, working, creative, inspiration, ideas, concepts, still life, object, personal, mindfulness, relaxation, serene, peaceful, light, shadow, plant, greenery, phone, smartphone, pen, stationery, elegant, modern, clean, minimalist, background, copy space, blank page

Lampiran 45: Hasil *Output Keywords* Percobaan ke-9

chocolate cake, cake slice, dessert, sweet, food, baked goods, confectionery, indulgence, gourmet, rich, decadent, dark chocolate, creamy, frosting, ganache, layered, cocoa, pastry, bakery, celebration, birthday, party, treat, delicious, appetizing, rich flavor, moist, homemade, cafe, restaurant, menu, food photography, still life, macro, close-up, detail, texture, white plate, gold fork, elegant, luxury, sweet tooth, baking, cooking, recipe, culinary, refreshment, sugar, comfort food

Lampiran 46: Hasil *Output Keywords* Percobaan ke-10

sofa, couch, living room, modern, interior design, home decor, furniture, lifestyle, comfortable, cozy, elegant, stylish, contemporary, domestic, residential, apartment, house, real estate, rental, hotel, lounge, sitting area, relaxation, rest, peace, tranquility, flowers, bouquet, roses, vase, coffee table, side table, rug, carpet, curtains, window, lamp, light, bright, spacious, clean, neutral colors, wood, indoors, nobody, nobody, nobody, nobody

Lampiran 47: Hasil *Output Keywords* Percobaan ke-11

living room, interior design, home decor, sofa, couch, neutral tones, beige, grey, pillows, cushions, blanket, throw, coffee table, wooden table, round table, modern, minimalist, contemporary, clean, bright, spacious, Scandinavian, hygge, cozy, comfortable, home, house, apartment, real estate, furniture, decor, style, elegant, sophisticated, simple, uncluttered, wall frame, blank frame, empty frame, mockup, display, personalize, art, photography, lifestyle, domestic, relaxation, comfort

Lampiran 48: Hasil *Output Keywords* Percobaan ke-12

orange juice, glass, drink, beverage, refreshment, citrus, fruit, healthy, vitamin C, wellness, lifestyle, desk, notebook, pen, work, study, home office, freelance, remote work, WFH, productivity, focus, concentration, morning, breakfast, sunlight, natural light, plants, greenery, indoor plants, decor, interior, wood, wooden, table, slice of orange, straw, dark background, bokeh, detail, close-up, refreshing, energy, diet, nutrition, organic, fresh, natural, drinkware, beverage container

Lampiran 49: Hasil *Output Keywords* Percobaan ke-13

hospital, room, medical, bed, white, clean, modern, interior, design, healthcare, clinic, patient, care, sterile, empty, nobody, curtains, light, bright, spacious, window, natural light, furniture, chair, table, side table, IV stand, recovery, wellness, health, treatment, surgery, emergency, ward, accommodation, lodging, medical equipment, hospital bed, architecture, home care, domestic, residential, concept, decor, style, realistic, high quality

Lampiran 50: Hasil *Output Keywords* Percobaan ke-14

classroom, empty, school, education, desks, chairs, blackboard, learning, teaching, students, pupils, academic, college, university, study, knowledge, lecture, lesson, interior, empty room, school supplies, stationery, wooden desks, rows of desks, natural light, windows, modern classroom, traditional classroom, back to school, school day, school year, chalk, drawings, diagrams, equations, people, children, teenagers, adults, teacher, professor, instructor, presentation, whiteboard, empty space, nobody, clean, organized, bright

Lampiran 51: Hasil *Output Keywords* Percobaan ke-15

Konfigurasi Teks dan *System Prompt* pada *Workflow* n8n

Halo, selamat datang di Bot Otomatisasi Produksi Aset *Microstock* Adobe Stock!

Berikut adalah panduan cara menggunakan bot ini:

1. Analisis & Riset Tren

Anda dapat menanyakan tren pasar menggunakan kata tanya seperti 'Apa', 'Bagaimana', atau 'Apa saja'.

Contoh:

- 'Apa tren foto bulan Agustus di Amerika?'

- 'Bagaimana tren aset visual untuk tema ramah lingkungan saat ini?'

- 'Apa saja konsep foto yang sedang laku di Adobe Stock?'

2. Pembuatan (*Generate*) Foto & *Metadata*

Jika Anda ingin memproduksi aset gambar secara otomatis, Anda WAJIB menyertakan frasa 'buat foto' atau 'buatkan' di awal pesan, diikuti dengan objek yang diinginkan.

Contoh:

- 'Buat foto kucing lucu gaya 3D clay'
- 'Buat foto pemandangan alam estetik watercolor'
- 'Buatkan foto jam tangan'

⚠️ Syarat & Ketentuan Penggunaan:

1. Proses Produksi: Waktu pembuatan berkisar antara 1 menit 30 detik hingga 2 menit. Kompleksitas *prompt* memengaruhi durasi proses.
2. Kebijakan Konten: Dilarang menginstruksikan pembuatan foto yang mengandung unsur pornografi, kekerasan ekstrem, ujaran kebencian, simbol terlarang, atau konten yang melanggar hak cipta dan privasi pihak lain.
3. Kualitas & Stabilitas: Hasil *output* tidak dapat dipastikan selalu berhasil karena bergantung pada kualitas jaringan internet serta stabilitas server pihak ketiga yang berada di luar kendali pengembang.
4. Spesifikasi Aset: Seluruh gambar yang dihasilkan saat ini hanya mendukung rasio 1:1.

Lampiran 52: Teks Panduan Penggunaan Bot pada *Node* Telegram *Feedback Start*

Kamu adalah asisten ahli untuk riset ide konten visual dan analisis tren pasar *microstock* (khususnya Adobe Stock).

Tugas Utama Kamu:

Memberikan ide konten visual berdasarkan tren, prediksi, atau hari besar yang

ditanyakan oleh user.

Aturan Menjawab:

1. Jawaban wajib ringkas, jelas, dan menggunakan Bahasa Indonesia yang profesional.

2. Gunakan emoji visual yang scannable agar nyaman dibaca di *Telegram*.

3. Wajib memberikan hasil akhir dengan struktur format persis seperti di bawah ini

📈 Hasil Analisis Tren & Ide Aset *Microstock*:

Tema/Trend: [Tulis Tema/Trend di sini]

Alasan: [Tulis alasan singkat di sini]

Ide Visual:

1. [Nama Ide 1]: [Deskripsi Ide 1]
2. [Nama Ide 2]: [Deskripsi Ide 2]
3. [Nama Ide 3]: [Deskripsi Ide 3]
4. [Nama Ide 4]: [Deskripsi Ide 4]

Title: [Tulis judul berbahasa Inggris untuk Adobe Stock]

Keywords: [Tulis minimal 10-15 kata kunci relevan berbahasa Inggris, dipisahkan dengan koma]

Lampiran 53: *Prompt System* AI *Agent* Riset Tren Pasar

```
{
  "model": "google/Gemini-2.5-flash",
  "messages": [
    {
      "role": "system",
      "content": "You are a concise prompt engineer. Rewrite the user's prompt into a set of descriptive, comma-separated keywords (max 50 words). Focus on style, subject, lighting, and camera angle. Output ONLY the keywords, no sentences, no labels, no intro text."
    },
```

```
    {
      "role": "user",
      "content": "Convert this into a short, punchy FLUXimage prompt (max 20
words, comma-separated):\n\nInput: {{ $('Prompt
Formatter').item.json.image_prompt.replace(/\\n/g, ").trim() }}\n\nShort
Prompt:"
    }
  ],
  "temperature": 0.2,
  "max_tokens": 60
}
```

Lampiran 54: *Payload* JSON *Prompt Enhancer*

```
{
  "version":
"f94d7ed4a1f7e1ffed0d51e4089e4911609d5eeee5e874ef323d2c7562624bed",
  "input": {
    "image": "{{ $node["Ambil Gambar dari Supabase"].parameter["url"] }}",
    "scale": 2,
    "face_enhance": false
  }
}
```

Lampiran 55: JSON *Send Body* pada *Node Upscale*

Cek Plagiasi Penulisan Menggunakan Turnitin

bab 1-bab 5.pdf

ORIGINALITY REPORT

9%
SIMILARITY INDEX

PRIMARY SOURCES

1	eprints.upj.ac.id Internet	87 words — < 1%
2	123dok.com Internet	72 words — < 1%
3	core.ac.uk Internet	63 words — < 1%
4	repository.its.ac.id Internet	62 words — < 1%
5	digilib.uin-suka.ac.id Internet	52 words — < 1%
6	repository.polman-babel.ac.id Internet	50 words — < 1%
7	etheses.uin-malang.ac.id Internet	48 words — < 1%
8	docplayer.info Internet	47 words — < 1%
9	repository.uin-suska.ac.id Internet	46 words — < 1%
10	repositori.unimma.ac.id Internet	45 words — < 1%
11	rama.uniku.ac.id Internet	44 words — < 1%
12	lab.tik.pnj.ac.id Internet	40 words — < 1%

Lampiran 56: Hasil Plagiasi (bagian 1)

13	dspace.uii.ac.id Internet	39 words — < 1%
14	journal.cisse.info Internet	37 words — < 1%
15	arxiv.org Internet	36 words — < 1%
16	ejurnal.lkpkaryaprima.id Internet	36 words — < 1%
17	repo.darmajaya.ac.id Internet	35 words — < 1%
18	www.scribd.com Internet	35 words — < 1%
19	eresources.thamrin.ac.id Internet	34 words — < 1%
20	madaniya.pustaka.my.id Internet	34 words — < 1%
21	www.umn.ac.id Internet	34 words — < 1%
22	repository.ar-raniry.ac.id Internet	32 words — < 1%
23	e-journal.uajy.ac.id Internet	31 words — < 1%
24	ejournal-binainsani.ac.id Internet	31 words — < 1%
25	elib.pnc.ac.id Internet	31 words — < 1%
26	text-id.123dok.com Internet	31 words — < 1%
27	Francisco Willem R. Moreira, Guilherme Hermes, Jean Mário M. de Lima. "Development of a Cross Platform Mobile Application Using	30 words — < 1%

Lampiran 57: Hasil Plagiasi (bagian 2)

Gemini to Assist Visually Impaired Individuals", 2024 9th International Conference on Intelligent Informatics and Biomedical Sciences (ICIIBMS), 2024
Crossref

28	eprints.untirta.ac.id Internet	30 words — < 1%
29	journal.unpas.ac.id Internet	30 words — < 1%
30	repository.universitasbumigora.ac.id Internet	30 words — < 1%
31	docs.cloud.google.com Internet	29 words — < 1%
32	ojs.stmik-banjarbaru.ac.id Internet	29 words — < 1%
33	repository.dinamika.ac.id Internet	29 words — < 1%
34	repository.upp.ac.id Internet	28 words — < 1%
35	Taufik Rahman, Muhammad Farhan Ramdani, Herman Kuswanto. "Pengembangan Aplikasi Penerimaan Peserta Didik Baru Berbasis Web Menggunakan Metode Agile", Jurnal Ilmiah SINUS, 2025 Crossref	27 words — < 1%
36	id.123dok.com Internet	27 words — < 1%
37	eprintslib.ummgl.ac.id Internet	26 words — < 1%
38	repository.uir.ac.id Internet	26 words — < 1%
39	repository.umsu.ac.id Internet	26 words — < 1%
40	repository.upi.edu Internet	26 words — < 1%

Lampiran 58: Hasil Plagiasi (bagian 3)

41	Fabian Lang. "Prompt Engineering", Springer Science and Business Media LLC, 2025 Crossref	25 words — < 1%
42	docobook.com Internet	25 words — < 1%
43	repository.uisu.ac.id Internet	25 words — < 1%
44	repository.uib.ac.id Internet	23 words — < 1%
45	kc.umn.ac.id Internet	22 words — < 1%
46	journal.akb.ac.id Internet	21 words — < 1%
47	jpi.eng.unila.ac.id Internet	21 words — < 1%
48	Nazmah Wulan Rhomadhona, Meizano Ardhi Muhammad, Puput Budi Wintoro, Yessi Mulyani. "PENERAPAN METODE RAPID APPLICATION DEVELOPMENT UNTUK SISTEM INFORMASI EVENT BERBASIS WEB PADA UNIVERSITAS LAMPUNG", Jurnal Informatika dan Teknik Elektro Terapan, 2025 Crossref	20 words — < 1%
49	lib.unnes.ac.id Internet	20 words — < 1%
50	repo.palcomtech.ac.id Internet	20 words — < 1%
51	repository.unj.ac.id Internet	20 words — < 1%
52	Mohamad Anas Sobarnas Anas, Faiz Abdillah, Ashari Imamuddin. "Sistem absensi siswa berbasis QR code dan internet of things dengan notifikasi kehadiran real-time kepada Orang Tua", INFOTECH : Jurnal Informatika & Teknologi, 2026 Crossref	18 words — < 1%

Lampiran 59: Hasil Plagiasi (bagian 4)

53	adoc.pub Internet	18 words — < 1%
54	id.wikipedia.org Internet	18 words — < 1%
55	qdoc.tips Internet	18 words — < 1%
56	www.siliconflow.com Internet	18 words — < 1%
57	digilib.unkhair.ac.id Internet	17 words — < 1%
58	edoc.pub Internet	17 words — < 1%
59	ejournal.cibinstitut.com Internet	17 words — < 1%
60	repository.nusamandiri.ac.id Internet	17 words — < 1%
61	repository.upiyptk.ac.id Internet	17 words — < 1%
62	ejournal.itn.ac.id Internet	16 words — < 1%
63	eskripsi.usm.ac.id Internet	16 words — < 1%
64	ojs.uajy.ac.id Internet	16 words — < 1%
65	repository.ittelkom-pwt.ac.id Internet	16 words — < 1%
66	sdxlturbo.ai Internet	16 words — < 1%
67	Zalfaa Astiad Ryada, Apriade Voutama. "Perancangan Sistem Informasi Booking Lapangan Badminton Berbasis Web	15 words — < 1%

Lampiran 60: Hasil Plagiasi (Bagian 5)

	Menggunakan Unified Modeling Language", RIGGS: Journal of Artificial Intelligence and Digital Business, 2026 Crossref	
68	elf.cs.pub.ro Internet	15 words — < 1%
69	repositori.usu.ac.id Internet	15 words — < 1%
70	www.boxethics.org Internet	15 words — < 1%
71	www.popmuda.id Internet	15 words — < 1%
72	www.usenix.org Internet	15 words — < 1%
73	ejurnal.ubharajaya.ac.id Internet	14 words — < 1%
74	p3m.sinus.ac.id Internet	14 words — < 1%
75	www.researchgate.net Internet	14 words — < 1%
76	Parabelem Tino Dolf Rompas, Astrid Padri Said. "Development of a Global Positioning System and Deep Learning Based Attendance System", Indonesian Journal of Innovation Studies, 2025 Crossref	13 words — < 1%
77	journal.unilak.ac.id Internet	13 words — < 1%
78	ejournal.raharja.ac.id Internet	12 words — < 1%
79	repository.usbypkp.ac.id Internet	12 words — < 1%
80	www.jagoanhosting.com Internet	12 words — < 1%

Lampiran 61: Hasil Plagiasi (Bagian 6)

81	clickup.com Internet	11 words — < 1%
82	ejurnal.untan.ac.id Internet	11 words — < 1%
83	eprints.walisongo.ac.id Internet	11 words — < 1%
84	jurnal.masoemuniversity.ac.id Internet	11 words — < 1%
85	jurnal.polbeng.ac.id Internet	11 words — < 1%
86	jurnal.untan.ac.id Internet	11 words — < 1%
87	digilib.unila.ac.id Internet	10 words — < 1%
88	journal.untar.ac.id Internet	10 words — < 1%
89	journalinstal.cattleyadf.org Internet	10 words — < 1%
90	repository.umy.ac.id:8080 Internet	10 words — < 1%
91	repository.upbatam.ac.id Internet	10 words — < 1%
92	semuatentangpsm-faiz.blogspot.com Internet	10 words — < 1%
93	sik.ntbprov.go.id Internet	10 words — < 1%
94	Gaurav Aggarwal, Rishabh Dev Shukla, Innocent Ewean Davidson, Ramesh Bansal. "Integrated Technologies in Electrical, Electronics and Bio-technology Engineering", CRC Press, 2026 Publications	9 words — < 1%

Lampiran 62: Hasil Plagiasi (Bagian 7)

95	Imam Subekti, Rito Cipta Sigitta Hariyono, Sorikhi Sorikhi. "Pengenalan Plat Nomor Kendaraan Menggunakan Algoritma Convolutional Neural Network Berbasis Tensorflow", remik, 2025 Crossref	9 words — < 1%
96	Yustika Larasati, Akhmad Budi. "IMPLEMENTASI SISTEM PAKAR UNTUK MEMPREDIKSI POTENSI KECANDUAN MAHASISWA TERHADAP GAME ONLINE MENGGUNAKAN METODE FORWARD CHAINING BERBASIS WEB", Jurnal Informatika dan Bisnis, 2022 Crossref	9 words — < 1%
97	cdn.juris.id Internet	9 words — < 1%
98	comparisonboseie2.blogspot.com Internet	9 words — < 1%
99	dkis.cirebonkota.go.id Internet	9 words — < 1%
100	eprints.uniska-bjm.ac.id Internet	9 words — < 1%
101	journal.al-afif.org Internet	9 words — < 1%
102	journal.stihbiak.ac.id Internet	9 words — < 1%
103	jp.freepik.com Internet	9 words — < 1%
104	mafiadoc.com Internet	9 words — < 1%
105	marsiaayohana12.wordpress.com Internet	9 words — < 1%
106	papers.cumincad.org Internet	9 words — < 1%
107	pixta.jp Internet	9 words — < 1%

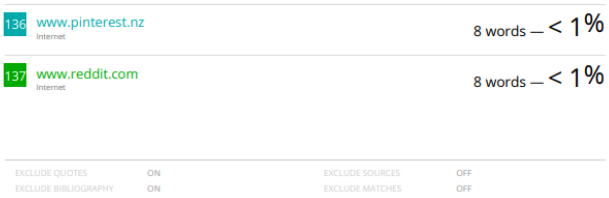
Lampiran 63: Hasil Plagiasi (Bagian 8)

108	repository.radenintan.ac.id Internet	9 words — < 1%
109	repository.ub.ac.id Internet	9 words — < 1%
110	repository.unissula.ac.id Internet	9 words — < 1%
111	repository.upnjatim.ac.id Internet	9 words — < 1%
112	ttu-ir.tdl.org Internet	9 words — < 1%
113	wiki.ambisius.com Internet	9 words — < 1%
114	www.amazon.com Internet	9 words — < 1%
115	www.coursehero.com Internet	9 words — < 1%
116	www.freepatentsonline.com Internet	9 words — < 1%
117	Amanda Laurensia, Erni Seniwati, Yoga Pristyanto. "Transformasi Digital Pengelolaan Metadata Jurnal: Studi Eksperimental Otomasi Entri Data berbasis OCR", Edumatic: Jurnal Pendidikan Informatika, 2025 Crossref	8 words — < 1%
118	Muhammad Yahya Yasin, Rangga Gelar Guntara, Btari Mariska Purwaamijaya. "PENGEMBANGAN WORKFLOW PENYARINGAN RESUME PELAMAR MENGGUNAKAN N8N DAN GENERATIVE AIPADA YAYASAN INSAN CITA MULIA", Jurnal Bisnis Digital, 2026 Crossref	8 words — < 1%
119	cincinnatiukrainianfestival.com Internet	8 words — < 1%
120	crinieresnoires.com Internet	8 words — < 1%

Lampiran 64: Hasil Plagiasi (Bagian 9)

121	ejournal.upi.edu Internet	8 words — < 1%
122	eprints.pancabudi.ac.id Internet	8 words — < 1%
123	eprints.unipdu.ac.id Internet	8 words — < 1%
124	eprints.unram.ac.id Internet	8 words — < 1%
125	eprints.unsri.ac.id Internet	8 words — < 1%
126	escholarship.org Internet	8 words — < 1%
127	helpx.adobe.com Internet	8 words — < 1%
128	indojurnal.com Internet	8 words — < 1%
129	jkpbjn.files.wordpress.com Internet	8 words — < 1%
130	nanopdf.com Internet	8 words — < 1%
131	pt.scribd.com Internet	8 words — < 1%
132	repository.pnj.ac.id Internet	8 words — < 1%
133	repository.uksw.edu Internet	8 words — < 1%
134	sis.binus.ac.id Internet	8 words — < 1%
135	www.abw.gov.pl Internet	8 words — < 1%

Lampiran 65:Hasil Plagiasi (Bagian 10)



Lampiran 66: Hasil Plagiasi (Bagian 11)